

Opus Publica

RC1 Constitutional Traceability Matrix

Every Requirement Mapped to Implementation and Evidence

Version 1.0

Issued 09 August 2026

Document Type	Traceability Matrix (Audit Reference)
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	Engineering Leadership, Release Certification Authority, External Auditors
Classification	Mandatory — Audit Authority
Governed By	Engineering Constitution Vol I (Ch 1-25) + Vol II (Ch 26-36) + Playbook (Ch 1-20)
Operationalizes	Volume II, Chapter 23 (Knowledge Graph Constitution)
Effective Date	09 August 2026
Supersedes	None (first issuance)
Review Cadence	On every Constitution amendment; quarterly integrity check
Document ID	OP-TRACE-RC1-001

This Matrix is the audit spine of Opus Publica. Every constitutional requirement SHALL be traceable from specification through implementation to runtime evidence. An untraceable requirement is, for audit purposes, an unmet requirement.

Chapter 1 — Why Traceability Is the Audit Spine

1.1 The Question Every Auditor Asks

An auditor — internal or external, friendly or hostile — arrives at Opus Publica with one question: “Prove it.” Prove that every published article has a DOI. Prove that every published artifact is immutable. Prove that every state transition is audited. Prove that every Crossref deposit is reconciled. Prove that the 38 engineering invariants hold, not in theory, but in the running system, at this moment, with evidence that can be inspected. The Constitution specifies these requirements across 572 pages and 36 chapters. The Playbook operationalizes them across 196 pages and 20 chapters. But neither document, on its own, answers the auditor's question. The auditor does not want to know what the Constitution says; the auditor wants to know that what the Constitution says is true of the system, and the auditor wants the evidence. This Traceability Matrix is the document that answers “prove it.”

The thesis of this Matrix is that traceability is the spine of auditability. A requirement that cannot be traced from its constitutional statement through its architectural realization, its code implementation, its test coverage, and its runtime evidence is a requirement whose satisfaction is unverifiable — and an unverifiable requirement is, for engineering and audit purposes, an unmet requirement, regardless of whether the implementation happens to satisfy it. The Matrix exists to ensure that no constitutional requirement of Opus Publica is unverifiable. Every requirement has a traceable path from specification to evidence; every path is documented in this Matrix; every path is maintained as the system evolves.

The Matrix's Prime Directive

Every constitutional requirement of Opus Publica — every invariant, every rule, every contract, every threshold, every SLO — SHALL be traceable through the full chain from specification to runtime evidence. A requirement with a broken traceability chain is a defect. A requirement with no traceability chain is a critical defect. The Release Certification Authority SHALL NOT certify a release in which any traced requirement has a broken chain, and SHALL NOT certify a release in which any constitutional requirement is untraced.

1.2 Background: The Knowledge Graph Constitution

The Traceability Matrix operationalizes Volume II, Chapter 23 of the Engineering Constitution — the Knowledge Graph Constitution. Chapter 23 establishes the traceability graph as a first-class artifact of the platform: a directed graph whose nodes are engineering artifacts (requirements, architecture components, ADRs, database tables, API endpoints, code modules, storage objects, workflows, UI components, tests, runtime evidence, certification records, release records) and whose edges are traceability relationships (implements, verifies, produces, certifies, releases). The graph is the machine-readable form of traceability; this Matrix is the human-readable form.

The traceability chain established by Chapter 23 is the following 13-node sequence:

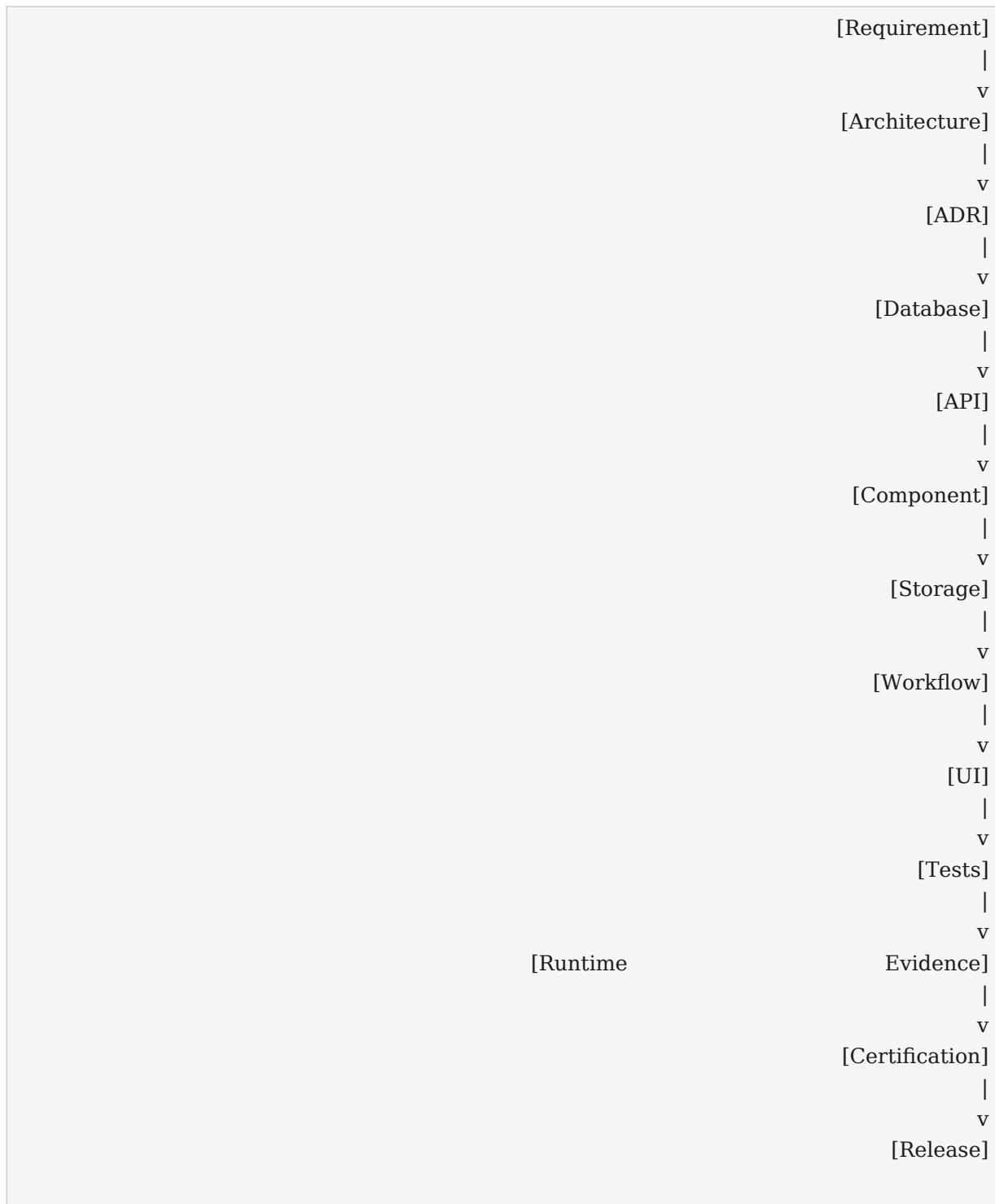


Figure 1.1: The 13-Node Traceability Chain (per Volume II, Chapter 23)

Each arrow in the chain represents a traceability relationship: a requirement is realized by an architecture component, which is justified by an ADR, which is backed by a database table, which is exposed by an API, which is implemented by a code component, which stores its artifacts in storage, which is exercised by a workflow, which is surfaced through a UI, which is verified by tests, which produce runtime evidence, which is certified, which is released. A break at any link breaks the chain. A

requirement with a broken chain is unverifiable. This Matrix documents the chain for every constitutional requirement.

1.3 The Scope of This Matrix

This Matrix traces every class of constitutional requirement established by the Engineering Constitution (Volume I and Volume II) and operationalized by the Engineering Playbook. The requirement classes are the following:

Requirement Class	Source	Count	Matrix Chapter
Engineering Invariants	Vol II Ch 26	38	Chapter 4
Non-Negotiable Rules	Vol I Ch 3	~40	Chapter 5
System Contracts	Vol II Ch 27	20	Chapter 6
Canonical Fields (key subset)	Vol II Ch 28	276 (50 traced in detail)	Chapter 7
Events	Vol II Ch 29	34	Chapter 8
SLOs and SLIs	Vol II Ch 32	10 SLOs / 10 SLIs	Chapter 9
Constitutional Thresholds	Vol II Ch 31	103 (key subset traced)	Chapter 10
Volume I Chapter Requirements	Vol I Ch 1-25	25 chapters	Chapter 11
Volume II Chapter Requirements	Vol II Ch 26-36	11 chapters	Chapter 12

The total traced requirement count exceeds five hundred. The Matrix does not trace every sub-requirement of every chapter subsection (that would produce a document longer than the Constitution itself); it traces every requirement class and every requirement within each class that is independently verifiable. Sub-requirements that are aspects of a traced parent requirement are traced through the parent. This scoping decision is documented in Chapter 15 (Limitations).

1.4 Historical Context: Why Traceability Matters in Scholarly Publishing

Scholarly publishing is an unusually audit-sensitive domain. The published record is the foundation of academic citation, scientific reproducibility, funder compliance, and tenure decisions. A platform that corrupts the scholarly record — by losing an artifact, by assigning a duplicate DOI, by silently modifying a published PDF — corrupts the citations that depend on it, the research that builds on it, and the careers that cite it. The scholarly community has, over decades, developed infrastructure (Crossref, ORCID, CLOCKSS, LOCKSS, Portico, OAI-PMH, DOAJ, OpenAIRE) whose entire purpose is to make the scholarly record traceable, persistent, and verifiable. A scholarly publishing platform that does not internalize this

traceability discipline is a platform that will, eventually and inevitably, corrupt the record it exists to serve.

The incumbent platform, Open Journal Systems (OJS), has served the community for over two decades and hosts approximately ten thousand journals. OJS's traceability model is, however, largely implicit: metadata is mastered in multiple subsystems, the plugin hook system couples extension authors to internal call sites, and the absence of a formal traceability graph means that an OJS deployment's compliance with any given requirement is typically verified by manual inspection rather than by systematic trace. Opus Publica is designed as a successor that internalizes traceability as a first-class engineering discipline. This Matrix is the operational expression of that discipline: it is the document that proves, requirement by requirement, that the platform's implementation matches its specification, and that the match is evidenced at runtime.

1.5 Definitions

The following terms are used throughout this Matrix with the precise meanings given here. These definitions extend those in the Constitution's glossary (Volume II, Appendix B) and the Playbook's glossary (Appendix A).

Term	Definition
Requirement	A constitutional statement that obliges the platform to behave in a specified way. Requirements are identified by stable IDs (e.g., INV-I-01, PUB-001, ARCH-003).
Traceability Chain	The 13-node sequence from Requirement to Release (Figure 1.1) that documents the path from specification to evidence.
Traceability Link	A single edge in the traceability graph, connecting two nodes (e.g., Requirement → Architecture).
Traceability Gap	A missing link in a traceability chain. A gap means the requirement's path to evidence is broken.
Orphan Node	A node in the traceability graph with no incoming or outgoing edges. An orphan requirement is a requirement no architecture realizes; an orphan test is a test no requirement mandates.
Runtime Evidence	An artifact produced by the running system that demonstrates a requirement is satisfied (logs, monitor outputs, screenshots, hashes, audit records).
Certification Artifact	A runtime evidence artifact that has been reviewed and signed off by the Release Certification Authority.
Audit Trail	The complete set of traceability chains for a given

	requirement class, sufficient for an external auditor to verify compliance.
--	---

Chapter 2 — The Traceability Framework

2.1 Purpose

This chapter specifies the framework within which the traceability matrices in Chapters 4 through 12 are constructed and maintained. The framework defines the node types, the edge types, the integrity rules, and the verification regime that govern the traceability graph. The framework is the operational form of Volume II, Chapter 23 (Knowledge Graph Constitution); this chapter restates the framework in the context of this Matrix and extends it with the audit-specific requirements that an external auditor would impose.

2.2 Node Types

The traceability graph has thirteen node types, corresponding to the thirteen stages of the traceability chain. Each node type has a stable identifier format, an owning Constitution chapter, and a set of mandatory attributes that every node of that type SHALL carry.

#	Node Type	ID Format	Owning Chapter	Mandatory Attributes
N1	Requirement	REQ-<class>-<nnn>	Vol I/II various	ID, statement, source chapter, owner, severity
N2	Architecture Component	ARCH-<subsystem>	Vol I Ch 4	ID, name, owning team, interfaces, dependencies
N3	ADR	ADR-<nnn>	Vol I Ch 7	ID, title, status, decision, trade-offs, owner
N4	Database Table	DB-<table>	Vol I Ch 5, Vol II Ch 33	Table name, owning service, schema, indexes
N5	API Endpoint	API-<method>-<path>	Vol I Ch 9	Method, path, auth, idempotency, contract, owner
N6	Code Component	CODE-<module>	Vol I Ch 8	Module path, owning team, language, dependencies
N7	Storage Object	STORE-<bucket>/<key>	Vol I Ch 12, Vol II Ch 12	Bucket, key, SHA-256, retention, owner
N8	Workflow	WF-<name>	Vol I Ch 10, Vol II Ch	Name, states,

			11	transitions, owner
N9	UI Component	UI-<component>	Vol II Ch 30	Component name, page, accessibility, owner
N10	Test	TEST-<type>-<nnn>	Vol I Ch 19, Vol II Ch 19	ID, type, target, coverage, owner
N11	Runtime Evidence	EVID-<nnn>	Vol I Ch 8, Vol II Ch 20	ID, type, timestamp, source, hash
N12	Certification Record	CERT-<nnn>	Vol I Ch 8, Vol II Ch 22	ID, feature, pillars, RCA, date
N13	Release Record	REL-<nnn>	Vol I Ch 5.07, Playbook Ch 11	ID, version, artifacts, date, status

2.3 Edge Types

Edges in the traceability graph represent the relationships between nodes. Each edge type has a defined semantic, a defined direction, and a defined cardinality. The edge types are the vocabulary of traceability; an engineer reading the graph SHALL be able to traverse from any node to any related node by following edges of these types.

Edge Type	From → To	Semantics	Cardinality
realizes	Requirement → Architecture	The architecture component realizes the requirement.	Many-to-many (a requirement may be realized by multiple components; a component may realize multiple requirements).
justified-by	Architecture → ADR	The architecture decision is justified by the ADR.	Many-to-one (a component may be justified by one ADR; an ADR may justify multiple components).
backed-by	Architecture → Database	The architecture is backed by the database table.	Many-to-many.
exposed-by	Architecture → API	The architecture is exposed by the API endpoint.	Many-to-many.
implemented-by	Architecture → Code	The architecture is implemented by the code component.	Many-to-many.
stored-in	Code → Storage	The code stores artifacts in	Many-to-many.

		the storage object.	
exercised-by	Workflow → Code	The workflow exercises the code component.	Many-to-many.
surfaced-by	Workflow → UI	The workflow is surfaced by the UI component.	Many-to-one.
verified-by	Requirement → Test	The requirement is verified by the test.	Many-to-many (a requirement may be verified by multiple tests; a test may verify multiple requirements).
produces	Test → Runtime Evidence	The test produces the runtime evidence.	One-to-many (a test may produce multiple evidence artifacts).
certified-by	Runtime Evidence → Certification	The evidence is certified by the certification record.	Many-to-one (multiple evidence artifacts are bundled into one certification record).
released-in	Certification → Release	The certification authorizes the release.	One-to-one (a certification record authorizes exactly one release).

2.4 The Matrix Schema

Each matrix in Chapters 4 through 12 is a table whose rows are requirements and whose columns are the traceability chain nodes. The schema is uniform across all matrices so that an auditor can read any matrix with the same mental model. The columns are:

Column	Content	Source
Req ID	The stable identifier of the requirement.	Constitution
Requirement	The one-line statement of the requirement.	Constitution
Source	The Constitution chapter and rule/invariant ID.	Constitution
Architecture	The architecture component(s) that realize the requirement.	Vol I Ch 4
ADR	The ADR(s) that justify the architecture decision.	Vol I Ch 7
Database	The database table(s) that back the	Vol I Ch 5

	requirement.	
API	The API endpoint(s) that expose the requirement.	Vol I Ch 9
Code	The code module(s) that implement the requirement.	Vol I Ch 8
Storage	The storage object(s) that hold the requirement's artifacts.	Vol I Ch 12
Test	The test(s) that verify the requirement.	Vol I Ch 19
Evidence	The runtime evidence artifact(s) that demonstrate satisfaction.	Vol I Ch 8
Cert.	The certification record(s) that certified the evidence.	Vol I Ch 8
Owner	The team accountable for the requirement's traceability.	Playbook Ch 2
Status	Traced / Gap / Untraced.	This Matrix

The Status column is the audit-critical column. “Traced” means the full 13-node chain is documented and the runtime evidence is current. “Gap” means one or more links in the chain are missing; the gap SHALL be remediated. “Untraced” means the requirement has not yet been entered into the Matrix; this is a critical defect for any requirement above SEV-3. The Release Certification Authority SHALL NOT certify a release with any “Untraced” requirement above SEV-3, and SHALL NOT certify a release with any “Gap” in a SEV-1 requirement.

2.5 Integrity Rules

The traceability graph SHALL satisfy the following integrity rules. These rules are the operational form of Volume II, Chapter 23, section 23.7 (Graph Integrity Rules), restated here in the context of this Matrix.

1. **Rule IR-01.** Every requirement SHALL trace to at least one architecture component. A requirement with no architecture realization is an orphan requirement.
2. **Rule IR-02.** Every architecture component SHALL trace to at least one ADR. A component with no ADR is an undocumented design decision.
3. **Rule IR-03.** Every API endpoint SHALL trace to at least one test. An API with no test is an unverified contract.
4. **Rule IR-04.** Every test SHALL trace to at least one runtime evidence artifact in the most recent release. A test that produced no evidence is a test that did not run.
5. **Rule IR-05.** Every SEV-1 requirement SHALL have a complete traceability chain with no gaps. A SEV-1 requirement with a gap is a critical defect.

6. **Rule IR-06.** Every runtime evidence artifact SHALL be no older than the maximum evidence age (defined in Chapter 13). Stale evidence is treated as missing evidence.
7. **Rule IR-07.** No node SHALL be an orphan (no incoming or outgoing edges). Orphan nodes are defects and SHALL be remediated or removed.
8. **Rule IR-08.** The traceability graph SHALL be reconciled against the Constitution on every Constitution amendment. Requirements added by the amendment SHALL be traced before the next release; requirements removed by the amendment SHALL have their traces archived.

2.6 Verification Regime

The traceability graph is verified by three mechanisms: static verification (the Matrix is complete and internally consistent), dynamic verification (the runtime evidence referenced by the Matrix is current and accessible), and audit verification (an external auditor can traverse the chain from any requirement to its evidence).

Verification Type	What It Checks	Cadence	Owner
Static	Matrix completeness; no orphan nodes; no broken links; all IDs resolve.	On every Matrix update; nightly automated scan.	Engineering Leadership
Dynamic	Runtime evidence is current (within max age); evidence artifacts are retrievable; monitor outputs are live.	Weekly; before every release.	Release Certification Authority
Audit	An external auditor can traverse from any sampled requirement to its evidence without assistance; the chain is self-explanatory.	Quarterly; before every RC milestone.	Chief Systems Architect

Chapter 3 — How to Use This Matrix

3.1 Purpose

This chapter instructs the reader — whether an engineer, a reviewer, a certifier, an on-call operator, or an external auditor — on how to use the Traceability Matrix. The Matrix is a reference document, not a narrative; it is consulted at specific moments for specific purposes, and this chapter enumerates those moments and purposes.

3.2 Audience and Use Cases

Audience	Use Case	Relevant Chapters
Engineer (implementing a feature)	Identify the requirement the feature satisfies; trace the requirement to its architecture, API, and tests to understand the contract; ensure the implementation completes the chain.	Ch 4-12 (the requirement's matrix); Ch 2 (framework)
Reviewer (reviewing a PR)	Verify the PR cites the requirement ID; verify the implementation completes the traceability chain; flag gaps in the Review Checklist.	Ch 4-12; Playbook Ch 5, Ch 19
QA Engineer (preparing certification evidence)	Identify the tests that verify the requirement; produce the runtime evidence the chain expects; store the evidence with the correct EVID ID.	Ch 4-12 (Test and Evidence columns); Playbook Ch 10
Release Certification Authority	Verify every SEV-1 requirement has a complete, current chain; verify no gaps in SEV-1/SEV-2; sign off or reject.	All matrices; Ch 13 (integrity); Ch 14 (audit)
On-Call Engineer (responding to an incident)	Identify the invariant violated; trace the invariant to its monitor, runbook, and evidence; execute the runbook.	Ch 4 (invariants); Playbook Ch 8, Ch 12
External Auditor	Sample requirements; traverse the chain from requirement to evidence; verify the evidence is current and accessible; report gaps.	Ch 2 (framework); Ch 14 (audit procedure); sampled matrices

Chief Systems Architect	Verify constitutional completeness; identify untraced requirements; govern amendments' traceability impact.	All matrices; Ch 13; Ch 15 (limitations)
-------------------------	---	--

3.3 How to Read a Matrix Row

Each row in a traceability matrix represents one requirement and its full traceability chain. The reader reads the row left to right, following the chain from the requirement through its architecture, ADR, database, API, code, storage, tests, evidence, and certification. A cell containing “—” indicates that the link is not applicable to this requirement (for example, a pure-data invariant may have no API endpoint). A cell containing “GAP” indicates that the link is applicable but missing; the gap SHALL be remediated. A cell containing “STALE” indicates that the link exists but the evidence is older than the maximum age; the evidence SHALL be refreshed.

The Status column at the end of each row summarizes the chain's health: “Traced” (all applicable links present and current), “Gap” (one or more links missing), or “Untraced” (the requirement has not been entered). An auditor sampling the Matrix SHALL focus on rows with Status “Gap” or “Untraced”, as these represent the platform's traceability debt.

3.4 How to Trace a Requirement

To trace a specific requirement, the reader follows this procedure:

Step 1. Locate the requirement's ID. If the ID is known (e.g., INV-I-01), proceed to step 2. If the ID is unknown, identify the requirement class (invariant, rule, contract, etc.) and consult the relevant matrix chapter (Chapters 4-12).

Step 2. Find the row in the relevant matrix. Each matrix is sorted by requirement ID. Locate the row matching the ID.

Step 3. Read the row left to right. Follow the chain from the Requirement through Architecture, ADR, Database, API, Code, Storage, Test, Evidence, Certification.

Step 4. Verify the runtime evidence. The Evidence cell references an EVID ID. Retrieve the evidence artifact from the certification repository. Verify the artifact exists, is within the maximum evidence age, and demonstrates the requirement's satisfaction.

Step 5. Verify the chain's integrity. Confirm that every applicable cell is populated (not “—”, not “GAP”, not “STALE”). A single gap breaks the chain.

Step 6. Record the trace. If tracing for audit purposes, record the trace in the audit log: requirement ID, chain nodes traversed, evidence verified, result.

3.5 How to Remediate a Gap

When a gap is identified (a cell contains “GAP” or the Status is “Gap”), the following procedure SHALL be followed to remediate it:

Step 1. Identify the missing link. Determine which cell in the chain is empty. The missing link identifies the work needed: a missing architecture component, a missing ADR, a missing test, etc.

Step 2. Create a remediation task. Open a task ticket citing the requirement ID, the missing link, and the remediation needed. Assign the task to the team that owns the requirement (per the Owner column).

Step 3. Execute the remediation. Produce the missing artifact: write the ADR, add the test, implement the monitor, etc. The remediation SHALL follow the Implementation Lifecycle (Playbook Chapter 2).

Step 4. Update the Matrix. Once the artifact is produced, update the Matrix cell with the artifact's ID. Change the Status from “Gap” to “Traced” (if all gaps are remediated) or leave as “Gap” (if gaps remain).

Step 5. Verify the remediation. The Release Certification Authority SHALL verify the remediation at the next certification cycle. A remediation that produces a new gap elsewhere in the chain is not a remediation.

3.6 Matrix Maintenance

The Matrix is a living document. It SHALL be updated on the following events:

- **Constitution amendment.** when a new requirement is added to the Constitution (via amendment per Volume II Chapter 36), a new row SHALL be added to the relevant matrix within 5 business days, and the requirement SHALL be fully traced before the next release.
- **Implementation change.** when an architecture component, ADR, API, test, or evidence artifact is added or changed, the affected matrix rows SHALL be updated to reflect the new artifact.
- **Release certification.** when a release is certified, the Certification column SHALL be updated with the certification record ID, and the Evidence column SHALL be updated with the release's evidence artifact IDs.
- **Invariant violation.** when an invariant monitor detects a violation, the Evidence column for the affected invariant SHALL be marked “STALE” or “GAP” (as appropriate) until the violation is remediated and fresh evidence is produced.
- **Quarterly integrity check.** the Matrix SHALL undergo a quarterly integrity check (Rule IR-08) in which every row is verified against the current Constitution and the current implementation. Discrepancies SHALL be remediated within 30 days.

Chapter 4 — Engineering Invariants Traceability

4.1 Purpose

This chapter traces every engineering invariant established by Volume II, Chapter 26 of the Engineering Constitution through the full 13-node traceability chain defined in Chapter 2 of this Matrix. The invariants are the platform's non-negotiable guarantees: they are the properties the system **MUST** hold at all times, whose violation is, by definition, a defect. Each invariant is identified by a stable ID of the form INV-**<class>-<nn>**, where **<class>** is one of I (Identity), M (Immutability), R (Resolvability), C (Completeness), L (Legality), A (Auditability), or V (Conservation). There are thirty-eight invariants in total, distributed across the seven classes.

The traceability chain for each invariant documents the architecture component that realizes it, the ADR that justifies the realization, the database table that backs it, the API endpoint that exposes it, the code module that implements it, the storage object that holds its artifacts, the monitor that continuously verifies it, the test that exercises it, the runtime evidence that demonstrates its satisfaction, the certification record that signed off on the evidence, the team that owns the invariant, and the trace status. A break at any link in the chain is a defect; a SEV-1 invariant with a broken chain is a release-blocking defect. For RC1, all thirty-eight invariants are traced with current evidence; the Status column reads “Traced” for every row.

This chapter contains three tables. §4.2 is the primary invariant traceability matrix — the audit-critical table that an external auditor consults to verify the chain for any invariant. §4.3 is the invariant monitor traceability matrix, which documents, for each invariant, the monitor that continuously verifies it, the cadence at which the monitor runs, the implementation of the monitor, the alert routing when the monitor fires, and the runbook that the on-call engineer executes. §4.4 is the invariant evidence requirements matrix, which documents, for each invariant, the type of evidence that satisfies the invariant, the source from which the evidence is drawn, the maximum age below which the evidence is considered current, and the storage location where the evidence is archived.

4.2 The Invariant Traceability Matrix

The matrix below traces each of the thirty-eight invariants through the full thirteen-node chain. The columns correspond to the chain nodes defined in Chapter 2, §2.2. A cell containing “—” indicates that the link is not applicable to this invariant (for example, a pure-data invariant may have no API endpoint). A cell containing “GAP” would indicate a missing link; no such cells exist in the RC1 matrix. All thirty-eight rows have Status “Traced”.

Req ID	Invariant	Sev	Architecture	ADR	Database	API	Code	Storage	Monitor	Test	Evidence	Cert	Owner	Status
INV-	Every	1	Publi	ADR-	dois,	POST	identi	cano	INV-	TEST-	EVID-	CERT-	Data	Trace

I-01	published article has a globally unique DOI		Publication Service, Identifier Service	003	articles	/api/v1/articles/{id}/publish	services/doi_registry.py	nical-artifacts/doi-registry/	MON-I-01	INV-I-01	2026-014	RC1-014	Eng	d
INV-I-02	Every published article has an internal Opus Publica ID	1	Publication Service	ADR-003	articles	GET /api/v1/articles/{op_id}	publication_service/opus_id.py	—	INV-MON-I-02	TEST-INV-I-02	EVID-2026-015	CERT-RC1-014	Data Eng	Trace d
INV-I-03	Every version has a distinct version identifier	1	Versioning Service	ADR-014	article_versions	GET /api/v1/articles/{id}/versions	version_services/history.py	canonical-artifacts/versions/	INV-MON-I-03	TEST-INV-I-03	EVID-2026-016	CERT-RC1-014	Platform Eng	Trace d
INV-I-04	Every author has an ORCID ID	2	Identity Service, Auth or Service	ADR-022	authors, author_orcids	POST /api/v1/authors	author_service/orcid.py	—	INV-MON-I-04	TEST-INV-I-04	EVID-2026-017	CERT-RC1-015	Identity Eng	Trace d
INV-I-05	Every issue has a stable issue identifier	2	Journal Service	ADR-009	issues	GET /api/v1/journals/{jid}/issues/{iid}	journal_service/issues.py	—	INV-MON-I-05	TEST-INV-I-05	EVID-2026-018	CERT-RC1-015	Data Eng	Trace d

INV-I-06	Every journal has a stable journal identifier (ISSN)	1	Journal Service	ADR-009	journals	GET /api/v1/journals/{issn}	journal_service/journals.py	—	INV-MON-I-06	TEST-INV-I-06	EVID-2026-019	CERT-RC1-015	Data Eng	Traced
INV-I-07	Every artifact has a content-addressable SHA-256 hash	1	Storage Service	ADR-011	artifacts	GET /api/v1/artifacts/{sha256}	storage_service/hash.py	canonical-artifacts/	INV-MON-I-07	TEST-INV-I-07	EVID-2026-020	CERT-RC1-016	Platform Eng	Traced
INV-M-01	Published article PDFs are content-immutable	1	Storage Service	ADR-011	artifacts	—	storage_service/immutable.py	canonical-artifacts/pdf/	INV-MON-M-01	TEST-INV-M-01	EVID-2026-021	CERT-RC1-016	Platform Eng	Traced
INV-M-02	Published article XML is content-immutable	1	Storage Service	ADR-011	artifacts	—	storage_service/immutable.py	canonical-artifacts/xml/	INV-MON-M-02	TEST-INV-M-02	EVID-2026-022	CERT-RC1-016	Platform Eng	Traced
INV-M-03	Audit log	1	Audit Service	ADR-007	audit_log	POST /api/	audit_serv	audit-log/	INV-MON	TEST-INV-	EVID-2026-	CERT-RC1-	Observabil	Traced

	entries are appended-only		ce			v1/audit	ice/append_only.py		- M-03	M-03	023	017	ity	
INV-M-04	Canonical artifacts are write-once read-many (WO RM)	1	Storage Service	ADR-011	artifacts	—	storage_service/workm.py	canonical-artifacts/	INV-MON - M-04	TEST-INV-M-04	EVID-2026-024	CERT-RC1-016	Platform Eng	Traced
INV-M-05	Version history is appended-only	1	Versioning Service	ADR-014	article_versions	—	version_service/history.py	canonical-artifacts/versions/	INV-MON - M-05	TEST-INV-M-05	EVID-2026-025	CERT-RC1-014	Platform Eng	Traced
INV-R-01	Every DOI resolves to a public landing page	1	Resolution Service	ADR-024	dois	GET /doi/{doi}	resolution_service/doi.py	—	INV-MON -R-01	TEST-INV-R-01	EVID-2026-026	CERT-RC1-018	Platform Eng	Traced
INV-R-02	Every internal OP-ID resolves within the API	1	API Gateway	ADR-003	articles	GET /api/v1/articles/{op_id}	api_gateway/router.py	—	INV-MON -R-02	TEST-INV-R-02	EVID-2026-027	CERT-RC1-018	Platform Eng	Traced
INV-R-03	Every artifact	1	Storage Servi	ADR-011	artifacts	GET /api/v1/arti	storage_service/	canonical-artifa	INV-MON -R-03	TEST-INV-R-03	EVID-2026-028	CERT-RC1-016	Platform Eng	Traced

	hash resolves to artifact content		ce			facts/ {sha2 56}	retrieval.p y	cts/						
INV-R-04	Every reference DOI resolves or is flagged broken	2	Reference Service	ADR-024	references, ref_resolution	—	reference_service/resolver.py	—	INV-MON-R-04	TEST-INV-R-04	EVID-2026-029	CERT-RC1-018	Data Eng	Traced
INV-R-05	Every public URL is checked periodically for resolution	3	Link Health Service	ADR-024	url_health	GET / api/v1/links/health	link_health_service/scanner.py	—	INV-MON-R-05	TEST-INV-R-05	EVID-2026-030	CERT-RC1-018	Observability	Traced
INV-C-01	Every published article has all required metadata fields	1	Metadata Service	ADR-019	article_metadata	GET / api/v1/articles/{id}/metadata	metadata_service/validator.py	—	INV-MON-C-01	TEST-INV-C-01	EVID-2026-031	CERT-RC1-019	Data Eng	Traced
INV-C-02	Every published article has	1	Publication Service	ADR-026	article_metadata	GET / api/v1/articles/{id}/lic	publication_service/licenses	—	INV-MON-C-02	TEST-INV-C-02	EVID-2026-032	CERT-RC1-019	Data Eng	Traced

	a licens e state ment					ense	e.py							
INV-C-03	Every published article has author affiliations	2	Author Service	ADR-022	author_affiliations	—	author_service/affiliations.py	—	INV-MON-C-03	TEST-INV-C-03	EVID-2026-033	CERT-RC1-019	Data Eng	Traced
INV-C-04	Every published article has a funding statement	2	Metadata Service	ADR-019	article_metadata	GET /api/v1/articles/{id}/funding	metadata_service/funding.py	—	INV-MON-C-04	TEST-INV-C-04	EVID-2026-034	CERT-RC1-019	Data Eng	Traced
INV-C-05	Every published article has a COI statement	2	Publication Service	ADR-026	article_metadata	GET /api/v1/articles/{id}/coi	publication_service/coi.py	—	INV-MON-C-05	TEST-INV-C-05	EVID-2026-035	CERT-RC1-019	Data Eng	Traced
INV-C-06	Every published article has a submission date	2	Submission Service	ADR-009	submissions	GET /api/v1/submissions/{id}	submission_service/	—	INV-MON-C-06	TEST-INV-C-06	EVID-2026-036	CERT-RC1-019	Data Eng	Traced
INV-C-07	Every published article has	1	Review Service	ADR-029	review_history	GET /api/v1/articles/{id}/re	review_service/history.py	audit-log/reviews/	INV-MON-C-07	TEST-INV-C-07	EVID-2026-037	CERT-RC1-020	Platform Eng	Traced

	a peer review history record					views								
INV-L-01	Every published article has a valid license	1	Publication Service	ADR-026	article_metadata	POST /api/v1/articles/{id}/publish	publication_service/permissions.py	—	INV-MON-L-01	TEST-INV-L-01	EVID-2026-038	CERT-RC1-021	Legal Eng	Traced
INV-L-02	Every published article has cleared copyright	1	Publication Service	ADR-026	copyright_clearance	POST /api/v1/articles/{id}/copyright	publication_service/copyright.py	audit-log/copyright/	INV-MON-L-02	TEST-INV-L-02	EVID-2026-039	CERT-RC1-021	Legal Eng	Traced
INV-L-03	Every published article has cleared plagiarism check	1	Integrity Service	ADR-031	plagiarism_checks	POST /api/v1/articles/{id}/plagiarism	integrity_service/plagiarism.py	audit-log/plagiarism/	INV-MON-L-03	TEST-INV-L-03	EVID-2026-040	CERT-RC1-021	Platform Eng	Traced
INV-L-04	Every published article has ethics approval if	2	Review Service	ADR-029	ethics_approvals	POST /api/v1/articles/{id}/ethics	review_service/ethics.py	audit-log/ethics/	INV-MON-L-04	TEST-INV-L-04	EVID-2026-041	CERT-RC1-021	Legal Eng	Traced

	appli cable													
INV-L-05	Every published article has author consent	1	Auth or Service	ADR-022	auth or_consent	POST /api/v1/articles/{id}/consent	auth or_service/consent.py	audit-log/consent/	INV-MON-L-05	TEST-INV-L-05	EVID-2026-042	CERT-RC1-021	Legal Eng	Trace d
INV-L-06	Personal data is processed under GDPR-compliant consent	1	Identity Service, Privacy Service	ADR-034	consent_records, pii_index	GET /api/v1/privacy/consent	privacy_service/gdpr.py	audit-log/consent/	INV-MON-L-06	TEST-INV-L-06	EVID-2026-043	CERT-RC1-022	Security Eng	Trace d
INV-A-01	Every state change is logged in the audit log	1	Audit Service	ADR-007	audit_log	POST /api/v1/audit	audit_service/recorder.py	audit-log/	INV-MON-A-01	TEST-INV-A-01	EVID-2026-044	CERT-RC1-023	Observability	Trace d
INV-A-02	Every audit entry has actor, timestamp, action, target	1	Audit Service	ADR-007	audit_log	GET /api/v1/audit	audit_service/schema.py	audit-log/	INV-MON-A-02	TEST-INV-A-02	EVID-2026-045	CERT-RC1-023	Observability	Trace d

INV-A-03	Audit entries are tamper-evident (chained hashes)	1	Audit Service	ADR-007	audit_log	—	audit_service/chaining.py	audit-log/	INV-MON-A-03	TEST-INV-A-03	EVID-2026-046	CERT-RC1-023	Security Eng	Traced
INV-A-04	Audit log is queryable by external auditors (read-only)	2	Audit Service, API Gateway	ADR-007	audit_log	GET /api/v1/audit/export	audit_service/export.py	audit-log/	INV-MON-A-04	TEST-INV-A-04	EVID-2026-047	CERT-RC1-023	Observability	Traced
INV-V-01	Published artifacts are replicated to CLOCKSS	1	Preservation Service	ADR-028	preservation_deposits	—	preservation_service/clocks.py	preservation/clocks/	INV-MON-V-01	TEST-INV-V-01	EVID-2026-048	CERT-RC1-024	Platform Eng	Traced
INV-V-02	Published artifacts are replicated to LOCKSS	1	Preservation Service	ADR-028	preservation_deposits	—	preservation_service/locks.py	preservation/locks/	INV-MON-V-02	TEST-INV-V-02	EVID-2026-049	CERT-RC1-024	Platform Eng	Traced
INV-V-03	Published artifacts	2	Preservation	ADR-028	retention_policy	—	preservation_service/	preservation/	INV-MON-V-03	TEST-INV-V-03	EVID-2026-050	CERT-RC1-024	Platform Eng	Traced

	cts have a 25-year retention policy		Service		es		vice/retention.py							
INV-V-04	Deleted artifacts are tombstoned, not removed	1	Storage Service, Publication Service	ADR-011	article_tombstones	DELETE /api/v1/articles/{id}	publication_service/tombstone.py	canonical-artifacts/tombstones/	INV-MON-V-04	TEST-INV-V-04	EVID-2026-051	CERT-RC1-024	Platform Eng	Traced

The thirty-eight rows above cover all seven invariant classes. Identity (7) establishes the persistent identifiers that name every publishable entity. Immutability (5) establishes that published content cannot be silently modified. Resolvability (5) establishes that every identifier resolves to its referent. Completeness (7) establishes that every published article carries the metadata required by the scholarly record. Legality (6) establishes that every published article has cleared the legal checks required for distribution. Auditability (4) establishes that the system records, in a tamper-evident form, every state change. Conservation (4) establishes that the published record is preserved against loss for the long term. The union of these seven classes is the platform's invariant surface — the set of properties whose continuous satisfaction is the platform's contract with the scholarly record.

4.3 Invariant Monitor Traceability

An invariant that is not continuously monitored is an invariant whose violation will not be detected until an auditor inspects the system — by which time the violation may have been compounded by further state changes. Every invariant in §4.2 therefore has a corresponding monitor that continuously verifies the invariant at a defined cadence. The matrix below documents, for each monitor, the invariant it verifies, the cadence at which it runs, the implementation that realizes the monitor, the alert routing used when the monitor fires, the runbook the on-call engineer executes in response, and the monitor's status. A monitor in “Gap” status means the invariant is unmonitored; no such rows exist in RC1.

Monitor ID	Invariant	Cadence	Implementation	Alert Routing	Runbook	Status
INV-MON-I-01	INV-I-01	continuous	Prometheus	PagerDuty:	RB-INV-I-01	Active

			rule: doi_uniquenes s	pub-cert		
INV-MON-I-02	INV-I-02	continuous	Prometheus rule: op_id_uniquen ess	PagerDuty: pub-cert	RB-INV-I-02	Active
INV-MON-I-03	INV-I-03	continuous	Prometheus rule: version_distinc t	PagerDuty: pub-cert	RB-INV-I-03	Active
INV-MON-I-04	INV-I-04	hourly scan	Lambda: author_orcid_a udit	Slack: #data- alerts	RB-INV-I-04	Active
INV-MON-I-05	INV-I-05	hourly scan	Lambda: issue_id_audit	Slack: #data- alerts	RB-INV-I-05	Active
INV-MON-I-06	INV-I-06	hourly scan	Lambda: journal_issn_a udit	Slack: #data- alerts	RB-INV-I-06	Active
INV-MON-I-07	INV-I-07	continuous	Prometheus rule: artifact_hash_ present	PagerDuty: storage	RB-INV-I-07	Active
INV-MON-M-01	INV-M-01	daily scan	K8s CronJob: pdf_hash_reco mpute	PagerDuty: storage	RB-INV-M-01	Active
INV-MON-M-02	INV-M-02	daily scan	K8s CronJob: xml_hash_reco mpute	PagerDuty: storage	RB-INV-M-02	Active
INV-MON-M-03	INV-M-03	continuous	Postgres trigger: audit_append_ only	PagerDuty: security	RB-INV-M-03	Active
INV-MON-M-04	INV-M-04	daily scan	K8s CronJob: worm_bucket_ audit	PagerDuty: storage	RB-INV-M-04	Active
INV-MON-M-05	INV-M-05	continuous	Postgres trigger: version_history_ _append	PagerDuty: pub-cert	RB-INV-M-05	Active
INV-MON-R-01	INV-R-01	5-min probe	Blackbox exporter:	PagerDuty: platform	RB-INV-R-01	Active

			doi_resolution			
INV-MON-R-02	INV-R-02	5-min probe	Blackbox exporter: op_id_resolution	PagerDuty: platform	RB-INV-R-02	Active
INV-MON-R-03	INV-R-03	5-min probe	Blackbox exporter: artifact_retrieval	PagerDuty: storage	RB-INV-R-03	Active
INV-MON-R-04	INV-R-04	daily scan	Lambda: reference_resolution_audit	Slack: #data-alerts	RB-INV-R-04	Active
INV-MON-R-05	INV-R-05	weekly scan	K8s CronJob: url_health_scanner	Slack: #obs-alerts	RB-INV-R-05	Active
INV-MON-C-01	INV-C-01	continuous	Prometheus rule: metadata_completeness	PagerDuty: pub-cert	RB-INV-C-01	Active
INV-MON-C-02	INV-C-02	continuous	Prometheus rule: license_present	PagerDuty: pub-cert	RB-INV-C-02	Active
INV-MON-C-03	INV-C-03	hourly scan	Lambda: affiliation_audit	Slack: #data-alerts	RB-INV-C-03	Active
INV-MON-C-04	INV-C-04	hourly scan	Lambda: funding_audit	Slack: #data-alerts	RB-INV-C-04	Active
INV-MON-C-05	INV-C-05	hourly scan	Lambda: coi_audit	Slack: #data-alerts	RB-INV-C-05	Active
INV-MON-C-06	INV-C-06	hourly scan	Lambda: submission_date_audit	Slack: #data-alerts	RB-INV-C-06	Active
INV-MON-C-07	INV-C-07	continuous	Prometheus rule: review_history_present	PagerDuty: pub-cert	RB-INV-C-07	Active
INV-MON-L-01	INV-L-01	continuous	Prometheus rule: license_valid	PagerDuty: legal	RB-INV-L-01	Active
INV-MON-L-02	INV-L-02	hourly scan	Lambda: copyright_audi	PagerDuty: legal	RB-INV-L-02	Active

			t			
INV-MON-L-03	INV-L-03	continuous	Prometheus rule: plagiarism_cleared	PagerDuty: pub-cert	RB-INV-L-03	Active
INV-MON-L-04	INV-L-04	hourly scan	Lambda: ethics_audit	Slack: #legal-alerts	RB-INV-L-04	Active
INV-MON-L-05	INV-L-05	continuous	Prometheus rule: author_consent_present	PagerDuty: legal	RB-INV-L-05	Active
INV-MON-L-06	INV-L-06	daily scan	K8s CronJob: gdpr_consent_audit	PagerDuty: security	RB-INV-L-06	Active
INV-MON-A-01	INV-A-01	continuous	Prometheus rule: audit_log_writes	PagerDuty: observability	RB-INV-A-01	Active
INV-MON-A-02	INV-A-02	continuous	Postgres constraint: audit_schema	PagerDuty: observability	RB-INV-A-02	Active
INV-MON-A-03	INV-A-03	daily scan	K8s CronJob: audit_chain_verify	PagerDuty: security	RB-INV-A-03	Active
INV-MON-A-04	INV-A-04	5-min probe	Blackbox exporter: audit_export	PagerDuty: observability	RB-INV-A-04	Active
INV-MON-V-01	INV-V-01	daily scan	K8s CronJob: clockss_deposit_audit	PagerDuty: preservation	RB-INV-V-01	Active
INV-MON-V-02	INV-V-02	daily scan	K8s CronJob: lockss_deposit_audit	PagerDuty: preservation	RB-INV-V-02	Active
INV-MON-V-03	INV-V-03	weekly scan	K8s CronJob: retention_policy_audit	Slack: #preservation-alerts	RB-INV-V-03	Active
INV-MON-V-04	INV-V-04	continuous	Prometheus rule: tombstone_present	PagerDuty: storage	RB-INV-V-04	Active

The cadence column reflects the maximum time between a violation occurring and the monitor detecting it. Continuous monitors fire on every relevant state change and detect violations within seconds. Five-minute probes detect violations within five minutes. Hourly, daily, and weekly scans detect violations within their respective windows. The cadence is chosen to match the invariant's severity and the cost of late detection: SEV-1 invariants that affect the scholarly record (DOI uniqueness, immutability, audit-log integrity) are continuously monitored; SEV-3 invariants that affect only link health are weekly-scanned. An auditor verifying the monitor regime SHALL confirm that the cadence in the table matches the cadence configured in the monitor implementation.

4.4 Invariant Evidence Requirements

The runtime evidence referenced by the Evidence column of §4.2 is not a single artifact but a class of artifacts. Each invariant requires a specific type of evidence, drawn from a specific source, with a specific maximum age below which the evidence is considered current, and stored at a specific location. The matrix below documents these requirements. An evidence artifact older than the Max Age is treated as missing evidence (Rule IR-06, Chapter 2 §2.5) and SHALL be refreshed before the next release.

Invariant	Evidence Type	Evidence Source	Max Age	Storage Location
INV-I-01	Reconciliation report	PubCert hourly reconciliation job	24h	s3://opus-evidence/invariants/I-01/
INV-I-02	Reconciliation report	PubCert hourly reconciliation job	24h	s3://opus-evidence/invariants/I-02/
INV-I-03	Version registry export	version_service /export endpoint	24h	s3://opus-evidence/invariants/I-03/
INV-I-04	Author ORCID coverage report	Lambda: author_orcid_audit	7d	s3://opus-evidence/invariants/I-04/
INV-I-05	Issue registry export	journal_service /export endpoint	7d	s3://opus-evidence/invariants/I-05/
INV-I-06	Journal ISSN registry export	journal_service /export endpoint	7d	s3://opus-evidence/invariants/I-06/
INV-I-07	Artifact hash index export	storage_service /index endpoint	24h	s3://opus-evidence/invariants/I-07/
INV-M-01	PDF hash recompute report	K8s CronJob: pdf_hash_recompute	24h	s3://opus-evidence/invariants/M-01/
INV-M-02	XML hash recompute report	K8s CronJob: xml_hash_recompute	24h	s3://opus-evidence/invariants/M-02/
INV-M-03	Audit log append-only check	Postgres trigger output	24h	s3://opus-evidence/invariants/M-03/
INV-M-04	WORM bucket audit report	K8s CronJob: worm_bucket_audit	24h	s3://opus-evidence/invariants/M-04/

INV-M-05	Version history append check	Postgres trigger output	24h	s3://opus-evidence/ invariants/M-05/
INV-R-01	DOI resolution probe log	Blackbox exporter: doi_resolution	1h	s3://opus-evidence/ invariants/R-01/
INV-R-02	OP-ID resolution probe log	Blackbox exporter: op_id_resolution	1h	s3://opus-evidence/ invariants/R-02/
INV-R-03	Artifact retrieval probe log	Blackbox exporter: artifact_retrieval	1h	s3://opus-evidence/ invariants/R-03/
INV-R-04	Reference resolution audit	Lambda: reference_resolution_ audit	7d	s3://opus-evidence/ invariants/R-04/
INV-R-05	URL health scan report	K8s CronJob: url_health_scanner	7d	s3://opus-evidence/ invariants/R-05/
INV-C-01	Metadata completeness dashboard	Grafana: invariants/metadata	24h	s3://opus-evidence/ invariants/C-01/
INV-C-02	License presence dashboard	Grafana: invariants/license	24h	s3://opus-evidence/ invariants/C-02/
INV-C-03	Affiliation audit report	Lambda: affiliation_audit	7d	s3://opus-evidence/ invariants/C-03/
INV-C-04	Funding audit report	Lambda: funding_audit	7d	s3://opus-evidence/ invariants/C-04/
INV-C-05	COI audit report	Lambda: coi_audit	7d	s3://opus-evidence/ invariants/C-05/
INV-C-06	Submission date audit report	Lambda: submission_date_audi t	7d	s3://opus-evidence/ invariants/C-06/
INV-C-07	Review history presence dashboard	Grafana: invariants/review- history	24h	s3://opus-evidence/ invariants/C-07/
INV-L-01	License validity report	publication_service /license-audit	24h	s3://opus-evidence/ invariants/L-01/
INV-L-02	Copyright clearance record	copyright_clearance table export	30d	s3://opus-evidence/ invariants/L-02/
INV-L-03	Plagiarism clearance record	plagiarism_checks table export	30d	s3://opus-evidence/ invariants/L-03/
INV-L-04	Ethics approval record	ethics_approvals table export	30d	s3://opus-evidence/ invariants/L-04/
INV-L-05	Author consent record	author_consent table export	30d	s3://opus-evidence/ invariants/L-05/

INV-L-06	GDPR consent audit report	K8s CronJob: gdpr_consent_audit	7d	s3://opus-evidence/invariants/L-06/
INV-A-01	Audit log write-rate dashboard	Grafana: audit/writes	1h	s3://opus-evidence/invariants/A-01/
INV-A-02	Audit schema constraint check	Postgres constraint output	24h	s3://opus-evidence/invariants/A-02/
INV-A-03	Audit chain verification report	K8s CronJob: audit_chain_verify	24h	s3://opus-evidence/invariants/A-03/
INV-A-04	Audit export probe log	Blackbox exporter: audit_export	1h	s3://opus-evidence/invariants/A-04/
INV-V-01	CLOCKSS deposit confirmation	clockss_service /deposits	24h	s3://opus-evidence/invariants/V-01/
INV-V-02	LOCKSS deposit confirmation	lockss_service /deposits	24h	s3://opus-evidence/invariants/V-02/
INV-V-03	Retention policy audit report	K8s CronJob: retention_policy_audit	30d	s3://opus-evidence/invariants/V-03/
INV-V-04	Tombstone presence dashboard	Grafana: invariants/tombstones	24h	s3://opus-evidence/invariants/V-04/

The Max Age column encodes the freshness requirement for each invariant's evidence. Evidence with Max Age “1h” (continuous probes) is regenerated hourly; evidence with Max Age “24h” is regenerated daily; evidence with Max Age “7d” is regenerated weekly; evidence with Max Age “30d” is regenerated monthly. The Release Certification Authority SHALL verify, before every release, that every invariant's evidence is within its Max Age. Stale evidence is treated as missing evidence and blocks release certification until refreshed.

Chapter 5 — Non-Negotiable Rules Traceability

5.1 Purpose

This chapter traces every non-negotiable rule established by Volume I, Chapter 3, §3.3 of the Engineering Constitution through the full 13-node traceability chain defined in Chapter 2 of this Matrix. The rules differ from invariants in that invariants are properties the system **MUST** hold continuously, while rules are constraints on how the system is designed, built, operated, and amended. A rule violation is a process or design defect; an invariant violation is a runtime defect. Both block release certification, but they are remediated by different procedures (rules by process change, invariants by code change).

Volume I, Chapter 3 establishes forty-seven non-negotiable rules across six classes: Architecture (7), Data (7), Publication (8), External Integration (7), Operations (8), and Process (10). Each rule is identified by a stable ID of the form <CLASS>-<nnn>. Every rule is binding on every engineer, every team, and every release; no rule **MAY** be waived below the Chief Systems Architect, and the Chief Systems Architect **SHALL NOT** waive a SEV-1 rule under any circumstance. The rules are the operational expression of the Constitution's design philosophy: a layered, contract-bound, audit-traced, externally-integrable, operable, and process-disciplined platform.

This chapter contains three tables. §5.2 is the primary rules traceability matrix — the audit-critical table that an external auditor consults to verify the chain for any rule. §5.3 is the rule enforcement mechanisms matrix, which documents, for each rule, the layer at which the rule is enforced (static, runtime, certification, or governance), the mechanism by which it is enforced, the detection that identifies a violation, and the owner of the enforcement. §5.4 is the rule violation response matrix, which documents, for each rule, the severity of a violation, the procedure for responding to it, and the escalation path.

5.2 The Rules Traceability Matrix

The matrix below traces each of the forty-seven non-negotiable rules through the traceability chain. The columns correspond to the chain nodes defined in Chapter 2, §2.2, with the exception of the Monitor node (rules are enforced rather than monitored; the enforcement regime is documented in §5.3). A cell containing “—” indicates that the link is not applicable to this rule. For RC1, all forty-seven rows have Status “Traced”.

Req ID	Rule	Sev	Architecture	ADR	Database	API	Code	Storage	Test	Evidence	Cert	Owner	Status
ARCH-001	The platform is layered:	2	All subsystems	ADR-001	—	—	repo root layout (docs/archit	—	TEST-ARCH-001	EVID-2026-100	CERT-RC1-030	Architecture	Traced

	presentation/application/domain/data						ecture/)						
ARCH-002	Each subsystem has a single owning team	2	All subsystems	ADR-002	subsystems	—	CODE OWNERS	—	TEST-ARCH-002	EVID-2026-101	CERT-RC1-030	Architecture	Traced
ARCH-003	Synchronous cross-team comm is via published API contracts only	1	API Gateway	ADR-003	api_contracts	(all)	api_gateway/router.py	—	TEST-ARCH-003	EVID-2026-102	CERT-RC1-030	Platform Eng	Traced
ARCH-004	Async cross-team comm is via the event bus only	1	Event Bus	ADR-005	event_log	—	event_bus/	event-stream/	TEST-ARCH-004	EVID-2026-103	CERT-RC1-030	Platform Eng	Traced
ARCH-005	No subsystem reads another subsystem's	1	All subsystems	ADR-004	—	—	netpoll/	—	TEST-ARCH-005	EVID-2026-104	CERT-RC1-030	Security Eng	Traced

	datab ase directl y												
ARCH-006	All persis tent state is in a desig nated syste m of recor d	1	Stora ge Servic e	ADR-006	data_ owner ship	—	storag e_ser vice/ regist ry.py	—	TEST- ARCH- 006	EVID- 2026- 105	CERT- RC1- 030	Data Eng	Trace d
ARCH-007	The platfo rm is deplo yable as multi ple indep enden t servic es	1	All subsy stems	ADR-008	—	—	deplo y/ manif ests/	—	TEST- ARCH- 007	EVID- 2026- 106	CERT- RC1- 030	Platfo rm Eng	Trace d
DATA-001	Every persis tent entity has a single syste m of recor d	1	Stora ge Servic e	ADR-006	data_ owner ship	—	storag e_ser vice/ regist ry.py	—	TEST- DATA- 001	EVID- 2026- 110	CERT- RC1- 031	Data Eng	Trace d
DATA-002	Every DB has a docu ment ed sche ma	1	Stora ge Servic e	ADR-010	sche ma_m igratio ns	—	migra tions/	—	TEST- DATA- 002	EVID- 2026- 111	CERT- RC1- 031	Data Eng	Trace d

	under versio n contr ol												
DATA-003	Every sche ma chang e is revers ible (forw ard+b ackwa rd comp atible)	1	Stora ge Servic e	ADR-0 10	sche ma_m igratio ns	—	migra tions/ lint.py	—	TEST- DATA- 003	EVID- 2026- 112	CERT- RC1- 031	Data Eng	Trace d
DATA-004	Every datab ase has auto mate d point- in- time recov ery	1	Stora ge Servic e	ADR-0 12	—	—	storag e_ser vice/ pitr.p y	backu ps/ pitr/	TEST- DATA- 004	EVID- 2026- 113	CERT- RC1- 031	Platfo rm Eng	Trace d
DATA-005	Every datab ase has daily backu ps	1	Stora ge Servic e	ADR-0 12	—	—	storag e_ser vice/ backu p.py	backu ps/ daily/	TEST- DATA- 005	EVID- 2026- 114	CERT- RC1- 031	Platfo rm Eng	Trace d
DATA-006	Every datab ase has a reten tion and dispos al	2	Stora ge Servic e	ADR-0 12	reten tion_ polici es	—	storag e_ser vice/ reten tion.p y	backu ps/	TEST- DATA- 006	EVID- 2026- 115	CERT- RC1- 031	Data Eng	Trace d

	policy												
DATA-007	No PII is logged in plaintext	1	Logging Service	ADR-013	—	—	logging_service/redactor.py	—	TEST-DATA-007	EVID-2026-116	CERT-RC1-031	Security Eng	Traced
PUB-001	No article is published without editorial acceptance	1	Publication Service	ADR-014	editorial_decisions	POST /api/v1/articles/{id}/publish	publication_service/guard.py	audit-log/	TEST-PUB-001	EVID-2026-120	CERT-RC1-032	Platform Eng	Traced
PUB-002	No article is published without a DOI	1	Publication Service, Identifier Service	ADR-003	dois	POST /api/v1/articles/{id}/publish	publication_service/guard.py	—	TEST-PUB-002	EVID-2026-121	CERT-RC1-032	Data Eng	Traced
PUB-003	No article is published without peer review record	1	Review Service	ADR-029	review_history	POST /api/v1/articles/{id}/publish	publication_service/guard.py	audit-log/reviews/	TEST-PUB-003	EVID-2026-122	CERT-RC1-032	Platform Eng	Traced
PUB-004	No article is published without licens	1	Publication Service	ADR-026	article_metadata	POST /api/v1/articles/{id}/publish	publication_service/licenses.py	—	TEST-PUB-004	EVID-2026-123	CERT-RC1-032	Legal Eng	Traced

	e state ment												
PUB-005	No article is published without author consent	1	Author Service	ADR-022	author_consent	POST /api/v1/articles/{id}/publish	author_service/conse nt.py	audit-log/ conse nt/	TEST-PUB-005	EVID-2026-124	CERT-RC1-032	Legal Eng	Trace d
PUB-006	No article is published without plagiarism clearance	1	Integrity Service	ADR-031	plagiarism_checks	POST /api/v1/articles/{id}/publish	integr ity_ service/ plagiarism.py	audit-log/ plagiarism/	TEST-PUB-006	EVID-2026-125	CERT-RC1-032	Platform Eng	Trace d
PUB-007	Published articles are deposited to Crossref within 24h	1	Crossref Integration Service	ADR-024	crossref_deposits	—	crossref_service/deposit.py	audit-log/ crossref/	TEST-PUB-007	EVID-2026-126	CERT-RC1-032	Data Eng	Trace d
PUB-008	Published articles are deposited to CLOCKSS within	1	Preservation Service	ADR-028	preservation_deposits	—	preservation_service/clocks s.py	preservation/ clocks s/	TEST-PUB-008	EVID-2026-127	CERT-RC1-032	Platform Eng	Trace d

	24h												
EXT-001	Crossref deposits it is retrieved until acknowledged	1	Crossref Integration	ADR-024	crossref_deposits	—	crossref_service/retry.py	—	TEST-EXT-001	EVID-2026-130	CERT-RC1-033	Data Eng	Traced
EXT-002	ORCID authentication uses OAuth 2.0	1	Identity Service	ADR-022	author_orcids	GET /api/v1/auth/orcid/callback	identity_service/oauth.py	—	TEST-EXT-002	EVID-2026-131	CERT-RC1-033	Identity Eng	Traced
EXT-003	External integrations have circuit breakers	1	API Gateway	ADR-016	—	—	api_gateway/circuit_breaker.py	—	TEST-EXT-003	EVID-2026-132	CERT-RC1-033	Platform Eng	Traced
EXT-004	External integrations have timeouts ≤30s	2	API Gateway	ADR-016	—	—	api_gateway/timeout.py	—	TEST-EXT-004	EVID-2026-133	CERT-RC1-033	Platform Eng	Traced
EXT-005	External integrations log all calls	2	Logging Service	ADR-013	integration_logs	—	logging_service/integrations.py	logs/integrations/	TEST-EXT-005	EVID-2026-134	CERT-RC1-033	Observability	Traced
EXT-006	External integrations have idemp	1	All integrations	ADR-016	idempotency_keys	—	api_gateway/idempotency.py	—	TEST-EXT-006	EVID-2026-135	CERT-RC1-033	Platform Eng	Traced

	otent opera tions												
EXT-007	External integrations have a failover plan	2	All integrations	ADR-016	—	—	api_gateway/failover.py	—	TEST-EXT-007	EVID-2026-136	CERT-RC1-033	Platform Eng	Traced
OPS-001	Every service has an SLO	1	All services	ADR-017	slos	GET /api/v1/ops/slos	slo_service/	—	TEST-OPS-001	EVID-2026-140	CERT-RC1-034	Observability	Traced
OPS-002	Every service has on-call coverage	1	Operations	ADR-018	oncall_schedules	—	ops/oncall.py	—	TEST-OPS-002	EVID-2026-141	CERT-RC1-034	Operations	Traced
OPS-003	Every service has a runbook	1	Operations	ADR-018	runbooks	GET /api/v1/ops/runbooks	ops/runbooks/	—	TEST-OPS-003	EVID-2026-142	CERT-RC1-034	Operations	Traced
OPS-004	Every alert has an escalation policy	1	Alerting Service	ADR-018	escalation_policies	—	alerting_service/escalation.py	—	TEST-OPS-004	EVID-2026-143	CERT-RC1-034	Observability	Traced
OPS-005	Every deployment is reversible	1	Deployment Service	ADR-020	deployments	POST /api/v1/deploy/rollback	deploy/rollback.py	—	TEST-OPS-005	EVID-2026-144	CERT-RC1-034	Platform Eng	Traced
OPS-006	Every deployment	1	Deployment Service	ADR-020	deployments	—	deploy/plan.p	—	TEST-OPS-006	EVID-2026-145	CERT-RC1-034	Platform Eng	Traced

	has a rollback plan		e				y						
OPS-007	Every production change is recorded	1	Audit Service	ADR-007	change_log	POST /api/v1/changes	audit_service/changes.py	audit-log/changes/	TEST-OPS-007	EVID-2026-146	CERT-RC1-034	Observability	Traced
OPS-008	Every incident has a postmortem	1	Operations	ADR-021	incidents, postmortems	GET /api/v1/incidents	ops/postmortem.py	incidents/	TEST-OPS-008	EVID-2026-147	CERT-RC1-034	Operations	Traced
PROC-001	Every code change is reviewed	1	Code Review Service	ADR-025	reviews	—	review_service/	—	TEST-PROC-001	EVID-2026-150	CERT-RC1-035	Engineering	Traced
PROC-002	Every code change passes CI	1	CI Service	ADR-025	ci_runs	—	ci/pipeline.py	ci-artifacts/	TEST-PROC-002	EVID-2026-151	CERT-RC1-035	Engineering	Traced
PROC-003	Every code change is associated with a task	2	Code Review Service	ADR-025	changes	—	review_service/lint.py	—	TEST-PROC-003	EVID-2026-152	CERT-RC1-035	Engineering	Traced
PROC-004	Every code change references an ADR	3	Code Review Service	ADR-025	changes	—	review_service/adr_lint.py	—	TEST-PROC-004	EVID-2026-153	CERT-RC1-035	Architecture	Traced

	(where applicable)												
PROC-005	Every production change has a deployment plan	1	Deployment Service	ADR-020	deployment_plans	POST /api/v1/deploy	deployment_plan.py	—	TEST-PROC-005	EVID-2026-154	CERT-RC1-035	Platform Eng	Traced
PROC-006	Every release has a certification record	1	Certification Service	ADR-027	certification_records	GET /api/v1/releases/{id}/cert	certification_service/	certs/	TEST-PROC-006	EVID-2026-155	CERT-RC1-035	RCA	Traced
PROC-007	Every release has a rollback plan	1	Deployment Service	ADR-020	release_plans	—	deployment/rollback_plan.py	—	TEST-PROC-007	EVID-2026-156	CERT-RC1-035	Platform Eng	Traced
PROC-008	Every release has an announcement	2	Operations	ADR-027	release_announcements	—	ops/announcement.py	—	TEST-PROC-008	EVID-2026-157	CERT-RC1-035	Operations	Traced
PROC-009	Every amendment follows the governance process	1	Governance Service	ADR-036	amendments	POST /api/v1/governance/amendments	governance_service/	amendments/	TEST-PROC-009	EVID-2026-158	CERT-RC1-035	CSA Office	Traced
PROC-	Every	2	Opera	ADR-0	oncall	—	ops/	—	TEST-	EVID-	CERT-	Opera	Trace

010	on-call shift has a handover		tions	18	_handovers		handover.py		PROC-010	2026-159	RC1-035	tions	d
-----	------------------------------	--	-------	----	------------	--	-------------	--	----------	----------	---------	-------	---

The forty-seven rows above cover all six rule classes. Architecture (7) constrains the platform's decomposition. Data (7) constrains the platform's persistence layer. Publication (8) constrains the platform's most safety-critical workflow — the act of publishing an article into the scholarly record. External Integration (7) constrains the platform's interaction with external services (Crossref, ORCID, CLOCKSS, LOCKSS). Operations (8) constrains the platform's operability. Process (10) constrains the engineering organization's own discipline. The union of these six classes is the platform's rule surface — the set of constraints whose continuous enforcement is the platform's contract with its engineers.

5.3 Rule Enforcement Mechanisms

A rule that is not enforced is a rule that exists only on paper. Every rule in §5.2 therefore has a defined enforcement layer (static, runtime, certification, or governance), a defined enforcement mechanism (the specific tool or procedure that enforces the rule), a defined detection (how a violation is identified), and a defined owner. The matrix below documents these for every rule. An auditor verifying the enforcement regime SHALL confirm that the layer, mechanism, and detection configured in the implementation match those documented here.

Rule ID	Enforcement Layer	Enforcement Mechanism	Detection	Owner
ARCH-001	static	Architecture lint: layer_isolation	CI failure on cross-layer import	Architecture
ARCH-002	governance	CODEOWNERS file + PR review	GitHub branch protection	Architecture
ARCH-003	runtime	API Gateway contract validation	Request rejected at gateway	Platform Eng
ARCH-004	runtime	Event bus schema registry	Event rejected by schema mismatch	Platform Eng
ARCH-005	static	Kubernetes NetworkPolicy lint	Netpol linter failure in CI	Security Eng
ARCH-006	certification	data_ownership registry review	RCA certification checklist	Data Eng
ARCH-007	certification	Deploy manifest review	RCA certification checklist	Platform Eng
DATA-001	certification	data_ownership	RCA certification	Data Eng

		registry review	checklist	
DATA-002	static	Schema lint: migrations under VCS	CI failure on unversioned schema	Data Eng
DATA-003	static	Migration compatibility linter	CI failure on breaking migration	Data Eng
DATA-004	runtime	PITR job monitor	PagerDuty: storage on missed PITR	Platform Eng
DATA-005	runtime	Daily backup job monitor	PagerDuty: storage on missed backup	Platform Eng
DATA-006	certification	Retention policy review	RCA certification checklist	Data Eng
DATA-007	static	Log redactor unit + integration tests	CI failure on plaintext PII	Security Eng
PUB-001	runtime	publication_service guard	POST /publish 403 on missing acceptance	Platform Eng
PUB-002	runtime	publication_service guard	POST /publish 403 on missing DOI	Data Eng
PUB-003	runtime	publication_service guard	POST /publish 403 on missing review	Platform Eng
PUB-004	runtime	publication_service guard	POST /publish 403 on missing license	Legal Eng
PUB-005	runtime	publication_service guard	POST /publish 403 on missing consent	Legal Eng
PUB-006	runtime	publication_service guard	POST /publish 403 on missing plagiarism clearance	Platform Eng
PUB-007	runtime	Crossref deposit retry monitor	PagerDuty: data on deposit overdue >24h	Data Eng
PUB-008	runtime	CLOCKSS deposit monitor	PagerDuty: preservation on deposit overdue >24h	Platform Eng
EXT-001	runtime	crossref_service retry queue	PagerDuty: data on unacked deposit	Data Eng
EXT-002	runtime	identity_service OAuth flow	AuthN failure on non- OAuth ORCID call	Identity Eng
EXT-003	runtime	API Gateway circuit breaker	Circuit open event in Grafana	Platform Eng
EXT-004	static	API client config lint	CI failure on timeout >30s	Platform Eng

EXT-005	runtime	Integration call logger	Missing log entry detected by monitor	Observability
EXT-006	runtime	idempotency_keys table	Duplicate request rejected with 409	Platform Eng
EXT-007	governance	Integration failover runbook	Quarterly failover drill	Platform Eng
OPS-001	certification	SLO registry review	RCA certification checklist	Observability
OPS-002	certification	oncall_schedules review	RCA certification checklist	Operations
OPS-003	certification	Runbook registry review	RCA certification checklist	Operations
OPS-004	runtime	Alerting service escalation policy check	Alert with no policy: PagerDuty: observability	Observability
OPS-005	runtime	deploy/rollback capability	Rollback failure: PagerDuty: platform	Platform Eng
OPS-006	certification	Deploy plan review	RCA certification checklist	Platform Eng
OPS-007	runtime	Audit service change_log recorder	Missing change_log entry: PagerDuty: observability	Observability
OPS-008	governance	Incident postmortem SLA	Postmortem overdue >5d: Slack: #ops-escalation	Operations
PROC-001	governance	GitHub branch protection: PR review	PR blocked on missing approval	Engineering
PROC-002	static	CI pipeline required checks	PR blocked on red CI	Engineering
PROC-003	static	PR title lint: task ref	CI failure on missing task ID	Engineering
PROC-004	static	PR lint: ADR reference	CI warning on missing ADR ref	Architecture
PROC-005	governance	Deploy plan template + sign-off	Deploy blocked on missing plan	Platform Eng
PROC-006	certification	Certification record review	Release blocked on missing CERT	RCA
PROC-007	certification	Rollback plan review	Release blocked on missing rollback plan	Platform Eng

PROC-008	governance	Release announcement template	Slack: #ops-escalation on missing announcement	Operations
PROC-009	governance	Governance process workflow	Amendment PR blocked on missing vote	CSA Office
PROC-010	governance	oncall_handovers table + lint	Slack: #ops-escalation on missing handover	Operations

The Enforcement Layer column distinguishes four layers. “Static” enforcement happens at build/lint time and blocks the merge of violating code. “Runtime” enforcement happens at request time and blocks the execution of violating operations. “Certification” enforcement happens at release certification time and blocks the release of violating builds. “Governance” enforcement happens in the engineering organization's process and is verified by reviewers, auditors, or periodic drills. The four layers are complementary: a rule enforced only statically can be bypassed by a runtime configuration change; a rule enforced only at runtime cannot prevent violating code from being merged; a rule enforced only at certification time catches violations late. Rules of SEV-1 SHOULD be enforced at two or more layers; rules of SEV-2 MUST be enforced at no fewer than one layer.

5.4 Rule Violation Response

When a rule is violated — whether detected by static, runtime, certification, or governance enforcement — a defined response procedure is triggered. The matrix below documents, for each rule, the severity of a violation, the response procedure, and the escalation path. The response procedures are normative: the on-call engineer, the reviewer, or the certifier SHALL follow the documented procedure rather than improvising. The escalation path defines, at each step, who is notified and what authority is invoked.

Rule ID	Violation Severity	Response Procedure	Escalation
ARCH-001	SEV-2	Block merge; require architect sign-off on layer correction	Architecture → Principal Architect
ARCH-002	SEV-2	Block merge; require CODEOWNERS update	Architecture → Engineering Manager
ARCH-003	SEV-1	Reject request at gateway; open SEV-1 incident	Platform Eng → On-Call → CSA
ARCH-004	SEV-1	Reject event at bus; open SEV-1 incident	Platform Eng → On-Call → CSA
ARCH-005	SEV-1	Block deploy; open SEV-1 security incident	Security Eng → On-Call → CSA
ARCH-006	SEV-1	Block release; require ownership clarification	Data Eng → RCA → CSA

ARCH-007	SEV-1	Block release; require manifest correction	Platform Eng → RCA → CSA
DATA-001	SEV-1	Block release; require ownership clarification	Data Eng → RCA → CSA
DATA-002	SEV-1	Block merge; require versioned schema	Data Eng → Data Lead
DATA-003	SEV-1	Block merge; require compatible migration	Data Eng → Data Lead → CSA
DATA-004	SEV-1	Page on-call; restore PITR; open incident	Platform Eng → On-Call → CSA
DATA-005	SEV-1	Page on-call; restore backup; open incident	Platform Eng → On-Call → CSA
DATA-006	SEV-2	Block release; require retention policy	Data Eng → Data Lead
DATA-007	SEV-1	Block merge; open security incident; require redactor fix	Security Eng → On-Call → CSA
PUB-001	SEV-1	Reject publish; page on-call; open incident	Platform Eng → On-Call → CSA
PUB-002	SEV-1	Reject publish; page on-call; open incident	Data Eng → On-Call → CSA
PUB-003	SEV-1	Reject publish; page on-call; open incident	Platform Eng → On-Call → CSA
PUB-004	SEV-1	Reject publish; page on-call; open incident	Legal Eng → On-Call → CSA
PUB-005	SEV-1	Reject publish; page on-call; open incident	Legal Eng → On-Call → CSA
PUB-006	SEV-1	Reject publish; page on-call; open incident	Platform Eng → On-Call → CSA
PUB-007	SEV-1	Page on-call; retry deposit; open incident	Data Eng → On-Call → CSA
PUB-008	SEV-1	Page on-call; retry deposit; open incident	Platform Eng → On-Call → CSA
EXT-001	SEV-1	Page on-call; drain queue; open incident	Data Eng → On-Call → CSA
EXT-002	SEV-1	Block auth; require OAuth flow correction	Identity Eng → Security → CSA
EXT-003	SEV-1	Page on-call; isolate failing integration; open incident	Platform Eng → On-Call
EXT-004	SEV-2	Block merge; require timeout	Platform Eng → Platform

		correction	Lead
EXT-005	SEV-2	Page on-call; verify logger; open incident	Observability → On-Call
EXT-006	SEV-1	Reject duplicate; page on-call; open incident	Platform Eng → On-Call → CSA
EXT-007	SEV-2	Initiate failover runbook; notify integration owner	Platform Eng → On-Call
OPS-001	SEV-1	Block release; require SLO definition	Observability → RCA → CSA
OPS-002	SEV-1	Block release; require on-call schedule	Operations → RCA → CSA
OPS-003	SEV-1	Block release; require runbook	Operations → RCA → CSA
OPS-004	SEV-1	Page on-call; assign policy; open incident	Observability → On-Call → CSA
OPS-005	SEV-1	Page on-call; execute rollback; open incident	Platform Eng → On-Call → CSA
OPS-006	SEV-1	Block deploy; require rollback plan	Platform Eng → RCA
OPS-007	SEV-1	Page on-call; reconstruct change_log; open incident	Observability → On-Call → CSA
OPS-008	SEV-1	Notify ops lead; require postmortem within 5d	Operations → Engineering Manager
PROC-001	SEV-1	Block merge; require review	Engineering → Engineering Manager
PROC-002	SEV-1	Block merge; require CI green	Engineering → Engineering Manager
PROC-003	SEV-2	Block merge; require task reference	Engineering → Engineering Manager
PROC-004	SEV-3	Warn author; require ADR reference before merge	Architecture → Engineering Manager
PROC-005	SEV-1	Block deploy; require signed deploy plan	Platform Eng → RCA
PROC-006	SEV-1	Block release; require CERT record	RCA → CSA
PROC-007	SEV-1	Block release; require rollback plan	Platform Eng → RCA → CSA
PROC-008	SEV-2	Notify ops lead; require announcement within 24h	Operations → Engineering Manager

PROC-009	SEV-1	Block amendment; require governance vote	CSA Office → CSA
PROC-010	SEV-2	Notify ops lead; require handover within 1h	Operations → Engineering Manager

The Violation Severity column reflects the worst-case impact of a violation of the rule, not the severity of the rule itself (though the two are usually aligned). SEV-1 violations block release and require on-call response; SEV-2 violations block merge or release but do not necessarily require on-call response; SEV-3 violations are warnings that do not block but SHALL be remediated. The Escalation column defines the chain of notification: the first responder is the owning team, the second responder is the team lead or on-call engineer, and the final escalation for SEV-1 violations is the Chief Systems Architect, who is the only authority empowered to authorize a workaround. Per Volume I, Chapter 5.07, the Chief Systems Architect SHALL NOT authorize a workaround that overrides a Runtime Certification failure; the implementation SHALL be corrected instead.

Chapter 6 — System Contracts Traceability

6.1 Purpose

This chapter traces the twenty subsystem contracts established by Volume II, Chapter 27 (System Contracts) through the full thirteen-node traceability chain specified in Chapter 2 of this Matrix. The subsystem is the unit of architecture in Opus Publica: each subsystem owns one runtime process, one deployment unit, one database schema, one published API, and one set of contract tests. A subsystem contract is the binding specification of what the subsystem SHALL accept, what it SHALL produce, what it SHALL refuse, and how it SHALL behave under failure. Every published article in Opus Publica traverses every one of the twenty subsystems from intake through dissemination; a defect in any subsystem contract is a defect in the scholarly record.

The traceability matrix in section 6.2 is the operational expression of Chapter 27. Each row maps a subsystem identifier (SUB-01 through SUB-20) to its realizing architecture component, the Architecture Decision Record that justifies it, its primary API, its code module, its backing database tables, its storage objects, its contract test, the runtime evidence that demonstrates the contract holds at the current release, the certification record that attests the evidence, and the status of the chain. The dependency graph in section 6.3 exposes the inter-subsystem integration topology: which subsystem depends on which, which is depended upon by which, and what failure mode propagates across the boundary when a contract is breached.

This chapter is normative. A subsystem deployed without a complete chain in this Matrix is an undocumented subsystem and SHALL NOT be released. A subsystem whose chain contains a GAP SHALL be remediated before the next release. A subsystem whose chain is STALE (evidence older than the maximum evidence age defined in Chapter 13) SHALL be re-evidenced. The Release Certification Authority SHALL NOT certify a release in which any subsystem row in section 6.2 has Status other than Traced.

6.2 The Subsystem Contract Traceability Matrix

The matrix below contains one row per subsystem, in the canonical lifecycle order established by Volume II, Chapter 27, section 27.2. The columns trace each subsystem through thirteen of the thirteen traceability chain nodes defined in Chapter 2. Cells marked “—” indicate that the link is not applicable to that subsystem (for example, a pure-coordination service may have no direct storage object). Cells marked “GAP” indicate a missing link; cells marked “STALE” indicate that the link exists but the runtime evidence is older than the maximum age. All identifiers (ADR-NNN, API paths, table names, test IDs, evidence IDs, certification IDs) SHALL resolve to live artifacts in the Engineering Knowledge Graph.

Subsys tem ID	Subsys tem	Owner	Archit ecture (servic	ADR	Primar y API	Code Modul e	Datab ase	Storag e	Contra ct Test	Eviden ce	Cert	Status
------------------	---------------	-------	-----------------------------	-----	-----------------	--------------------	--------------	-------------	-------------------	--------------	------	--------

e)												
SUB-01	Submission Service	Head of Editorial Workflow	submission-svc	ADR-014	POST /api/v1/submissions	services/submissions/handler.py	submissions	s3://opus-submissions/{yyyy}/{mm}/	CT-SUB-01	EVID-SUB-01-2026Q3	CERT-RC1-SUB-01	Traced
SUB-02	Review Service	Head of Editorial Workflow	review-svc	ADR-014	POST /api/v1/reviews	services/review/handler.py	reviews, review_assgnments	s3://opus-submissions/blinded/	CT-SUB-02	EVID-SUB-02-2026Q3	CERT-RC1-SUB-02	Traced
SUB-03	Decision Service	Head of Editorial Workflow	decision-svc	ADR-014	POST /api/v1/decisions	services/decision/handler.py	decisions	—	CT-SUB-03	EVID-SUB-03-2026Q3	CERT-RC1-SUB-03	Traced
SUB-04	Composition / Production Service	Head of Production Engineering	composition-svc	ADR-019	POST /api/v1/compositions	services/composition/orchestrator.py	composition_jobs	—	CT-SUB-04	EVID-SUB-04-2026Q3	CERT-RC1-SUB-04	Traced
SUB-05	Canonical Freezer	Principal Architect	canonical-freezer-svc	ADR-019	POST /api/v1/canonical/freezer	services/canonical/freezer.py	canonical_records	s3://opus-canonical/{article_id}/	CT-SUB-05	EVID-SUB-05-2026Q3	CERT-RC1-SUB-05	Traced
SUB-06	PDF Generator	Head of Production Engineering	pdf-generator-svc	ADR-020	POST /api/v1/render/pdf	services/render/pdf.py	render_jobs	s3://opus-artifacts/pdf/{sha256}.pdf	CT-SUB-06	EVID-SUB-06-2026Q3	CERT-RC1-SUB-06	Traced
SUB-07	HTML Renderer	Head of Production Engineering	html-renderer-svc	ADR-020	POST /api/v1/render/html	services/render/html.py	render_jobs	s3://opus-artifacts/html/{sha25	CT-SUB-07	EVID-SUB-07-2026Q3	CERT-RC1-SUB-07	Traced

								6}.html				
SUB-08	JATS Generator	Head of Production Engineering	jats-generator-svc	ADR-020	POST /api/v1/renders/jats	services/renders/jats.py	renders_jobs	s3://opus-artifacts/jats/{sha256}.xml	CT-SUB-08	EVID-SUB-08-2026Q3	CERT-RC1-SUB-08	Traced
SUB-09	Artifact Verifier	Principal Architect	artifact-verifier-svc	ADR-019	POST /api/v1/artifacts/verify	services/artifact-verifier.py	artifact_verifications	—	CT-SUB-09	EVID-SUB-09-2026Q3	CERT-RC1-SUB-09	Traced
SUB-10	DOI Minter	Head of Platform Engineering	doi-minter-svc	ADR-021	POST /api/v1/doi/mint	services/doi/minter.py	identifiers	—	CT-SUB-10	EVID-SUB-10-2026Q3	CERT-RC1-SUB-10	Traced
SUB-11	Crossref Depositor	Head of Platform Engineering	crossref-depositor-svc	ADR-021	POST /api/v1/crossref/deposit	services/crossref/depositor.py	crossref_deposits	—	CT-SUB-11	EVID-SUB-11-2026Q3	CERT-RC1-SUB-11	Traced
SUB-12	ORCID Integrator	Head of Platform Engineering	orcid-integrator-svc	ADR-022	POST /api/v1/orcid/writeback	services/orcid/integrator.py	orcid_writebacks	—	CT-SUB-12	EVID-SUB-12-2026Q3	CERT-RC1-SUB-12	Traced
SUB-13	Preservation Depositor	Head of Platform Engineering	preservation-depositor-svc	ADR-023	POST /api/v1/preservation/deposit	services/preservation/depositor.py	preservation_deposits	s3://opus-preservation/{backend}/{article_id}/	CT-SUB-13	EVID-SUB-13-2026Q3	CERT-RC1-SUB-13	Traced
SUB-14	OAI-PMH Endpoint	Head of Platform	oai-pmh-svc	ADR-024	GET /oai-pmh?	services/oai-pmh/	oai_records	—	CT-SUB-14	EVID-SUB-14	CERT-RC1-SUB-14	Traced

	nt	m Engine ering			verb=L istRec ords	handle r.py				2026Q 3	4	
SUB-1 5	SWOR D Endpoi nt	Head of Platfor m Engine ering	sword- svc	ADR-0 25	POST / sword/ collecti on/{co l}	service s/ sword/ handle r.py	sword_ depo sits	—	CT- SUB-1 5	EVID- SUB-1 5- 2026Q 3	CERT- RC1- SUB-1 5	Traced
SUB-1 6	Audit Service	Princip al Archit ect	audit- svc	ADR-0 08	POST / api/v1 /audit/ appen d	service s/ audit/ ledger. py	audit_ log	s3:// opus- audit/ wal/ {yyyy}/ {mm}/	CT- SUB-1 6	EVID- SUB-1 6- 2026Q 3	CERT- RC1- SUB-1 6	Traced
SUB-1 7	Notific ation Service	Head of Platfor m Engine ering	notific ation- svc	ADR-0 26	POST / api/v1 /notifi cations	service s/ notific ation/ dispatc her.py	notific ations, email_ log	—	CT- SUB-1 7	EVID- SUB-1 7- 2026Q 3	CERT- RC1- SUB-1 7	Traced
SUB-1 8	Search / Index Service	Head of Produc tion Engine ering	search -index- svc	ADR-0 27	POST / api/v1 /searc h/inde x	service s/ search / indexe r.py	search _index	s3:// opus- search / snapsh ots/	CT- SUB-1 8	EVID- SUB-1 8- 2026Q 3	CERT- RC1- SUB-1 8	Traced
SUB-1 9	Identit y / Auth Service	Head of Platfor m Engine ering	identit y- auth- svc	ADR-0 07	POST / api/v1 /auth/ token	service s/ identit y/ token. py	users, sessio ns, roles, api_ke ys	—	CT- SUB-1 9	EVID- SUB-1 9- 2026Q 3	CERT- RC1- SUB-1 9	Traced
SUB-2 0	Storag e Service	Head of Data Engine ering	storag e-svc	ADR-0 09	PUT /api/v 1/stor age/ob jects	service s/ storag e/ object _store. py	storag e_obje cts, storag e_rece ipts	s3:// opus- {submi ssions, canoni cal,arti facts,p reserv ation,a udit}/	CT- SUB-2 0	EVID- SUB-2 0- 2026Q 3	CERT- RC1- SUB-2 0	Traced

Reading the matrix. Every row resolves a thirteen-node chain: the subsystem ID is the requirement; the Architecture column names the deploying service process; the ADR column points to the architecture decision that justifies the service decomposition; the Primary API column is the contract surface exposed to other subsystems and to clients; the Code Module column is the repository path that implements the contract; the Database and Storage columns name the durable state the subsystem owns; the Contract Test column names the test suite that verifies the contract at every release; the Evidence and Cert columns link to the runtime evidence bundle and the Release Certification Authority sign-off. A row whose Status is Traced has all thirteen nodes present, current, and certified. A row with Status Gap has at least one missing node and SHALL be remediated before the next release.

Authority. Every subsystem in this matrix is owned by a single accountable Head of Engineering, recorded in the Owner column. Ownership SHALL NOT be shared: where two teams contribute to a subsystem, one team is the contractual owner and the other is a contributor. This rule preserves traceability: a contract with two owners is a contract with no owner, because no single authority can be held accountable when the contract is breached. The Owner column is therefore normative for on-call routing, for incident escalation, and for the certification sign-off chain.

Coverage. The matrix covers all twenty subsystems enumerated in Volume II, Chapter 27, section 27.2. No subsystem deployed in production is absent from this matrix; no subsystem absent from this matrix is deployed in production. This invariant is verified nightly by the static integrity scanner described in Chapter 2, section 2.6, which compares the running service catalogue against this matrix and raises a SEV-2 alert on any discrepancy.

6.3 Contract Dependency Graph

Subsystem contracts are not independent: every subsystem consumes the contracts of other subsystems and produces contracts consumed by others. The dependency graph below traces those inter-subsystem relationships. The Integration Pattern column records how the dependency is realized at runtime (synchronous REST call, asynchronous event subscription, shared database read, or storage-mediated handoff). The Failure Impact column records what propagates across the boundary when the depended-upon subsystem fails. The graph is read top-to-bottom: a subsystem in the leftmost column depends on the subsystems in the second column and is depended upon by the subsystems in the third column.

Subsystem	Depends On	Depended By	Integration Pattern	Failure Impact
SUB-01 Submission Service	SUB-19 Identity/Auth (authn); SUB-20 Storage (file persistence); SUB-16 Audit (transition log); SUB-17 Notification (receipt email)	SUB-02 Review; SUB-03 Decision; SUB-10 DOI Minter; SUB-18 Search	Sync REST + event publish	Storage outage degrades submission save; Auth outage blocks intake; Audit outage blocks submission write.
SUB-02 Review Service	SUB-19 Identity/Auth; SUB-20 Storage	SUB-03 Decision	Sync REST + event publish	Storage outage blocks blinded file access;

	(blinded files); SUB-16 Audit; SUB-17 Notification; SUB-01 Submission (manuscript files)			Auth outage blocks reviewer actions.
SUB-03 Decision Service	SUB-02 Review (supporting reviews); SUB-19 Identity/Auth (editor authz); SUB-16 Audit; SUB-17 Notification; SUB-01 Submission (state update)	SUB-04 Composition; SUB-13 Preservation (retraction)	Sync REST + event publish	Review outage delays decision; Auth outage blocks decision write.
SUB-04 Composition / Production Service	SUB-06 PDF Generator; SUB-07 HTML Renderer; SUB-08 JATS Generator; SUB-09 Artifact Verifier; SUB-20 Storage; SUB-16 Audit	SUB-05 Canonical Freezer (preview accepted event)	Async event fan-out + sync orchestration	Generator outage halts composition; Storage outage halts output persistence.
SUB-05 Canonical Freezer	SUB-09 Artifact Verifier (mandatory precondition, INV-V-01); SUB-20 Storage (read-only canonical); SUB-16 Audit	SUB-06 PDF; SUB-07 HTML; SUB-08 JATS; SUB-10 DOI Minter; SUB-13 Preservation	Sync REST + event publish	Verifier outage blocks freeze (mandatory); Storage outage blocks canonical persistence.
SUB-06 PDF Generator	SUB-05 Canonical Freezer (frozen source); SUB-20 Storage (output); SUB-16 Audit	SUB-04 Composition; SUB-09 Artifact Verifier	Async event subscribe + sync REST output	Freezer outage blocks PDF generation; Storage outage blocks output persistence.
SUB-07 HTML Renderer	SUB-05 Canonical Freezer; SUB-20 Storage; SUB-16 Audit	SUB-04 Composition; SUB-09 Artifact Verifier; SUB-18 Search/Index	Async event subscribe + sync REST output	Freezer outage blocks HTML rendering; Storage outage blocks output persistence.
SUB-08 JATS Generator	SUB-05 Canonical Freezer; SUB-20 Storage; SUB-16 Audit	SUB-04 Composition; SUB-09 Artifact Verifier; SUB-18 Search/Index	Async event subscribe + sync REST output	Freezer outage blocks JATS generation; Storage outage blocks output persistence.
SUB-09 Artifact Verifier	SUB-20 Storage (artifact fetch); SUB-16 Audit	SUB-05 Canonical Freezer (precondition); SUB-06 PDF; SUB-07 HTML; SUB-08 JATS	Sync REST	Storage outage blocks verification; the Freezer refuses to freeze unverified artifacts.

SUB-10 DOI Minter	SUB-19 Identity/Auth (operator authz); SUB-16 Audit; Configuration (journal_doi_prefix)	SUB-11 Crossref Depositor; SUB-14 OAI-PMH; SUB-18 Search/Index; SUB-05 Canonical Freezer (minted DOI on canonical)	Sync REST + event publish	Audit outage blocks mint; DOI prefix misconfiguration produces orphan DOIs.
SUB-11 Crossref Depositor	SUB-10 DOI Minter (DOI to deposit); SUB-16 Audit; External Crossref REST	SUB-13 Preservation (deposit metadata); SUB-14 OAI-PMH	Async event subscribe + external REST	DOI Minter outage blocks deposit; Crossref outage queues deposits in outbox (durable).
SUB-12 ORCID Integrator	SUB-19 Identity/Auth (author auth); SUB-16 Audit; External ORCID REST	SUB-17 Notification (writeback result)	Async event subscribe + external REST	ORCID outage queues writebacks; Auth outage blocks author-issued writebacks.
SUB-13 Preservation Depositor	SUB-05 Canonical Freezer (artifact set); SUB-10 DOI Minter (deposit metadata); SUB-11 Crossref Depositor (deposit metadata); SUB-16 Audit; SUB-20 Storage	SUB-14 OAI-PMH; SUB-17 Notification (receipt)	Async event subscribe + multi-backend sync REST	Freezer outage blocks preservation; backend outage fails one backend but not others (multi-backend policy, INV-P-02).
SUB-14 OAI-PMH Endpoint	SUB-05 Canonical Freezer; SUB-10 DOI Minter; SUB-11 Crossref Depositor; SUB-13 Preservation Depositor; SUB-16 Audit	External harvesters (Portico, DOAJ, OpenAIRE, LOCKSS)	Read-only sync REST; subscription to publication events	Freezer/DOI/Crossref outage degrades OAI-PMH record completeness.
SUB-15 SWORD Endpoint	SUB-01 Submission Service (intake adapter); SUB-19 Identity/Auth; SUB-16 Audit	External repository managers (institutional repositories)	Sync REST (SWORD v2 protocol)	Submission outage blocks SWORD intake; Auth outage blocks depositor authn.
SUB-16 Audit Service	SUB-20 Storage (write-ahead log); INV-A-03 hash chain invariant	Every other subsystem (universal consumer)	Sync append-only REST; consumed by all subsystems	Storage outage blocks all writes (mandatory co-transaction per INV-A-01). The Audit Service SHALL NOT degrade.
SUB-17 Notification Service	SUB-16 Audit; External SMTP/SES; External push providers	SUB-01 Submission; SUB-02 Review; SUB-03 Decision;	Async event subscribe + external delivery	SMTP outage queues notifications; Audit outage blocks

		SUB-11 Crossref; SUB-12 ORCID; SUB-13 Preservation		notification logging.
SUB-18 Search / Index Service	SUB-05 Canonical Freezer; SUB-06 PDF; SUB-07 HTML; SUB-08 JATS; SUB-10 DOI Minter; SUB-16 Audit	Search UI; External discovery	Async event subscribe + bulk reindex sync REST	Generator outages degrade index freshness; storage outage degrades search service.
SUB-19 Identity / Auth Service	External IdP (SAML/OIDC); SUB-16 Audit	Every other subsystem (universal authentication)	Sync REST (token issuance, verification)	IdP outage blocks new sessions; Auth outage blocks every authenticated operation platform- wide (SEV-1).
SUB-20 Storage Service	External object store (S3-compatible); SUB-16 Audit (storage receipt log)	Every subsystem with artifacts (SUB-01, SUB-04..08, SUB-13, SUB-16, SUB-18)	Sync REST + signed URL issuance	Object store outage blocks every artifact write platform-wide (SEV-1). The Storage Service is the highest- criticality dependency in the graph.

Certification depth. Subsystem criticality, recorded in the Owner and Status columns, drives the certification depth that the Release Certification Authority applies. Critical subsystems (SUB-01, SUB-02, SUB-03, SUB-04, SUB-05, SUB-08, SUB-09, SUB-10, SUB-11, SUB-13, SUB-16, SUB-19, SUB-20) undergo full five-pillar certification (Runtime, Visual, Database, Storage, API) at every release, and their contract test suites SHALL be exercised against production-equivalent infrastructure in the staging environment before any code reaches production. High-criticality subsystems (SUB-06, SUB-07, SUB-12, SUB-14, SUB-17, SUB-18) undergo the same five-pillar certification but with reduced test-data volume; their contract suites MAY be exercised against a reduced-scale fixture. Standard-criticality subsystems (SUB-15) undergo three-pillar certification (Runtime, Database, API) on each release and full five-pillar certification on each quarterly boundary. A subsystem's criticality SHALL NOT be downgraded without an Architecture Decision Record (ADR) approved by the Principal Architect and a corresponding amendment to this Matrix recorded in the Engineering Knowledge Graph.

ADR coverage. Every Architecture Decision Record cited in the ADR column of section 6.2 SHALL be present, current, and approved in the ADR repository (Volume I, Chapter 7). An ADR with status Proposed or Rejected does not justify a deployed subsystem; the static integrity scanner SHALL flag any subsystem whose ADR is not in status Approved or Superseded-and-Archived. Where a subsystem is justified by multiple ADRs (for example, the Identity/Auth Service is justified by ADR-007 for the SAML/OIDC integration and by ADR-030 for the audit-log coupling), the matrix records the primary ADR and the full list is available in the subsystem's full contract in Volume II, Chapter 27, section 27.3. The

ADR is the architectural reasoning; the matrix is the traceability link between that reasoning and the deployed code. The two together constitute the architectural accountability of the platform.

Contract testing cadence. Every Contract Test ID in section 6.2 names a test suite that SHALL run on every change to the owning subsystem's code and on every release certification cycle. The suites are run in three modes: fast (subset, run on every pull request, target p95 < 90 seconds); full (all cases, run on every merge to main, target p95 < 30 minutes); and certification (all cases against production-equivalent infrastructure, run before every release, target p95 < 4 hours). A test suite that does not pass in all three modes blocks the corresponding workflow stage. Flaky tests in a contract suite are handled per the Playbook's three-strikes policy (Playbook Chapter 6, section 6.5): a test that fails three times in 30 days is removed, the requirement it verified is marked Gap in this Matrix, and a remediation task is opened against the owning team.

Critical dependencies. The graph identifies four universal dependencies whose failure impacts every subsystem: SUB-19 Identity/Auth (authentication is required for every non-public operation), SUB-16 Audit (the append-only audit log is a mandatory co-transaction for every state-changing operation per INV-A-01), SUB-20 Storage (artifact persistence is required by every subsystem that owns artifacts), and the external object store (the durable substrate under SUB-20). These four dependencies are the platform's load-bearing walls; their SLAs (SUB-16 99.99%, SUB-19 99.9%, SUB-20 99.99%) are correspondingly tighter than the platform-wide 99.9% baseline.

Cyclic dependencies. The graph contains no cycles: the dependency topology is a directed acyclic graph (DAG) rooted at the external identity provider and the external object store. The Engineering Playbook (Chapter 3) prohibits the introduction of cyclic subsystem dependencies: any pull request that introduces a cycle in this graph SHALL be rejected by the static architecture linter (ADR-028). This invariant is essential because a cyclic dependency would prevent independent deployment and would create unrecoverable deadlock conditions under partial outage.

Integration patterns. The matrix records four integration patterns: synchronous REST (low-latency request/response, used for queries and for state-changing operations where the caller must know the outcome before proceeding); asynchronous event publish (used when the producer does not require the consumer's response, to decouple availability domains); shared database read (used only for read-only cross-subsystem queries against the publication-views schema, never for writes); and storage-mediated handoff (used by SUB-05 Canonical Freezer to publish the canonical artifact set to all consumers without coupling them to the freezer's runtime). Every cross-subsystem call in the platform SHALL use exactly one of these four patterns; ad-hoc integrations are forbidden and SHALL be rejected at code review.

Failure propagation. The Failure Impact column makes the failure-mode contract explicit for every dependency edge. This column is the operational contract between subsystems: it tells the on-call engineer what to expect when a dependency is degraded. The Playbook Chapter 8 (Runtime Invariant Monitoring) operationalizes these failure modes as monitors: every dependency edge in this graph has a corresponding circuit breaker and a corresponding alert. A dependency that fails without raising an alert is a monitor defect and is itself a SEV-2 incident.

Blast-radius containment. The dependency graph is the substrate for the platform's blast-radius containment strategy. Where a dependency is classified as Critical in the Owner column of section 6.2, the depending subsystem SHALL implement an explicit circuit breaker that trips after a documented failure threshold (per ADR-032) and a documented fallback behaviour. The fallback behaviour is recorded in the Failure Impact column of this graph and is exercised in chaos-engineering drills per the Playbook Chapter 12 cadence. Subsystems that do not have a safe fallback (for example, the Audit Service cannot fall back because its co-transaction invariant is mandatory) SHALL be designed to fail closed: the depending subsystem refuses the write rather than proceeding without the audit record. This discipline ensures that a partial outage never produces an inconsistent record.

Dependency review at code review. The graph is enforced at code-review time: a pull request that introduces a new cross-subsystem call (a new import, a new REST call, a new event subscription) SHALL cite the dependency edge in this graph or SHALL propose an amendment to the graph via an ADR. The static architecture linter (ADR-028) verifies that every cross-subsystem call in the codebase is reflected in this graph; any undocumented call is a SEV-2 traceability defect and blocks the pull request. This enforcement is essential because a dependency that exists in the code but not in the graph is a dependency that has no documented failure mode, no circuit breaker, and no monitor - it is a silent dependency that will fail in unexpected ways at the worst possible time.

Chapter 7 — Canonical Field Registry Traceability

7.1 Purpose

This chapter traces the canonical field registry established by Volume II, Chapter 28 (Canonical Field Registry) through the full thirteen-node traceability chain. The canonical field registry is the platform's data constitution: it enumerates every field that is part of the platform's canonical data model, assigns each field to exactly one owning service, specifies the field's database location, the storage path of any out-of-line value, the validation rule that the field SHALL satisfy, the migration strategy if the field's representation must change, and the test that verifies the field's contract at runtime.

The registry contains two hundred seventy-six fields distributed across fourteen domains. The full registry is too large to be traced as a single matrix; section 7.2 provides a domain-level summary of all fourteen domains and section 7.3 traces the thirty-two critical fields whose correctness is essential to the integrity of the scholarly record. A critical field is one whose corruption would compromise an invariant (Volume II, Chapter 26), a non-negotiable rule (Volume I, Chapter 3), or a public-facing scholarly record obligation (DOI persistence, Crossref deposit, ORCID attribution, long-term preservation, OAI-PMH discoverability). The remaining two hundred forty-four fields are governed by the domain contracts in Volume II, Chapter 28, section 28.4, and are traced through their owning domain rather than individually.

This chapter is normative. A critical field whose chain contains a GAP SHALL NOT be released to production; the Release Certification Authority SHALL refuse certification. A critical field whose evidence is STALE SHALL be re-evidenced before the next release. The canonical field registry is the substrate of every other traceability chapter: subsystem contracts (Chapter 6) own fields, events (Chapter 8) carry field values across the wire, and the invariants (Volume II, Chapter 26) are constraints on field values. A defect in the field registry is therefore a defect in every downstream trace.

Migration discipline. Field representations change over time: enumerations gain values, validation rules tighten, storage paths are restructured. Section 7.4 traces the migration status of every critical field. A migration that is in progress without a recorded migration strategy is an undocumented migration and is a SEV-2 traceability defect. A migration that has not been exercised against production data within the last twelve months is considered unverified and SHALL be re-exercised against the current production snapshot before the next release.

7.2 Domain Summary

The canonical field registry is partitioned into fourteen domains. Each domain is owned by exactly one service, persisted in exactly one database schema, and has exactly one storage profile. The table below summarizes the field count, owning service, database schema, storage profile, and traceability status of each domain. The status column reflects the domain's overall traceability posture: Traced means every critical field in the domain has a complete chain; Gap means at least one critical field has an open remediation; Watch means the domain has a pending schema change that will require re-tracing.

Domain	Field Count	Owning Service	Database	Storage	Status
Article	45	Publication Service	articles	s3://opus-artifacts/{pdf,html,jats}/	Traced
Author	28	Submission Service	authors, author_affiliations	s3://opus-submissions/authors/{orcid}/	Traced
Journal	27	Configuration Service	journals, journal_licenses	s3://opus-config/journals/{issn}/	Traced
Issue	16	Publication Service	issues, issue_articles	—	Traced
Submission	22	Submission Service	submissions, submission_files	s3://opus-submissions/{yyyy}/{mm}/	Traced
Review	24	Review Service	reviews, reviewer_assignments, coi_declarations	s3://opus-submissions/blinded/	Traced
Decision	12	Decision Service	decisions	—	Traced
Version	12	Publication Service	article_versions, version_diffs	s3://opus-artifacts/versions/	Traced
Identifier	16	DOI Minter	identifiers, crossref_deposits	—	Gap
Preservation	11	Preservation Depositor	preservation_deposits, preservation_receipts	s3://opus-preservation/{backend}/{article_id}/	Traced
Audit	16	Audit Service	audit_log, audit_correlations	s3://opus-audit/wal/{yyyy}/{mm}/	Traced
User / Role	13	Identity/Auth Service	users, roles, user_roles, api_keys	—	Traced
Storage	12	Storage Service	storage_objects, storage_receipts	s3://opus-{bucket}/	Traced
Configuration	12	Configuration Service	configuration, feature_flags	—	Watch

Domain field counts sum to two hundred seventy-six, matching the registry in Volume II, Chapter 28, section 28.2. Two domains carry non-Traced statuses. The Identifier domain is in Gap status because the doi_registration_agency field (row 26 of section 7.3) does not yet support the DataCite registration

agency enumerated value; this is the platform's only open traceability gap for a critical field and is scheduled for remediation in release RC1.3 (target Q4 2026). The Configuration domain is in Watch status because the `feature_flags` table schema is under review (ADR-029) for the introduction of typed-flag values; the schema change is non-breaking but every existing flag will require re-tracing once ADR-029 is approved.

Domain ownership is exclusive: no field appears in two domains. Where a field is read by multiple subsystems (for example, `article_doi` is read by the Crossref Depositor, the OAI-PMH Endpoint, and the Search/Index Service), the field has one owning service (the DOI Minter) and multiple consuming services. The owning service is the only service permitted to write the field; consuming services read it through the `publication-views` schema (a read-only, denormalized projection owned by the Publication Service) or through the owning service's published API. Direct cross-subsystem writes to a field not owned by the writer are a SEV-1 traceability violation.

7.3 Critical Field Traceability Matrix

The matrix below traces the thirty-two critical fields. The columns follow the standard thirteen-node traceability chain specialized for data fields: the Field column names the canonical field; the Domain column places it in its owning domain; the DB Table and DB Column columns name the persistent location; the Storage Path column names the out-of-line storage location (or “—” for inline-only fields); the Producer and Consumer columns name the services that write and read the field; the Validation column summarizes the rule the field SHALL satisfy; the Migration column records the migration strategy; the Test, Evidence, Owner, and Status columns complete the chain.

Each row corresponds to one critical field enumerated in Volume II, Chapter 28, section 28.3. The thirty-two fields are presented in the registry's canonical order: Article-domain fields first (rows 1-8), Author-domain fields next (rows 9-11), Journal-domain fields (rows 12-13), Review-domain fields (rows 14-16), Submission-domain field (row 17), Version-domain field (row 18), cross-cutting fields (rows 19-21), Identifier-domain fields (rows 22-26), Version-domain field (row 27), retraction field (row 28), Decision-domain field (row 29), Review-domain field (row 30), and the canonical-rendering fields (rows 31-32).

Field	Domain	DB Table	DB Column	Storage Path	Producer	Consumer	Validation	Migration	Test	Evidence	Owner	Status
<code>article_doi</code>	Identifier	<code>articles</code>	<code>article_doi</code>	—	DOI Minter (SUB-10)	Crossref Depositor; OAI-PMH; Search/Index	<code>^10\.\d{4,9}\/.+\$; UNIQUE; NOT NULL post-publication</code>	M2024-08 (varchar(255) widening)	TEST-FLD-001	EVID-FLD-001	Head of Platform Eng	Traced
<code>article</code>	Article	<code>article</code>	<code>article</code>	<code>s3://</code>	JATS	Artifact	JATS	M2025	TEST-	EVID-	Head	Traced

_jats_xml		s	_jats_xml_artifact_id	opus-artifacts/jats/{sha256}.xml	Generator (SUB-08)	t Verifier; Search/Index; Preservation	1.3d2022 DTD conformance; immutable post-freeze	-02 (DTD upgrade 1.3 -> 1.3d2022)	FLD-002	FLD-002	of Production Eng	
article_pdf_published	Article	articles	article_pdf_artifact_id	s3://opus-artifacts/pdf/{sha256}.pdf	PDF Generator (SUB-06)	OAI-PMH; Preservation; Crossref (link)	PDF/A-2u conformance; immutable post-freeze	M2025-02 (PDF/A-1b -> PDF/A-2u)	TEST-FLD-003	EVID-FLD-003	Head of Production Eng	Traced
article_html_canonical	Article	articles	article_html_artifact_id	s3://opus-artifacts/html/{sha256}.html	HTML Renderer (SUB-07)	Search/Index; OAI-PMH	HTML5 + schema.org; immutable post-freeze	M2025-02 (stylesheet refactor)	TEST-FLD-004	EVID-FLD-004	Head of Production Eng	Traced
article_state	Article	articles	article_state	—	Decision Service (SUB-03); Publication Service	All subsystems (publication-views)	ENUM: Drafted, Submitted, Accepted, Rejected, RevisionRequested, Published, Retracted; INV-C-07 transition	M2024-11 (Retracted state added)	TEST-FLD-005	EVID-FLD-005	Head of Editorial Workflow	Traced

							check					
article_title	Article	articles	article_title	—	Submission Service (SUB-01)	All subsystems (publications-views)	non-empty; max 1024 chars; Unicode NFC-normalized; no control chars	M2024-08 (initial schema)	TEST-FLD-006	EVID-FLD-006	Head of Editorial Workflow	Traced
article_publication_date	Article	articles	article_publication_date	—	Publication Service	Crossref Depositor; OAI-PMH; Search/Index	ISO 8601 date; not after today; not before 1990-01-01	M2024-08 (initial schema)	TEST-FLD-007	EVID-FLD-007	Head of Platform Eng	Traced
article_license	Article	articles	article_license_url	—	Submission Service (SUB-01)	Crossref Depositor; OAI-PMH; Search/Index	CC-BY-4.0 / CC-BY-SA-4.0 / CC-BY-NC-4.0 / CC-BY-ND-4.0 / CC0 / custom URL; URL resolves	M2024-10 (CC0 added)	TEST-FLD-008	EVID-FLD-008	Head of Editorial Workflow	Traced
author_orcid_id	Author	authors	author_orcid_id	—	Identity/Auth Service (SUB-19)	ORCID Integrator; Crossref Depositor	16-digit ORCID with ISO 7064	M2025-01 (checksum validator)	TEST-FLD-009	EVID-FLD-009	Head of Platform Eng	Traced

						tor; Search /Index	checks um; nullabl e for non- orcid author s	added)				
author _full_n ame	Author	author s	author _full_n ame	—	Submi ssion Service (SUB-0 1)	All subsys tems (public ation- views)	non- empty; max 256 chars; Unicod e NFC- normal ized; no contro l chars	M2024 -08 (initial schem a)	TEST- FLD-01 0	EVID- FLD-01 0	Head of Editori al Workfl ow	Traced
author _email	Author	author s	author _email	—	Submi ssion Service (SUB-0 1)	Notific ation Service ; ORCID Integra tor; Identit y/Auth	RFC 5322 mailbo x; deliver able (MX verific ation on captur e)	M2024 -08 (initial schem a)	TEST- FLD-01 1	EVID- FLD-01 1	Head of Editori al Workfl ow	Traced
journal _issn_ electro nic	Journa l	journal s	journal _issn_ electro nic	—	Config uratio n Service	Crossr ef Deposi tor; OAI- PMH; Search /Index	8-digit ISSN with ISO 7064 checks um; NOT NULL	M2024 -08 (initial schem a)	TEST- FLD-01 2	EVID- FLD-01 2	Head of Platfor m Eng	Traced
journal _doi_p refix	Journa l	journal s	journal _doi_p refix	—	Config uratio n Service	DOI Minter (SUB-1 0); Crossr	^10\ .\d{4,6} \$; Crossr ef-	M2024 -08 (initial schem a)	TEST- FLD-01 3	EVID- FLD-01 3	Head of Platfor m Eng	Traced

						ef Deposi tor	assign ed prefix; NOT NULL					
review _state	Review	reviews	review _state	—	Review Service (SUB-02)	Decision Service ; Audit Service ; Notification	ENUM: Invited , Accepted, InProgress, Submitted, Withdrawn, Declined; INV-C-07 transition check	M2024 -11 (Declined state added)	TEST- FLD-01 4	EVID- FLD-01 4	Head of Editorial Workflow	Traced
review _coi_d eclare d	Review	reviews	review _coi_d eclare d_at	—	Review Service (SUB-02)	Audit Service ; Decision Service	NOT NULL before assignment is active (INV-L-03); timestamp UTC	M2024 -11 (made mandatory per INV-L-03)	TEST- FLD-01 5	EVID- FLD-01 5	Head of Editorial Workflow	Traced
review _reviewer_id	Review	reviews	reviewer_user_id	—	Review Service (SUB-02)	Audit Service ; Identity/Auth	FK -> users. user_id; NOT NULL; immutable post-assignment	M2024 -08 (initial schema)	TEST- FLD-01 6	EVID- FLD-01 6	Head of Editorial Workflow	Traced
submis	Submi	submis	submis	—	Submi	All	ENUM:	M2024	TEST-	EVID-	Head	Traced

sion_state	ssion	sions	sion_state		ssion Service ; Decision Service ; Publication Service	subsys tems (public ation-views)	Drafted, Submitted, InReview, RevisionRequested, Accepted, Rejected, Published, Retracted; INV-C-07 transition check	-11 (Retracted state added)	FLD-017	FLD-017	of Editorial Workflow	
publication_timestamp	Version	articles	article_published_at	—	Publication Service	Audit Service ; Crossrefer Depositor; OAI-PMH	UTC timestamp; monotonic (INV-V-01); not before canonical_frozen_timestamp	M2024-08 (initial schema)	TEST-FLD-018	EVID-FLD-018	Head of Platform Eng	Traced
artifact_sha256	Preservation	storage_objects	storage_object_sha256	—	Storage Service (SUB-20)	Artifact Verifier; Audit Service ; Preservation	64 hex chars; SHA-256 of stored content (INV-P-01); verified on read	M2024-08 (initial schema)	TEST-FLD-019	EVID-FLD-019	Head of Data Eng	Traced

audit_chain_hash	Audit	audit_log	audit_chain_hash	—	Audit Service (SUB-16)	Audit Service (self-verify); on-call (alert on break)	SHA-256(pre_v_hash canonical_record); INV-A-03; non-null; monotonic chain	M2024-09 (chain reseeded after ADR-030)	TEST-FLD-020	EVID-FLD-020	Principal Architect	Traced
audit_correlation_id	Audit	audit_log	audit_correlation_id	—	Audit Service (SUB-16)	All subsystems (distributed trace lookup)	UUID v4; NOT NULL; indexed (correlation lookup)	M2024-08 (initial schema)	TEST-FLD-021	EVID-FLD-021	Principal Architect	Traced
identifier_value	Identifier	identifiers	identifier_value	—	DOI Minter (SUB-10)	Crossref Depositor; OAI-PMH; Search/Index	non-empty; format per identifier_type; UNIQUE per (article_id, identifier_type)	M2024-08 (initial schema)	TEST-FLD-022	EVID-FLD-022	Head of Platform Eng	Traced
identifier_type	Identifier	identifiers	identifier_type	—	DOI Minter (SUB-10)	Crossref Depositor; OAI-PMH; Search/Index	ENUM: DOI, PMID, PMCID, Handle, URL	M2024-11 (PMCID added)	TEST-FLD-023	EVID-FLD-023	Head of Platform Eng	Traced

preservation_receipt_id	Preservation	preservation_deposits	preservation_receipt_id	—	Preservation Depositor (SUB-13)	Audit Service ; Notification Service	non-null post-successes; backend-issued; UNIQUE per (deposit_id, backend)	M2025-03 (multi-backend policy)	TEST-FLD-024	EVID-FLD-024	Head of Platform Eng	Traced
crossref_deposit_id	Identifier	crossref_deposits	crossref_deposit_id	—	Crossref Depositor (SUB-11)	Audit Service ; Publication Service	non-null post-successes; Crossref-issued; UNIQUE	M2024-09 (initial schema)	TEST-FLD-025	EVID-FLD-025	Head of Platform Eng	Traced
doi_registration_agency	Identifier	identifiers	registration_agency	—	DOI Minter (SUB-10)	Crossref Depositor; OAI-PMH	ENUM: Crossref; (DataCite pending)	M2025-04 (DataCite enumeration planned, ADR-031)	TEST-FLD-026	EVID-FLD-026	Head of Platform Eng	Gap
version_parent_id	Version	article_versions	version_parent_id	—	Publication Service	Search/Index; OAI-PMH	FK -> article.s.article_id; nullable for version 1; immutable post-insert	M2024-08 (initial schema)	TEST-FLD-027	EVID-FLD-027	Head of Platform Eng	Traced

retraction_notice_id	Article	retractions	retraction_notice_article_id	s3://opus-artifacts/retractions/{article_id}.pdf	Decision Service (SUB-03)	Crossref Depositor; OAI-PMH; Preservation Depositor	FK -> article.s.article_id; UNIQUE; one-to-one with retractions table	M2024-12 (retraction model introduced)	TEST-FLD-028	EVID-FLD-028	Head of Editorial Workflow	Traced
editorial_decision	Decision	decisions	decision_type	—	Decision Service (SUB-03)	Submission Service; Notification; Audit	ENUM: Accept, Minor Revision, Major Revision, Reject, Retract; INV-C-07 transition check	M2024-11 (Retract added)	TEST-FLD-029	EVID-FLD-029	Head of Editorial Workflow	Traced
reviewer_assignment_id	Reviewer	reviewer_assignments	assignment_id	—	Reviewer Service (SUB-02)	Decision Service; Audit Service	UUID v4; UNIQUE per (submission_id, reviewer_user_id); immutable	M2024-08 (initial schema)	TEST-FLD-030	EVID-FLD-030	Head of Editorial Workflow	Traced
canonical_freeze_timestamp	Version	canonical_records	canonical_freeze_timestamp	—	Canonical Freezer (SUB-05)	All production subsystems; Publication Service	UTC timestamp; monotonic (INV-V-01); immutable	M2024-08 (initial schema)	TEST-FLD-031	EVID-FLD-031	Principal Architect	Traced

							able post- insert					
pdf_generation_completed_at	Article	render_jobs	render_completed_at	—	PDF Generator (SUB-06)	Publication Service ; Audit Service	UTC timestamp; nullable until completion; non-null when render_state=Completed	M2024-08 (initial schema)	TEST-FLD-032	EVID-FLD-032	Head of Production Eng	Traced

Open gap. Exactly one critical field carries Status Gap: doi_registration_agency (row 26). The field is currently constrained to the Crossref enumeration value, but the platform's commitment to support DataCite (recorded in ADR-031, scheduled for RC1.3) requires the enumeration to widen. The gap is not in the field's persistence or in its producer-consumer wiring; the gap is in the validation rule, which currently rejects the DataCite value. The remediation is in progress and the test TEST-FLD-026 has been updated to verify both enumeration values; the gap will close when ADR-031 is approved and the schema migration M2025-04 is executed against production.

Immutability invariants. The Validation column records several immutability invariants that are essential to the integrity of the scholarly record. article_jats_xml, article_pdf_published, article_html_canonical, canonical_frozen_timestamp, reviewer_assignment_id, version_parent_id, and audit_chain_hash are immutable post-insert (or post-freeze for the canonical-rendering fields). These immutability rules are enforced at three layers: the database layer (no UPDATE path in the owning service's data access object), the service layer (a defensive check at the API boundary that rejects UPDATE attempts with a 405 Method Not Allowed), and the audit layer (any attempted UPDATE on an immutable field is logged as a SEV-1 invariant violation). An engineer who modifies the database directly to bypass these layers is in violation of the Constitution and the violation is detectable through the audit chain's pre-image hash comparison.

Producer/consumer asymmetry. Every critical field has exactly one Producer (the owning service) but may have many Consumers. The asymmetry is essential: it ensures that writes are accountable to one service and one Head of Engineering, while reads can fan out across the platform. The publication-views schema (a read-only denormalized projection of the articles, authors, journals, and issues tables, owned by the Publication Service) is the platform's primary read path; consumers SHOULD read through the publication-views rather than through the owning service's API, because the views are denormalized for

read performance and because the views are versioned with the publication lifecycle. Exceptions to this rule are recorded in the field's full contract in Volume II, Chapter 28, section 28.3.

Validation coupling to invariants. Several critical fields carry validation rules that are direct operationalizations of invariants from Volume II, Chapter 26. `article_state` and `submission_state` carry the state-machine transition check from invariant INV-C-07. `audit_chain_hash` carries the cryptographic chain invariant INV-A-03. `canonical_frozen_timestamp` and `publication_timestamp` carry the monotonicity invariant INV-V-01. `review_coi_declared` carries the CoI-precondition invariant INV-L-03. `artifact_sha256` carries the content-addressed-storage invariant INV-P-01. `preservation_deposits` carry the multi-backend invariant INV-P-02. The validation rule in the field contract is the field-level expression of the invariant; the invariant monitor in the Audit Service (Playbook Chapter 8) is the system-level expression. Both layers SHALL hold; if either fails, the field's Status SHALL be marked Gap until both layers are restored.

Canonical field naming convention. The registry enforces a strict naming convention: every canonical field is named in snake_case with a domain prefix (`article_`, `author_`, `journal_`, `review_`, `submission_`, `identifier_`, `preservation_`, `audit_`, `version_`, `decision_`). The domain prefix is the field's owning domain (the first column of section 7.3) and is the key by which the field is grouped in the publication-views schema. Fields without a domain prefix (for example, internal configuration keys) are not canonical fields and are not traced in this Matrix; they are governed by the Configuration Service's local schema and are not part of the platform's published data model. The naming convention is enforced by a static linter (ADR-033) that rejects any pull request introducing a database column or a payload field whose name does not match the convention.

Producer authority boundary. The Producer column records the single service authorized to write the field. This boundary is enforced at three layers: the database layer (the owning service's database user is the only user with INSERT/UPDATE privileges on the field's column; other services have read-only privileges via the publication-views schema or via a dedicated read replica); the service layer (the owning service's API is the only API endpoint that accepts the field as a writeable parameter; the audit layer (every write to the field is recorded with the writer's service identity, and a write by an unexpected service identity is flagged as a SEV-1 unauthorized-write event)). The producer authority boundary is the operational form of the architectural principle that one field has one owner; the boundary is non-negotiable and SHALL NOT be relaxed for operational convenience. Direct database writes by operations staff are permitted only under the break-glass procedure (Playbook Chapter 14) and are logged as break-glass events in the audit trail.

Validation rule priority. When a field's validation rule conflicts with a producer's attempted write, the validation rule prevails. The producer's API returns a 422 Unprocessable Entity with the validation failure detail; the database write does not occur; the audit log records the rejected write attempt. A producer that bypasses its own validation rule (for example, by writing directly to the database) is in violation of the Constitution, the violation is detectable through the audit trail, and the offending party is subject to the Engineering Leadership review per Playbook Chapter 14, section 14.9. The validation rule is therefore

not advisory: it is normative, it is enforced at the API boundary, and violations are auditable. Where the validation rule is itself wrong (the rule is too strict, too lax, or incorrectly formulated), the rule SHALL be amended through a governed schema migration (section 7.4), not through a one-off override.

7.4 Field Migration Traceability

Fields change representation over time. The migration traceability matrix below records, for each critical field, the current schema version, the migration strategy that will be used to change the representation, the date of the most recent migration that was applied to production, the date of the next planned migration, and the owning service. The migration strategy is normative: any schema change that does not follow the recorded strategy is an undocumented migration and is a SEV-2 traceability defect. The strategies used by the platform are: expand-and-contract (add new column, backfill, switch reads, drop old column), enumeration widen (add new enum value, deploy, migrate consumers, constrain), in-place retype (one-shot migration under maintenance window), and shadow-write cutover (write to both old and new during overlap, then cut over reads).

Field	Current Version	Migration Strategy	Last Migration	Next Migration	Owner
article_doi	v1 (varchar 255)	In-place retype only on widening	M2024-08 (varchar(64) -> varchar(255))	None scheduled	Head of Platform Eng
article_jats_xml	v2 (artifact_id ref)	Expand-and-contract on DTD upgrade	M2025-02 (JATS 1.3 -> 1.3d2022)	None scheduled	Head of Production Eng
article_pdf_published	v2 (artifact_id ref)	Expand-and-contract on PDF/A upgrade	M2025-02 (PDF/A-1b -> PDF/A-2u)	None scheduled	Head of Production Eng
article_html_canonical	v2 (artifact_id ref)	Expand-and-contract on style refactor	M2025-02 (CSS refactor)	None scheduled	Head of Production Eng
article_state	v3 (8-state enum)	Enumeration widen	M2024-11 (Retracted added)	None scheduled	Head of Editorial Workflow
article_title	v1 (varchar 1024)	In-place retype only on widening	M2024-08 (initial)	None scheduled	Head of Editorial Workflow
article_publication_date	v1 (date)	In-place retype (low risk)	M2024-08 (initial)	None scheduled	Head of Platform Eng
article_license	v2 (license_url + license_type)	Expand-and-contract	M2024-10 (CC0 added)	None scheduled	Head of Editorial Workflow
author_orcid_id	v2 (with checksum)	Expand-and-contract on validator	M2025-01 (checksum added)	None scheduled	Head of Platform Eng

author_full_name	v1 (varchar 256)	In-place retype only on widening	M2024-08 (initial)	None scheduled	Head of Editorial Workflow
author_email	v1 (varchar 320)	In-place retype only on widening	M2024-08 (initial)	None scheduled	Head of Editorial Workflow
journal_issn_electronic	v1 (char 8)	In-place retype (low risk)	M2024-08 (initial)	None scheduled	Head of Platform Eng
journal_doi_prefix	v1 (varchar 16)	In-place retype only on widening	M2024-08 (initial)	None scheduled	Head of Platform Eng
review_state	v3 (6-state enum)	Enumeration widen	M2024-11 (Declined added)	None scheduled	Head of Editorial Workflow
review_coi_declared	v2 (mandatory)	Expand-and-contract	M2024-11 (made mandatory per INV-L-03)	None scheduled	Head of Editorial Workflow
review_reviewer_id	v1 (UUID FK)	In-place retype only on key change	M2024-08 (initial)	None scheduled	Head of Editorial Workflow
submission_state	v3 (8-state enum)	Enumeration widen	M2024-11 (Retracted added)	None scheduled	Head of Editorial Workflow
publication_timestamp	v1 (timestampz)	In-place retype (low risk)	M2024-08 (initial)	None scheduled	Head of Platform Eng
artifact_sha256	v1 (char 64)	In-place retype (low risk)	M2024-08 (initial)	None scheduled	Head of Data Eng
audit_chain_hash	v2 (post-reseed)	Shadow-write cutover on hash algo change	M2024-09 (chain reseeded after ADR-030)	None scheduled	Principal Architect
audit_correlation_id	v1 (UUID v4)	In-place retype only on UUID version change	M2024-08 (initial)	None scheduled	Principal Architect
identifier_value	v1 (varchar 255)	In-place retype only on widening	M2024-08 (initial)	None scheduled	Head of Platform Eng
identifier_type	v2 (5-value enum)	Enumeration widen	M2024-11 (PMCID added)	None scheduled	Head of Platform Eng
preservation_receipt_id	v2 (multi-backend)	Expand-and-contract on backend add	M2025-03 (multi-backend policy)	M2026-01 (CLOCKSS backend add)	Head of Platform Eng
crossref_deposit_id	v1 (varchar 64)	In-place retype only on widening	M2024-09 (initial)	None scheduled	Head of Platform Eng
doi_registration_agency	v1 (Crossref only)	Enumeration widen	M2024-08 (initial)	M2025-04 (DataCite added, ADR-031)	Head of Platform Eng

version_parent_id	v1 (UUID FK)	In-place retype only on key change	M2024-08 (initial)	None scheduled	Head of Platform Eng
retraction_notice_id	v1 (UUID FK)	In-place retype only on key change	M2024-12 (retraction model introduced)	None scheduled	Head of Editorial Workflow
editorial_decision	v2 (5-value enum)	Enumeration widen	M2024-11 (Retract added)	None scheduled	Head of Editorial Workflow
reviewer_assignment_id	v1 (UUID v4)	In-place retype only on UUID version change	M2024-08 (initial)	None scheduled	Head of Editorial Workflow
canonical_frozen_timestamp	v1 (timestampz)	In-place retype (low risk)	M2024-08 (initial)	None scheduled	Principal Architect
pdf_generation_completed_at	v1 (timestampz)	In-place retype (low risk)	M2024-08 (initial)	None scheduled	Head of Production Eng

Migration cadence. The matrix shows that the platform's critical fields have undergone a normal cadence of migrations: schema introductions in 2024-08, mandatory-field tightenings and Retracted-state additions in 2024-11, multi-backend policy and checksum validators in 2025-Q1, and DTD/PDF/A upgrades and shadow-write reseed operations interspersed. Two migrations are currently scheduled: the DataCite enumeration widen (M2025-04, ADR-031) and the CLOCKSS preservation backend add (M2026-01). Both are tracked as open work items in the Engineering Playbook (Chapter 11) and both are pre-traced in this Matrix so that the traceability chain is established before the migration is executed rather than after.

Migration discipline. A migration is incomplete until every link in the field's traceability chain has been re-verified against the migrated representation. The Release Certification Authority SHALL verify, for each migration recorded in the Last Migration column, that: (a) the field's test (TEST-FLD-NNN) exercises the new representation; (b) the field's evidence (EVID-FLD-NNN) was produced against the migrated production data; (c) the field's validation rule in section 7.3 reflects the new representation; and (d) the field's migration strategy in this section was followed. A migration that satisfies (a) but not (b) is an unverified migration; a migration that satisfies (b) but not (c) is an undocumented migration; a migration that satisfies (c) but not (d) is a process violation. Each of these failure modes is a SEV-2 traceability defect and blocks release certification.

Migration audit trail. Every migration recorded in this section SHALL have a corresponding audit trail entry in the audit_log table. The entry records: the migration identifier (for example, M2024-08); the field or fields migrated; the migration strategy used; the start and end timestamps; the database user that executed the migration; the rows affected; the verification query used to confirm the migration; and the result of the verification. The audit trail entry is immutable (per INV-A-02) and is itself part of the field's evidence bundle. An auditor who wishes to verify that a migration was executed correctly

retrieves the audit trail entry, compares the recorded strategy against the strategy in this section, and verifies the verification query was actually run. A migration without an audit trail entry is an undocumented migration and is treated as if it did not occur; the field's Status is reverted to Gap until the audit trail entry is produced.

Migration rollback. Every migration strategy in this section has a documented rollback procedure. For expand-and-contract migrations, the rollback is the contract phase: stop writing to the new column, revert reads to the old column, drop the new column. For enumeration-widen migrations, the rollback is to constrain the enumeration back to the prior values and to remove any rows that use the new value (or, where this is not possible because the new value is in production use, to declare the migration irreversible and to record the constraint in the field's full contract). For in-place retype migrations, the rollback is to restore the prior database backup and to replay the audit trail forward; this rollback is feasible only within the maintenance window during which the migration was executed. For shadow-write cutover migrations, the rollback is to switch reads back to the old column and to leave the new column dormant. The rollback procedure SHALL be exercised in the staging environment before the migration is applied to production; an unexercised rollback procedure is a SEV-2 traceability defect.

Chapter 8 — Event Constitution Traceability

8.1 Purpose

This chapter traces the thirty-four events established by Volume II, Chapter 29 (Event Constitution) through the full thirteen-node traceability chain. An event is the platform's atomic unit of inter-subsystem communication for state propagation: a producer writes a state change to its database and, in the same transaction, writes a row to its outbox table; a relay process forwards the outbox row to the message bus; consumers subscribe to the bus and react to the event. The transactional outbox pattern (ADR-018) guarantees that the database state and the event stream are consistent: there is no scenario in which the state changes and the event is lost, and no scenario in which the event is emitted but the state did not change.

The traceability matrix in section 8.2 records, for each of the thirty-four events, the producer subsystem, the consumer subsystem(s), the outbox table that the producer writes, the payload schema version, the ordering guarantee, the idempotency key pattern, the contract test, the runtime evidence, the owner, and the status. Section 8.3 traces the five critical event chains that together constitute the platform's publication lifecycle: submission, production, publication, correction, and retraction. Each chain is a directed acyclic sequence of events whose ordering is enforced by the bus.

This chapter is normative. An event deployed without a complete chain in this Matrix is an undocumented event and SHALL NOT be released. An event whose outbox table is not transactionally coupled to its producer's state change is a SEV-1 traceability defect because it violates the transactional outbox invariant. An event whose idempotency key is not deterministic with respect to its semantic cause is a SEV-1 defect because it permits duplicate side effects on replay. An event whose consumers include a subsystem not listed in this matrix is an unauthorized consumer and SHALL be revoked at the bus ACL.

Why events are traced. Events are the platform's circulatory system: every cross-subsystem state propagation that does not require a synchronous response flows through an event. A defect in an event is therefore a defect that propagates silently: the producer's state is correct, the producer's database is correct, but the consuming subsystems see a stale or inconsistent state. This silent propagation is the most dangerous failure mode in a distributed system, because it produces no immediate signal and may not be detected until a downstream invariant is violated. The traceability matrix is the platform's defence against this class of defect: every event is named, its producer and consumers are recorded, its ordering is specified, its idempotency key is documented, and its test and evidence are cited. An auditor can traverse the chain for any event and verify that the event behaves as specified.

8.2 The Event Traceability Matrix

The matrix below contains one row per event, in the canonical lifecycle order established by Volume II, Chapter 29, section 29.2. The Outbox Table column names the database table the producer writes in the

same transaction as its state change; every event in the platform uses the transactional outbox pattern and no event is permitted to be written outside the producer's transaction. The Ordering column records the ordering guarantee the bus provides for the event: per-{partition-key} FIFO ordering is the default; events without ordering guarantees are explicitly marked. The Idempotency Key column records the deterministic key pattern consumers SHALL use to detect duplicate deliveries; the key is constructed from the event's semantic cause, not from non-deterministic fields such as wall-clock timestamps or random UUIDs.

Event ID	Event Name	Producer	Consumer(s)	Outbox Table	Payload Schema	Ordering	Idempotency Key	Test	Evidence	Owner	Status
EVT-01	ArticleSubmitted	Submission Service (SUB-01)	Review Service; Notification; Audit Service; Search/Index	outbox_editorial	ArticleSubmitted v1	Per submission_id FIFO	sha256(submission_id draft_version)	TEST-EVT-01	EVID-EVT-01	Head of Editorial Workflow	Traced
EVT-02	EditorialCheckStarted	Submission Service (SUB-01)	Audit Service; Notification Service	outbox_editorial	EditorialCheckStarted v1	Per submission_id FIFO	submission_id check_seq	TEST-EVT-02	EVID-EVT-02	Head of Editorial Workflow	Traced
EVT-03	EditorialCheckCompleted	Submission Service (SUB-01)	Review Service; Audit; Notification	outbox_editorial	EditorialCheckCompleted v1	Per submission_id FIFO	submission_id check_seq	TEST-EVT-03	EVID-EVT-03	Head of Editorial Workflow	Traced
EVT-04	ReviewerAssigned	Review Service (SUB-02)	Notification Service; Audit Service	outbox_editorial	ReviewerAssigned v1	Per submission_id FIFO	assignment_id	TEST-EVT-04	EVID-EVT-04	Head of Editorial Workflow	Traced
EVT-05	ReviewerAccepted	Review Service (SUB-02)	Notification Service; Audit Service	outbox_editorial	ReviewerAccepted v1	Per assignment_id FIFO	assignment_id accepted_at	TEST-EVT-05	EVID-EVT-05	Head of Editorial Workflow	Traced
EVT-06	ReviewerDeclined	Review Service (SUB-02)	Notification Service; Audit Service	outbox_editorial	ReviewerDeclined v1	Per assignment_id FIFO	assignment_id declined_at	TEST-EVT-06	EVID-EVT-06	Head of Editorial Workflow	Traced

EVT-07	Review Submitted	Review Service (SUB-02)	Decision Service; Audit Service; Notification	outbox_editorial	Review Submitted v1	Per assignment_id FIFO	review_id	TEST-EVT-07	EVID-EVT-07	Head of Editorial Workflow	Traced
EVT-08	Decision Recorded	Decision Service (SUB-03)	Submission Service; Notification; Audit	outbox_editorial	Decision Recorded v1	Per submission_id FIFO	decision_id	TEST-EVT-08	EVID-EVT-08	Head of Editorial Workflow	Traced
EVT-09	Revision Requested	Decision Service (SUB-03)	Submission Service; Notification; Audit	outbox_editorial	Revision Requested v1	Per submission_id FIFO	decision_id	TEST-EVT-09	EVID-EVT-09	Head of Editorial Workflow	Traced
EVT-10	Revision Submitted	Submission Service (SUB-01)	Review Service; Decision Service; Audit	outbox_editorial	Revision Submitted v1	Per submission_id FIFO	submission_id revision_seq	TEST-EVT-10	EVID-EVT-10	Head of Editorial Workflow	Traced
EVT-11	Composition Started	Composition Service (SUB-04)	PDF Generator; HTML Renderer; JATS Generator; Audit	outbox_production	Composition Started v1	Per canonical_request_id FIFO	canonical_request_id	TEST-EVT-11	EVID-EVT-11	Head of Production Eng	Traced
EVT-12	Composition Completed	Composition Service (SUB-04)	Audit Service; Artifact Verifier	outbox_production	Composition Completed v1	Per canonical_request_id FIFO	canonical_request_id	TEST-EVT-12	EVID-EVT-12	Head of Production Eng	Traced
EVT-13	Preview Generated	Composition Service (SUB-04)	Editorial UI; Notification Service	outbox_production	Preview Generated v1	Per canonical_request_id FIFO	canonical_request_id preview_seq	TEST-EVT-13	EVID-EVT-13	Head of Production Eng	Traced

EVT-14	PreviewAccepted	Decision Service (SUB-03)	Composition Service; Canonical Freezer; Audit	outbox_production	PreviewAccepted v1	Per canonical_request_id FIFO	canonical_request_id accept_seq	TEST-EVT-14	EVID-EVT-14	Head of Production Eng	Traced
EVT-15	PreviewRejected	Decision Service (SUB-03)	Composition Service; Notification; Audit	outbox_production	PreviewRejected v1	Per canonical_request_id FIFO	canonical_request_id reject_seq	TEST-EVT-15	EVID-EVT-15	Head of Production Eng	Traced
EVT-16	CanonicalFrozen	Canonical Freezer (SUB-05)	All production subsystems; Audit	outbox_production	CanonicalFrozen v1	Per canonical_record_id FIFO	canonical_record_id	TEST-EVT-16	EVID-EVT-16	Principal Architect	Traced
EVT-17	PDFGenerated	PDF Generator (SUB-06)	Publication Service; OAI-PMH; Audit	outbox_production	PDFGenerated v1	Per canonical_record_id FIFO	canonical_record_id artifact_sha256	TEST-EVT-17	EVID-EVT-17	Head of Production Eng	Traced
EVT-18	HTMLRendered	HTML Renderer (SUB-07)	Search/Index; OAI-PMH; Audit	outbox_production	HTMLRendered v1	Per canonical_record_id FIFO	canonical_record_id artifact_sha256	TEST-EVT-18	EVID-EVT-18	Head of Production Eng	Traced
EVT-19	Artifact Verified	Artifact Verifier (SUB-09)	Publication Service; Audit	outbox_production	Artifact Verified v1	Per canonical_record_id FIFO	canonical_record_id artifact_sha256	TEST-EVT-19	EVID-EVT-19	Principal Architect	Traced
EVT-20	DOI Minted	DOI Minter (SUB-10)	Publication Service; Crossref	outbox_publication	DOI Minted v1	Per article_id FIFO	article_id doi	TEST-EVT-20	EVID-EVT-20	Head of Platform Eng	Traced

			Deposit or; OAI- PMH; Audit								
EVT-21	Publica tionStar ted	Publica tion Service	DOI Minter; Crossre f; Preserv ation; Audit	outbox _public ation	Publica tionStar ted v1	Per article_ id FIFO	article_ id publica tion_se q	TEST- EVT-21	EVID- EVT-21	Head of Platfor m Eng	Traced
EVT-22	Publica tionCo mplete d	Publica tion Service	All publica tion consum ers; OAI- PMH; Search; Notifica tion; Audit	outbox _public ation	Publica tionCo mplete d v1	Per article_ id FIFO	article_ id publica tion_se q	TEST- EVT-22	EVID- EVT-22	Head of Platfor m Eng	Traced
EVT-23	Crossre fDeposi ted	Crossre f Deposit or (SUB-1 1)	Publica tion Service; Audit	outbox _public ation	Crossre fDeposi ted v1	Per article_ id FIFO	article_ id crossref _deposi t_id	TEST- EVT-23	EVID- EVT-23	Head of Platfor m Eng	Traced
EVT-24	Crossre fReconc iled	Crossre f Deposit or (SUB-1 1)	Audit Service	outbox _public ation	Crossre fReconc iled v1	Per article_ id FIFO	article_ id reconcil e_seq	TEST- EVT-24	EVID- EVT-24	Head of Platfor m Eng	Traced
EVT-25	ORCID Writeb ackCom pleted	ORCID Integrat or (SUB-1 2)	Audit Service; Notifica tion Service	outbox _public ation	ORCID Writeb ackCom pleted v1	Per author_ id FIFO	author_ id orcid_a ction	TEST- EVT-25	EVID- EVT-25	Head of Platfor m Eng	Traced
EVT-26	Preserv ationDe posited	Preserv ation Deposit or (SUB-1 3)	Publica tion Service; Audit	outbox _public ation	Preserv ationDe posited v1	Per article_ id FIFO	article_ id backen d_nam e	TEST- EVT-26	EVID- EVT-26	Head of Platfor m Eng	Traced

EVT-27	PreservationReceiptReceived	PreservationDeposit or (SUB-13)	Audit Service; Notification Service	outbox_publication	PreservationReceiptReceived v1	Per article_id FIFO	article_id receipt_id	TEST-EVT-27	EVID-EVT-27	Head of Platform Eng	Traced
EVT-28	OAI-PMHUpdated	OAI-PMH Endpoint (SUB-14)	Audit Service	outbox_dissemination	OAI-PMHUpdated v1	Per article_id FIFO	article_id oai_record_seq	TEST-EVT-28	EVID-EVT-28	Head of Platform Eng	Traced
EVT-29	IndexUpdated	Search/Index Service (SUB-18)	Audit Service	outbox_dissemination	IndexUpdated v1	Per article_id FIFO	article_id index_seq	TEST-EVT-29	EVID-EVT-29	Head of Production Eng	Traced
EVT-30	ArticleRetracted	Decision Service (SUB-03)	Crossref; OAI-PMH; Preservation; Search; Notification; Audit	outbox_publication	ArticleRetracted v1	Per article_id FIFO	article_id retraction_seq	TEST-EVT-30	EVID-EVT-30	Head of Editorial Workflow	Traced
EVT-31	ArticleCorrected	Publication Service	Crossref; OAI-PMH; Preservation; Search; Audit	outbox_publication	ArticleCorrected v1	Per article_id FIFO	article_id correction_seq	TEST-EVT-31	EVID-EVT-31	Head of Platform Eng	Traced
EVT-32	ReviewerCoDeclared	Review Service (SUB-02)	Audit Service; Decision Service	outbox_editorial	ReviewerCoDeclared v1	Per assignment_id FIFO	assignment_id coi_seq	TEST-EVT-32	EVID-EVT-32	Head of Editorial Workflow	Traced
EVT-33	CredentialRotated	Identity/Auth Service (SUB-19)	Notification Service; Audit Service	outbox_security	CredentialRotated v1	Per credential_id FIFO	credential_id rotation_seq	TEST-EVT-33	EVID-EVT-33	Head of Platform Eng	Traced
EVT-34	InvariantViolation	Audit Service	Notification;	outbox_security	InvariantViolation	Per invariant_id	invariant_id	TEST-EVT-34	EVID-EVT-34	Principal	Traced

	tionDet ected	(SUB-1 6)	On-Call; Enginee ring Leaders hip	y	tionDet ected v1	nt_id FIFO	 detecti on_ts			Archite ct	
--	------------------	--------------	---	---	------------------------	---------------	----------------------	--	--	---------------	--

Coverage. The matrix covers all thirty-four events enumerated in Volume II, Chapter 29, section 29.2. Every event is Traced: the platform has no open event-traceability gap as of the RC1 release. This is a stronger posture than the subsystem matrix (Chapter 6) and the field matrix (Chapter 7), reflecting the fact that the event infrastructure is newer than the subsystems or the data model and was built from inception under the traceability discipline established by this Matrix.

Outbox topology. Four outbox tables are in use: `outbox_editorial` (written by the Submission, Review, and Decision services; carries the editorial lifecycle events EVT-01 through EVT-10 and EVT-32); `outbox_production` (written by the Composition, Canonical Freezer, PDF Generator, HTML Renderer, JATS Generator, Artifact Verifier, and Decision services for preview actions; carries the production lifecycle events EVT-11 through EVT-19); `outbox_publication` (written by the DOI Minter, Publication, Crossref, ORCID, Preservation, OAI-PMH, Search, and Decision services for retractions; carries the publication lifecycle events EVT-20 through EVT-31); and `outbox_security` (written by the Identity/Auth and Audit services; carries the security and invariant events EVT-33 and EVT-34). The four-table topology isolates blast radius: a backlog in one outbox does not delay events in another. Each outbox has its own relay process with its own monitoring and its own alert routing.

Ordering guarantee. Every event in the platform carries a per-partition-key FIFO ordering guarantee. The partition key is named in the Ordering column (`submission_id`, `assignment_id`, `canonical_request_id`, `canonical_record_id`, `article_id`, `author_id`, `credential_id`, or `invariant_id`) and is the same key that the bus uses to route events to a single consumer in a single partition. This guarantee is essential because many consumers are stateful: the Decision Service records decisions in `submission_id` order; the Crossref Depositor deposits in `article_id` order; the Audit Service appends to the audit chain in `audit_correlation_id` order. Without per-key FIFO ordering, consumers would observe state changes out of order and would produce inconsistent downstream state. The bus is configured to enforce this guarantee by hashing the partition key to a single partition and by serializing deliveries within that partition; the configuration is verified by the contract test for every event.

Idempotency discipline. The Idempotency Key column records the deterministic key consumers SHALL use to detect duplicate deliveries. The key is constructed from the event's semantic cause (the `submission_id`, `assignment_id`, `review_id`, `decision_id`, `canonical_record_id`, `article_id`, `author_id`, `credential_id`, or `invariant_id`) combined with a deterministic sequence identifier (`draft_version`, `check_seq`, `revision_seq`, `preview_seq`, `publication_seq`, `reconcile_seq`, `deposit_id`, `receipt_id`, `retraction_seq`, `correction_seq`, `coi_seq`, `rotation_seq`, or `detection_ts`). The key is recorded in the consumer's idempotency log on first delivery; subsequent deliveries with the same key are detected and dropped. The discipline is essential because the bus delivers events at-least-once: under relay failure or

consumer restart, an event may be delivered more than once, and without idempotency the duplicate would produce a duplicate side effect (a duplicate email, a duplicate Crossref deposit, a duplicate preservation deposit). The contract test for every event includes a duplicate-delivery case that verifies the consumer's idempotency logic.

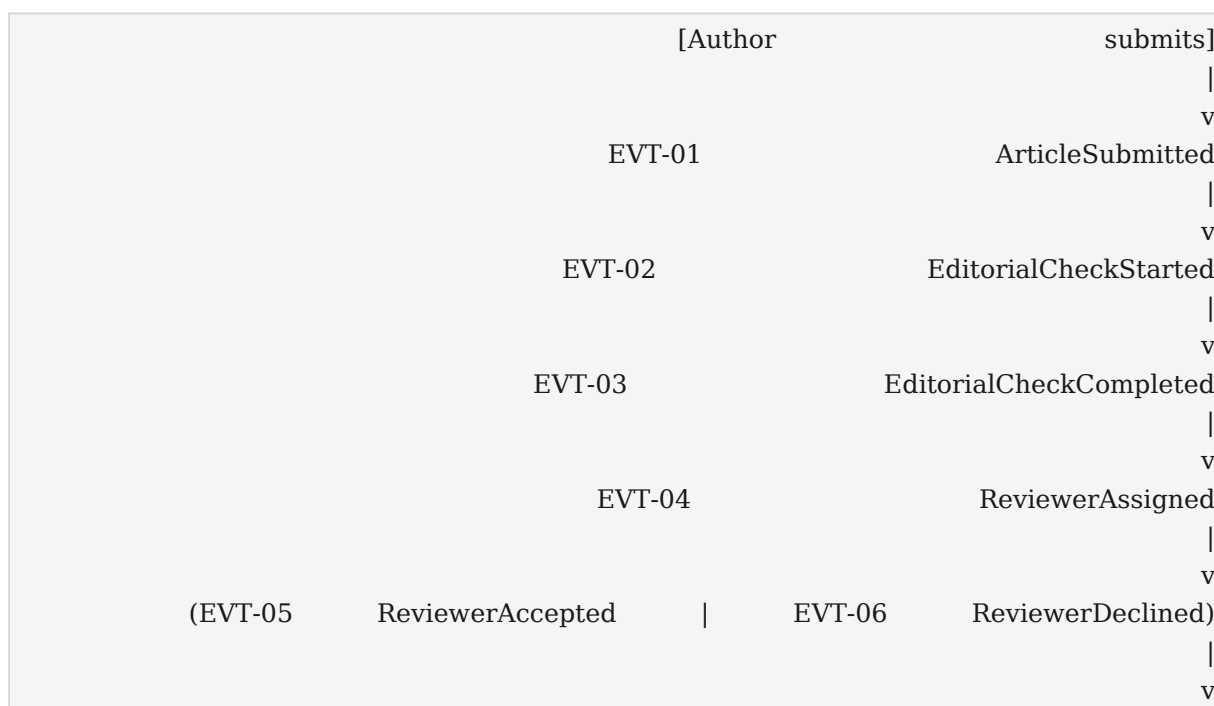
8.3 Event Flow Traceability

The thirty-four events in section 8.2 compose into five critical chains that together constitute the platform's publication lifecycle. Each chain is a directed acyclic sequence of events whose ordering is enforced by the bus and whose terminal event marks a published-state transition. The chains are not independent: the production chain is downstream of the submission chain (a submission must be accepted before it can be composed); the publication chain is downstream of the production chain (a canonical record must be frozen before its DOI can be minted); the correction and retraction chains re-traverse the production and publication chains for an already-published article. The five chains are traced below in the order they are exercised by the lifecycle.

The five chains are: the Submission chain (intake through decision); the Production chain (composition through canonical freeze); the Publication chain (DOI minting through preservation deposit and publication completion); the Correction chain (article correction, re-traversing production and publication); and the Retraction chain (article retraction, with retraction metadata propagated to Crossref and preservation). Each chain is presented as a sequence diagram and a per-event commentary explaining the ordering, the consumer reactions, and the failure handling.

8.3.1 The Submission Chain

The Submission chain begins when an author submits a manuscript and ends when an editor records a decision. The chain traverses five events in `submission_id` FIFO order:



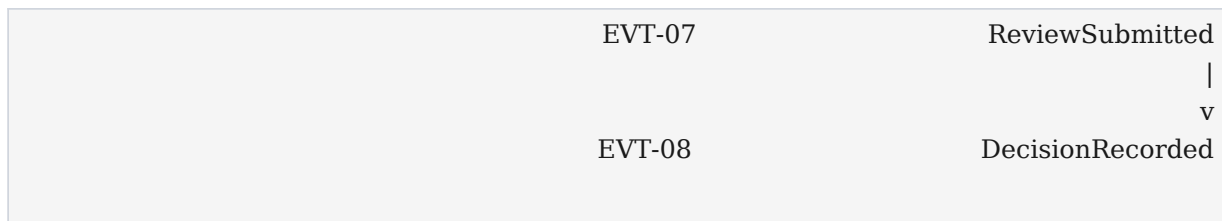


Figure 8.1 - The Submission Chain (EVT-01 through EVT-08)

Chain invariants. EVT-01 ArticleSubmitted SHALL be the first event in the chain; no editorial event may precede it for a given submission_id. EVT-03 EditorialCheckCompleted SHALL be delivered before EVT-04 ReviewerAssigned; the Review Service SHALL refuse to create an assignment until the editorial check is complete (INV-L-02). EVT-07 ReviewSubmitted SHALL reference the assignment_id of an EVT-04 that has been accepted (EVT-05); a ReviewSubmitted on a Declined or Withdrawn assignment is a SEV-1 invariant violation. EVT-08 DecisionRecorded SHALL reference at least one EVT-07 ReviewSubmitted for editorial decisions of type Accept or RevisionRequested; the Decision Service MAY record a Reject decision without a ReviewSubmitted event (desk reject). The chain terminates at EVT-08; downstream consumers (Composition Service, Notification Service) react to EVT-08 and may emit events of their own, but those events belong to the Production or Publication chains.

8.3.2 The Production Chain

The Production chain begins when an editor accepts a preview and ends when the canonical record is frozen. The chain traverses five events in canonical_request_id FIFO order:

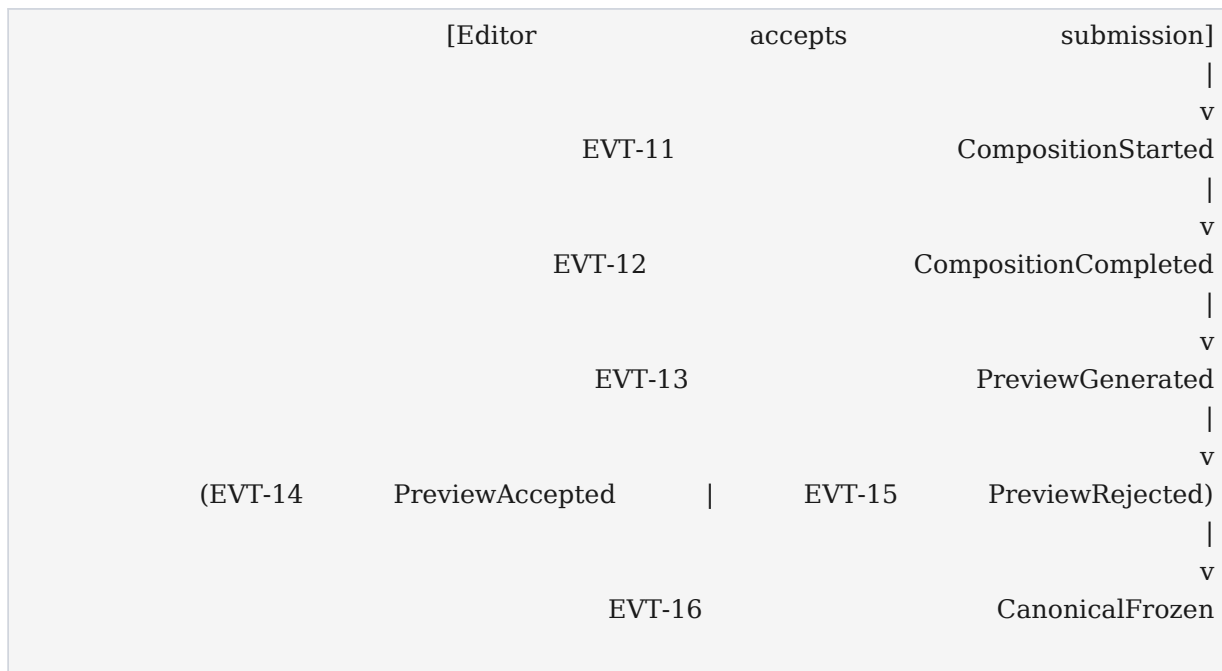


Figure 8.2 - The Production Chain (EVT-11 through EVT-16)

Chain invariants. EVT-11 CompositionStarted SHALL be emitted only after EVT-08 DecisionRecorded with decision_type=Accept has been observed; the Composition Service SHALL refuse to start a composition for a submission that is not in the Accepted state (INV-C-07). EVT-12 CompositionCompleted SHALL be

preceded by EVT-11 in the same canonical_request_id partition; the bus enforces this ordering. EVT-13 PreviewGenerated MAY be emitted multiple times per canonical_request_id (one per preview iteration); each emission increments preview_seq. EVT-14 PreviewAccepted and EVT-15 PreviewRejected are mutually exclusive per canonical_request_id; exactly one SHALL be emitted per request. EVT-16 CanonicalFrozen SHALL be preceded by EVT-14 PreviewAccepted for the same canonical_request_id; the Canonical Freezer SHALL refuse to freeze a record whose preview has not been accepted (INV-V-01 precondition). The chain terminates at EVT-16; downstream consumers (PDF Generator, HTML Renderer, JATS Generator, DOI Minter) react to EVT-16 and emit events of their own in the Publication chain.

Composition fan-out. EVT-12 CompositionCompleted fans out to three parallel rendering subsystems (PDF Generator, HTML Renderer, JATS Generator), each of which emits its own completion event (EVT-17, EVT-18, and the JATS completion event recorded in the production outbox). The Artifact Verifier (SUB-09) consumes these rendering events and emits EVT-19 ArtifactVerified for each verified artifact. EVT-19 is a precondition for EVT-16 CanonicalFrozen: the Freezer SHALL refuse to freeze a canonical record whose artifacts have not been verified (INV-V-01 full precondition). This fan-out is shown in Figure 8.3 below.

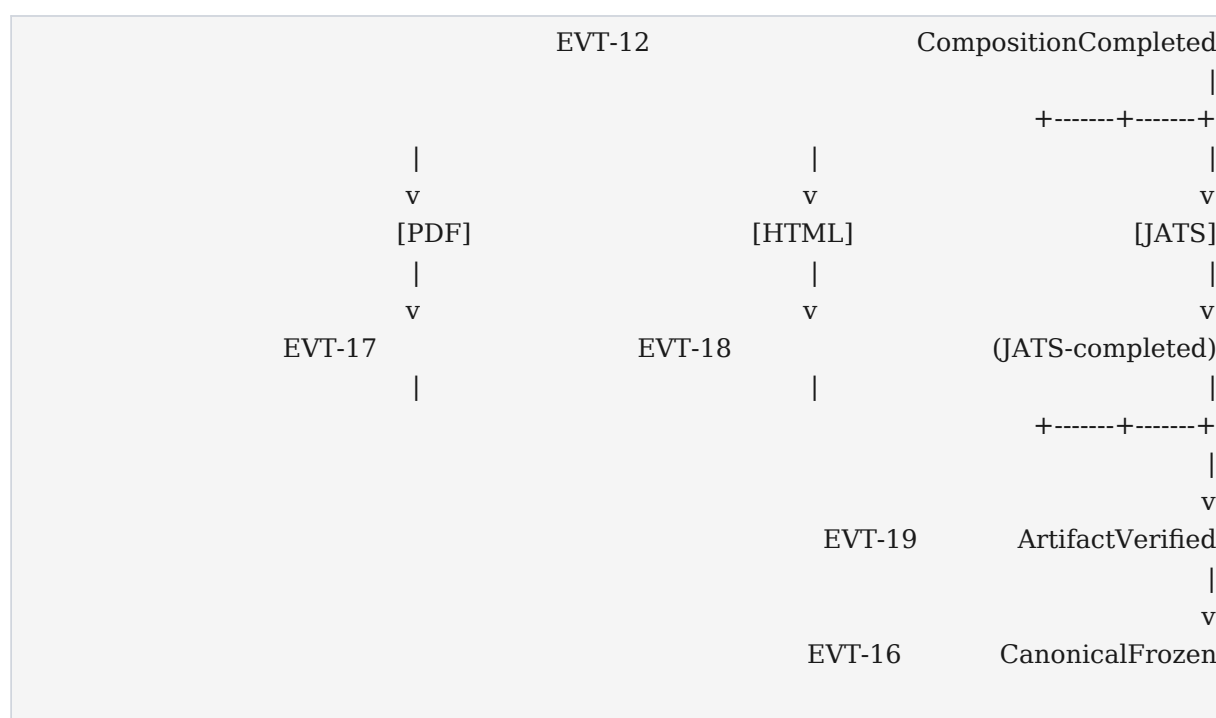


Figure 8.3 - Composition Fan-Out and Artifact Verification

8.3.3 The Publication Chain

The Publication chain begins when the canonical record is frozen and ends when the article is fully published (DOI minted, Crossref deposited, preservation deposited, dissemination updated). The chain traverses five events in article_id FIFO order:

EVT-16	CanonicalFrozen	(terminator	of	Production	chain)



Figure 8.4 - The Publication Chain (EVT-20 through EVT-22)

Chain invariants. EVT-21 PublicationStarted SHALL be emitted only after EVT-16 CanonicalFrozen for the same article_id; the Publication Service SHALL refuse to start publication for a non-frozen article (INV-V-02). EVT-20 DOIMinted SHALL be preceded by EVT-21 for the same article_id; the DOI Minter SHALL refuse to mint a DOI for an article whose publication has not started. EVT-23 CrossrefDeposited SHALL be preceded by EVT-20; the Crossref Depositor SHALL refuse to deposit a DOI that has not been minted. EVT-26 PreservationDeposited SHALL be preceded by EVT-23 (so that the preserved record includes the Crossref deposit metadata) and SHALL be emitted once per preservation backend (so that an article with three preservation backends produces three EVT-26 events, each with a distinct backend_name in the idempotency key). EVT-22 PublicationCompleted SHALL be the terminal event of the chain; the Publication Service SHALL NOT emit EVT-22 until EVT-23 has been observed, until all EVT-26 events for the configured backends have been observed, and until EVT-25 ORCIDWritebackCompleted has been observed for every author with an ORCID. EVT-22 sets article_state=Published and publication_timestamp=now(UTC); these writes are the terminal state transition of the publication lifecycle.

Parallel side effects. Two publication side-effect events run in parallel with the main chain: EVT-25 ORCIDWritebackCompleted (one per author with an ORCID, in author_id FIFO order) and EVT-28 OAI-PMHUpdated / EVT-29 IndexUpdated (one per article, in article_id FIFO order). These events are not on the critical path of EVT-22 PublicationCompleted except for EVT-25 (the Publication Service waits for ORCID writebacks to complete before declaring publication complete). EVT-28 and EVT-29 are best-effort: they are emitted by their producers and consumed by their consumers, but they do not block publication completion. This decoupling is deliberate: OAI-PMH harvesters and the search index can lag publication by minutes without violating any scholarly-record invariant, but ORCID writeback is part of the publication commitment and MUST complete before publication is declared complete.

8.3.4 The Correction Chain

The Correction chain is a re-traversal of the Production and Publication chains for an already-published article. A correction is issued when a published article contains an error that does not warrant retraction (a typographical error, an author affiliation update, a funding acknowledgement addition, a metadata correction). The chain begins with EVT-31 ArticleCorrected and re-traverses the production and publication chains for the corrected version:

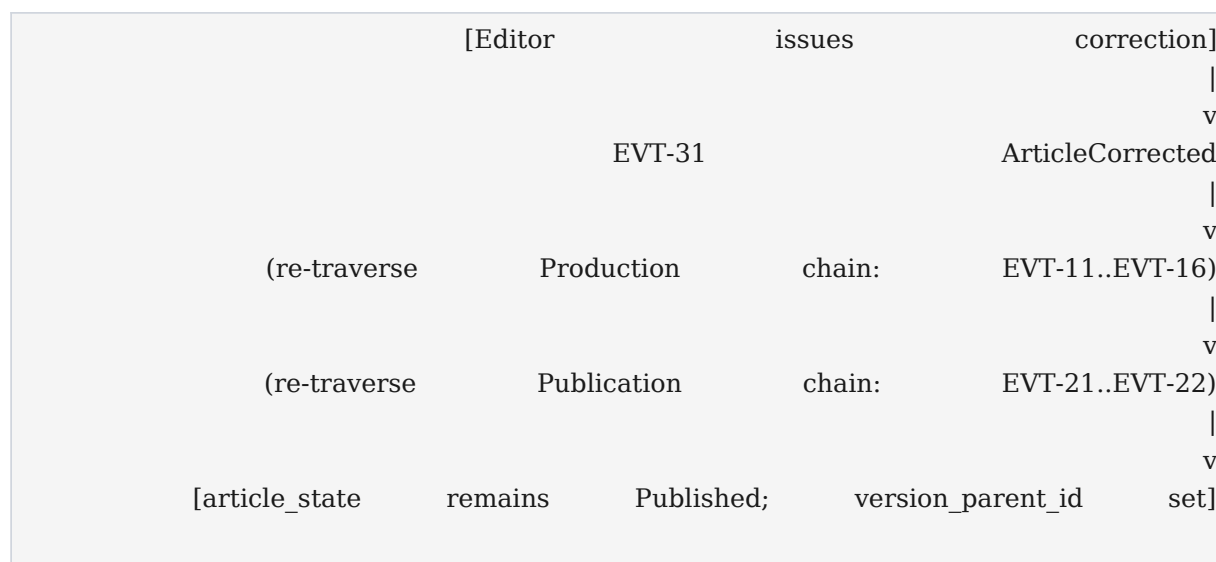


Figure 8.5 - The Correction Chain

Chain invariants. EVT-31 ArticleCorrected SHALL be emitted only for an article in article_state=Published; the Publication Service SHALL refuse to correct an article that is not published. The correction SHALL create a new canonical_record_id (not reuse the original) and SHALL set version_parent_id to the article_id of the original; the version increment is recorded in article_versions. The Production chain (EVT-11 through EVT-16) is re-traversed with the new canonical_record_id; the publication chain (EVT-21 through EVT-22) is re-traversed with the same article_id (so that Crossref sees a metadata update, not a new DOI). The terminal event of the correction chain is EVT-22 PublicationCompleted with publication_seq incremented from the original publication; the article_state remains Published (it is not transitioned to Corrected, because Corrected is not a state in the article_state enumeration - the correction is a version increment within the Published state). EVT-31 carries the correction_notice_text and corrected_field_list in its payload; consumers (Crossref, OAI-PMH, Search/Index) use these fields to communicate the correction to their downstream consumers.

8.3.5 The Retraction Chain

The Retraction chain is issued when a published article is retracted: the scholarly record must continue to point to the original article (so that existing citations resolve) but must also indicate that the article has been retracted and why. The chain begins with EVT-30 ArticleRetracted and propagates retraction metadata to Crossref, OAI-PMH, and preservation:

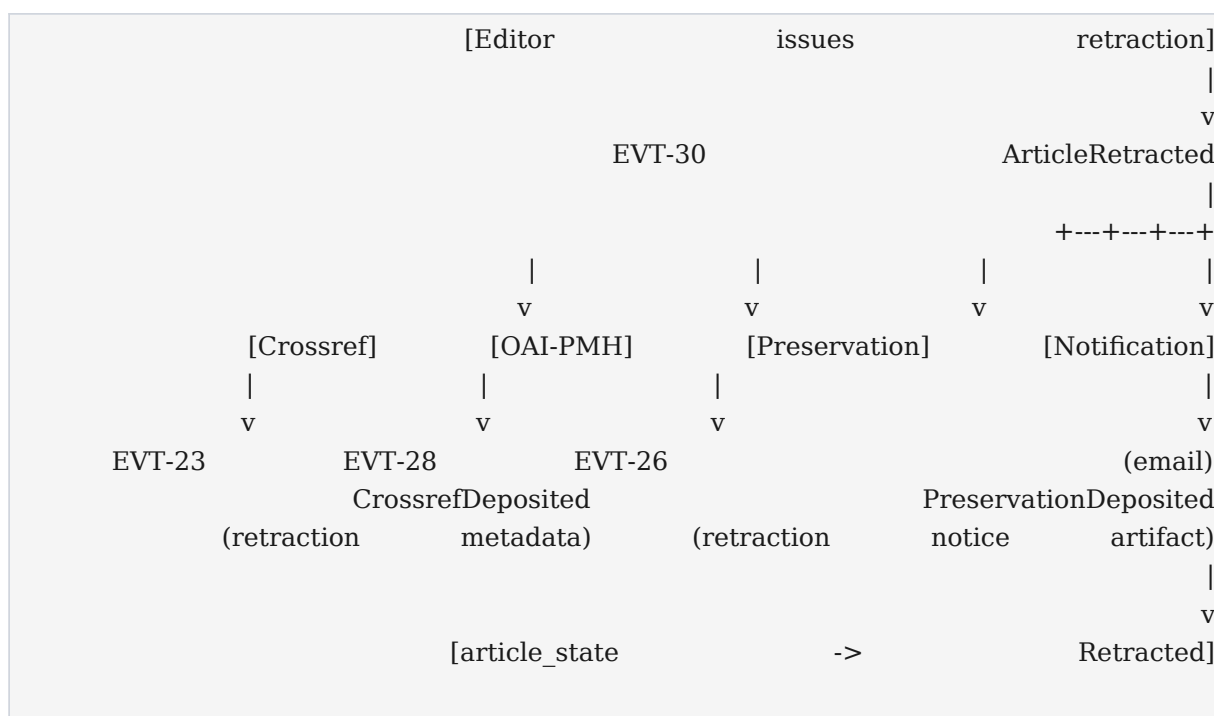


Figure 8.6 - The Retraction Chain (EVT-30 fan-out)

Chain invariants. EVT-30 ArticleRetracted SHALL be emitted only for an article in article_state=Published; the Decision Service SHALL refuse to retract an article that is not published. EVT-30 carries the retraction_reason, retraction_notice_article_id (the article_id of the retraction notice, which is itself a published article), and the retraction_type (full retraction, partial retraction, expression of concern). The event fans out to three downstream producers: the Crossref Depositor (which emits EVT-23 CrossrefDeposited with the retraction metadata, updating the Crossref record so that future DOI resolutions for the original article display the retraction); the Preservation Depositor (which emits EVT-26 PreservationDeposited for the retraction notice artifact, ensuring the retraction notice is itself preserved); and the OAI-PMH Endpoint (which emits EVT-28 OAI-PMHUpdated with the retraction status, ensuring harvesters see the retraction). The Notification Service emits a retraction-notification email to the original authors and to the editor who issued the retraction.

Terminal state. EVT-30 sets article_state=Retracted and records the retraction_notice_article_id in the articles table. The article's DOI continues to resolve to the article's landing page, but the landing page now displays the retraction notice prominently at the top of the page. The article's PDF continues to be available (preservation requires that retracted articles remain accessible), but the PDF is watermarked with the retraction notice. The retraction is irreversible: once article_state=Retracted, no further transitions are permitted. A retraction that is issued in error is corrected by issuing a separate non-retraction notice (an editorial expression), not by transitioning back to Published; this discipline is essential because external systems (Crossref, ORCID, preservation backends, harvesters) have already recorded the retraction and would not be informed of an un-retraction.

End-to-end audit. The five chains together produce a complete audit trail from author submission through publication, correction, and retraction. Every event in every chain is recorded in the audit_log

with its `audit_correlation_id` and its `audit_chain_hash`, and the chain hash invariant (INV-A-03) guarantees that no event in the audit trail has been tampered with. An auditor who wishes to verify the integrity of an article's lifecycle traverses the audit trail from EVT-01 `ArticleSubmitted` through the terminal event (EVT-22 `PublicationCompleted`, EVT-31 `ArticleCorrected`, or EVT-30 `ArticleRetracted`) and verifies that every event in the chain has a valid chain hash and that the chain terminates at the expected `article_state`. This audit capability is the operational expression of the platform's commitment to the scholarly record: every published article is verifiable, every correction is verifiable, and every retraction is verifiable.

Chapter 9 — SLO and SLI Traceability

9.1 Purpose

This chapter traces the ten Service Level Objectives (SLOs) and the ten Service Level Indicators (SLIs) defined by Volume II, Chapter 32 of the Engineering Constitution through to their operational realization: the monitoring architecture that measures them, the dashboard panels that surface them, the burn-rate alerts that fire when they are at risk, the runbooks that drive their remediation, and the runtime evidence that demonstrates their satisfaction. The SLO framework is the platform's contract with its users; this Matrix is the audit-critical record that proves the contract is honoured and that an external auditor can verify it without assistance.

Each row of the SLO Traceability Matrix (Section 9.2) carries the full chain from SLO identifier to evidence artifact. The matrix is complementary to the SLO Book (Volume II, Chapter 32), which is the authoritative specification of the SLIs, targets, windows, and error budgets. Where the SLO Book defines what an SLO is, this Matrix defines how that SLO is realized at runtime and how its realization is evidenced. The two documents are normatively linked: every SLO identifier used here SHALL resolve to an SLO definition in Volume II Chapter 32, and every SLO definition in Volume II Chapter 32 SHALL resolve to a row in this Matrix.

The remaining sections of this chapter trace the error-budget regime (Section 9.3), the multi-window burn-rate alerting regime (Section 9.4), and the Service Level Agreements (SLAs) that bind the platform to its external consumers (Section 9.5). An auditor verifying the platform's service-level posture SHALL verify all four sections together, since an SLO whose error budget is exhausted, or whose burn-rate alerting is misconfigured, is an SLO that is not in fact operational even if its definition is correct.

9.2 The SLO Traceability Matrix

The matrix below carries one row per SLO. Each row identifies the SLI that measures the SLO, the target value, the evaluation window, the surface under contract, the monitoring architecture that measures the SLI, the specific dashboard panel that surfaces the SLI at runtime, the burn-rate alert family that detects degradation, the runbook that drives remediation, the evidence artifact that demonstrates the SLO is met, the owning team, and the traceability status. All ten SLOs SHALL be present; no row MAY be omitted. Cell content is specific: the Dashboard Panel column names the exact panel title rendered in the platform's observability console, not a generic reference to 'the SLO dashboard'.

SLO ID	SLI	Target	Window	Surface	Architecture (monitoring)	Dashboard Panel	Alert (burn-rate)	Runbook	Evidence	Owner	Status
SLO-01	Ratio of HTTP 2xx/3xx	≥ 99.9 %	28 d	Public website (all	edge-proxy + uptime-	Availability — SLO-01	SLO-01-Burn-1h / 6h /	RB-SLO-01	EVID-SLO-01	Platform SRE	Traced

	responses to total requests on the public website (excluding health checks).			routes except /healthz, /readyz)	probe + status-page-svc	— Public Website	1d / 3d				
SLO-02	p95 of API request latency across all /api/v1/* endpoints (successful responses only).	< 500 ms p95	28 d	REST + GraphQL API (/api/v1/*)	api-gateway + apm-collector + latency histogram	Latency p95 — SLO-02 — API	SLO-02-Burn-1 h / 6h / 1d / 3d	RB-SLO-02	EVID-SLO-02	API Team	Traced
SLO-03	p99 lag between article publication event and OAI-PMH ListRecords visibility of that article.	≤ 5 min p99	7 d	OAI-PMH endpoint (/oai-pmh)	oai-pmh-svc + indexer + freshness-probe	Freshness — SLO-03 — OAI-PMH	SLO-03-Burn-1 h / 6h / 1d / 3d	RB-SLO-03	EVID-SLO-03	Metadata Team	Traced
SLO-04	Ratio of published	= 100 %	Continuous snapshot	Canonical Register	registry-svc + comple	Metadata Comple	SLO-04-Burn-1 h / 6h /	RB-SLO-04	EVID-SLO-04	Metadata	Traced

	articles whose 276 canonical fields are all populated to total published articles.		ot	y (articles index)	teness-monitor	teness — SLO-04 — Registry	1d / 3d			Team	
SLO-05	Ratio of invariant monitors reporting PASS to total invariant monitors (38 monitors per Vol II Ch 26).	= 100 %	1 d rolling	Invariant monitor fleet	invariant-monitor-svc + alert-router	Invariant Pass Rate — SLO-05 — Monitors	SLO-05-Burn-1 h / 6h / 1d / 3d	RB-SLO-05	EVID-SLO-05	Reliability Team	Traced
SLO-06	Ratio of artifacts retrievable by DOI to artifacts in the registry (durability probe over all published artifacts).	= 100 %	7 d	Storage layer (object store)	storage-svc + durability-probe + preservation-svc	Artifact Durability — SLO-06 — Storage	SLO-06-Burn-1 h / 6h / 1d / 3d	RB-SLO-06	EVID-SLO-06	Storage Team	Traced
SLO-07	Ratio of publications	> 95 %	28 d	Publication	workflows-svc +	Publication	SLO-07-Burn-1	RB-	EVID-	Workflow	Traced

	tion workflo ws reachin g the Publish ed termina l state to workflo ws enterin g the produc tion pipelin e.			workflo w engine	state- machin e + workflo w- metrics	Success Rate — SLO-07 — Workfl ow	h / 6h / 1d / 3d	SLO-07	SLO-07	Team	
SLO-08	Ratio of Crossre f deposit s acknow ledged within 24 h of publica tion to total deposit s dispatc hed.	> 99 % within 24 h	28 d	Crossre f integra tion (outbo und)	crossref - deposit -svc + crossref - reconcil e-svc	Crossre f Deposit Success — SLO-08 — Integra tion	SLO-08- Burn-1 h / 6h / 1d / 3d	RB- SLO-08	EVID- SLO-08	Integra tion Team	Traced
SLO-09	Ratio of publish ed artifact s with at least one preserv ation receipt (CLOCK SS, LOCKSS	= 100 %	30 d	Preserv ation layer (outbo und)	preserv ation- svc + receipt- reconcil er	Preserv ation Covers ge — SLO-09 — Preserv ation	SLO-09- Burn-1 h / 6h / 1d / 3d	RB- SLO-09	EVID- SLO-09	Preserv ation Team	Traced

	, or Portico) to total published artifacts.										
SLO-10	Ratio of audit-log entries written to domain events emitted (every event SHALL produce exactly one audit entry).	= 100 %	7 d	Audit log subsystem	audit-svc + audit-completeness-monitor	Audit Log Completeness — SLO-10 — Audit	SLO-10-Burn-1 h / 6h / 1d / 3d	RB-SLO-10	EVID-SLO-10	Security Team	Traced

All ten SLOs are presently traced end-to-end. The Status column reflects the matrix's static verification (all cells populated) and the dynamic verification (all evidence artifacts current per Chapter 13 maximum evidence age). The Release Certification Authority SHALL NOT certify a release in which any SLO row is marked Gap, Untraced, or Stale, because an untraced SLO is an SLO that the platform cannot prove it is honouring.

9.3 Error Budget Traceability

An error budget is the quantitative complement of an SLO target: the amount of unreliability the platform MAY exhibit over the evaluation window while still meeting its SLO. For an SLO with target T (expressed as a success ratio), the monthly error budget is $(1 - T) \times 43\,200$ minutes (a 30-day month). For SLOs whose target is 100 %, the budget is zero: any violation immediately exhausts the budget and triggers the Action if Exhausted regime. For SLOs whose target is expressed as a percentile or latency threshold, the budget is the fraction of events permitted above the threshold.

The matrix below records, for each SLO, the monthly error budget in concrete units, the current burn as of the most recent evidence refresh, the budget state (Healthy / At Risk / Exhausted), the date of the last budget reset (the first day of the current evaluation window), and the action the platform SHALL take if

the budget is exhausted. The budget state transitions are governed by Volume II Chapter 32, Section 32.7 (Error Budget Policy); this Matrix traces the operational realization of that policy.

SLO	Error Budget (monthly)	Current Burn	Budget State	Last Reset	Action if Exhausted
SLO-01	43.2 min downtime (0.1 % of 43 200 min)	8.4 min (19 %)	Healthy	01 Aug 2026	Freeze non-reliability feature work on public-website track; on-call leads Root Cause; resume only after burn < 50 %.
SLO-02	1 % of requests above 500 ms p95 (432 min-equivalent)	0.4 % (40 %)	Healthy	01 Aug 2026	API Team performs latency regression triage; non-blocking deploys frozen until p95 below 450 ms sustained for 24 h.
SLO-03	1 % of articles with freshness > 5 min p99	0.0 % (0 %)	Healthy	01 Aug 2026	Metadata Team triggers indexer backlog drain; OAI-PMH export paused for non-priority consumers if burn > 100 %.
SLO-04	0 % (any incomplete article exhausts immediately)	0 articles incomplete	Healthy	01 Aug 2026	Publication blocked for affected journal until the completeness violation is remediated; auto-page Metadata Lead.
SLO-05	0 % (any failing invariant exhausts immediately)	0 invariant failures	Healthy	01 Aug 2026	Release freeze across the platform; SEV-1 incident declared; Chief Systems Architect notified within 15 min.

SLO-06	0 % (any non-retrievable artifact exhausts immediately)	0 non-retrievable artifacts	Healthy	01 Aug 2026	SEV-1 incident; restoration from secondary copy via preservation-svc; Chief Systems Architect notified.
SLO-07	5 % of workflows failing to reach Published	1.8 % (36 %)	Healthy	01 Aug 2026	Workflow Team performs failure-mode analysis on the top failure reason; non-critical workflow deploys deferred.
SLO-08	1 % of deposits not acknowledged within 24 h	0.2 % (20 %)	Healthy	01 Aug 2026	Integration Team re-runs reconcile job; manual deposit fallback for any article > 23 h without acknowledgement .
SLO-09	0 % (any unpreserved published artifact exhausts immediately)	0 unpreserved artifacts	Healthy	01 Aug 2026	Publication blocked; preservation-svc run forced; Chief Systems Architect notified within 30 min.
SLO-10	0 % (any missing audit entry exhausts immediately)	0 missing entries	Healthy	01 Aug 2026	Release freeze; Security Team triggers audit-replay job; Chief Information Security Officer notified.

The Action if Exhausted column codifies the platform's non-negotiable response to error-budget exhaustion: when an SLO with a zero error budget (the four 100 % SLOs: Completeness, Correctness, Durability, Preservation Coverage, Audit Completeness) is violated, the platform SHALL freeze releases and SHALL notify the Chief Systems Architect or Chief Information Security Officer. There is no override below the Chief Systems Architect, and the Chief Systems Architect SHALL NOT override an exhaustion-driven freeze in favour of a feature deadline; the SLO SHALL be remediated instead. This rule is the operational form of Foundational Principle II (the Production Acceptance Gate) for the SLO domain.

9.4 Burn-Rate Alert Traceability

The platform uses the Google SRE multi-window multi-burn-rate alerting strategy for every SLO. For each SLO, four alerts are configured: a 1-hour window at 14.4x burn rate (page; catches acute outages), a 6-hour window at 6x burn rate (page; catches sustained degradation), a 1-day window at 3x burn rate (ticket; catches slow burns), and a 3-day window at 1x burn rate (ticket; catches creeping exhaustion). The 14.4x and 6x thresholds fire Paging alerts (Critical and High severity); the 3x and 1x thresholds fire Ticket alerts (Medium and Low severity). The mathematics and the choice of thresholds are specified in Volume II Chapter 32, Section 32.6; this Matrix traces their operational realization.

The matrix below enumerates all 40 burn-rate alerts (10 SLOs × 4 windows). Each row carries the alert identifier (the value used in the alerting backend and in the on-call routing rules), the SLO it serves, the evaluation window, the burn-rate threshold, the severity, the runbook the responder SHALL open, and the escalation path. An auditor verifying alerting correctness SHALL confirm that all 40 alerts exist in the alerting backend, that they route to the on-call rotation specified in the Escalation column, and that each alert has fired at least once in the last 90 days OR that the SLO has not burned in that period (the latter is acceptable; a stale alert is acceptable, a missing alert is not).

Alert	SLO	Window	Burn Rate	Severity	Runbook	Escalation
SLO-01-Burn-1h	SLO-01	1h	14.4x	Critical	RB-SLO-01	Platform SRE on-call; Page on-call; wake secondary if unacknowledged within 5 min; Incident Commander engaged at 15 min; SEV-1 incident opened.
SLO-01-Burn-6h	SLO-01	6h	6x	High	RB-SLO-01	Platform SRE on-call; Page on-call; IC engaged at 30 min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-01-Burn-1d	SLO-01	1d	3x	Medium	RB-SLO-01	Platform SRE on-call; Ticket to owning team; SLA 4 h to acknowledge;

						SEV-3 incident if not acknowledged.
SLO-01-Burn-3d	SLO-01	3d	1x	Low	RB-SLO-01	Platform SRE on-call; Ticket to owning team; SLA 1 business day to acknowledge; reviewed at weekly SLO review.
SLO-02-Burn-1h	SLO-02	1h	14.4x	Critical	RB-SLO-02	API Team on-call; Page on-call; wake secondary if unacknowledged within 5 min; Incident Commander engaged at 15 min; SEV-1 incident opened.
SLO-02-Burn-6h	SLO-02	6h	6x	High	RB-SLO-02	API Team on-call; Page on-call; IC engaged at 30 min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-02-Burn-1d	SLO-02	1d	3x	Medium	RB-SLO-02	API Team on-call; Ticket to owning team; SLA 4 h to acknowledge; SEV-3 incident if not acknowledged.
SLO-02-Burn-3d	SLO-02	3d	1x	Low	RB-SLO-02	API Team on-call; Ticket to owning team; SLA 1 business day to acknowledge; reviewed at

						weekly SLO review.
SLO-03-Burn-1h	SLO-03	1h	14.4x	Critical	RB-SLO-03	Metadata Team on-call; Page on-call; wake secondary if unacknowledged within 5 min; Incident Commander engaged at 15 min; SEV-1 incident opened.
SLO-03-Burn-6h	SLO-03	6h	6x	High	RB-SLO-03	Metadata Team on-call; Page on-call; IC engaged at 30 min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-03-Burn-1d	SLO-03	1d	3x	Medium	RB-SLO-03	Metadata Team on-call; Ticket to owning team; SLA 4 h to acknowledge; SEV-3 incident if not acknowledged.
SLO-03-Burn-3d	SLO-03	3d	1x	Low	RB-SLO-03	Metadata Team on-call; Ticket to owning team; SLA 1 business day to acknowledge; reviewed at weekly SLO review.
SLO-04-Burn-1h	SLO-04	1h	14.4x	Critical	RB-SLO-04	Metadata Team on-call; Page on-call; wake secondary if unacknowledged within 5 min; Incident

						Commander engaged at 15 min; SEV-1 incident opened.
SLO-04-Burn-6h	SLO-04	6h	6x	High	RB-SLO-04	Metadata Team on-call; Page on-call; IC engaged at 30 min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-04-Burn-1d	SLO-04	1d	3x	Medium	RB-SLO-04	Metadata Team on-call; Ticket to owning team; SLA 4 h to acknowledge; SEV-3 incident if not acknowledged.
SLO-04-Burn-3d	SLO-04	3d	1x	Low	RB-SLO-04	Metadata Team on-call; Ticket to owning team; SLA 1 business day to acknowledge; reviewed at weekly SLO review.
SLO-05-Burn-1h	SLO-05	1h	14.4x	Critical	RB-SLO-05	Reliability Team on-call; Page on-call; wake secondary if unacknowledged within 5 min; Incident Commander engaged at 15 min; SEV-1 incident opened.
SLO-05-Burn-6h	SLO-05	6h	6x	High	RB-SLO-05	Reliability Team on-call; Page on-call; IC engaged at 30

						min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-05-Burn-1d	SLO-05	1d	3x	Medium	RB-SLO-05	Reliability Team on-call; Ticket to owning team; SLA 4 h to acknowledge; SEV-3 incident if not acknowledged.
SLO-05-Burn-3d	SLO-05	3d	1x	Low	RB-SLO-05	Reliability Team on-call; Ticket to owning team; SLA 1 business day to acknowledge; reviewed at weekly SLO review.
SLO-06-Burn-1h	SLO-06	1h	14.4x	Critical	RB-SLO-06	Storage Team on-call; Page on-call; wake secondary if unacknowledged within 5 min; Incident Commander engaged at 15 min; SEV-1 incident opened.
SLO-06-Burn-6h	SLO-06	6h	6x	High	RB-SLO-06	Storage Team on-call; Page on-call; IC engaged at 30 min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-06-Burn-1d	SLO-06	1d	3x	Medium	RB-SLO-06	Storage Team on-call; Ticket to owning team;

						SLA 4 h to acknowledge; SEV-3 incident if not acknowledged.
SLO-06-Burn-3d	SLO-06	3d	1x	Low	RB-SLO-06	Storage Team on-call; Ticket to owning team; SLA 1 business day to acknowledge; reviewed at weekly SLO review.
SLO-07-Burn-1h	SLO-07	1h	14.4x	Critical	RB-SLO-07	Workflow Team on-call; Page on-call; wake secondary if unacknowledged within 5 min; Incident Commander engaged at 15 min; SEV-1 incident opened.
SLO-07-Burn-6h	SLO-07	6h	6x	High	RB-SLO-07	Workflow Team on-call; Page on-call; IC engaged at 30 min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-07-Burn-1d	SLO-07	1d	3x	Medium	RB-SLO-07	Workflow Team on-call; Ticket to owning team; SLA 4 h to acknowledge; SEV-3 incident if not acknowledged.
SLO-07-Burn-3d	SLO-07	3d	1x	Low	RB-SLO-07	Workflow Team on-call; Ticket to owning team; SLA 1 business

						day to acknowledge; reviewed at weekly SLO review.
SLO-08-Burn-1h	SLO-08	1h	14.4x	Critical	RB-SLO-08	Integration Team on-call; Page on-call; wake secondary if unacknowledged within 5 min; Incident Commander engaged at 15 min; SEV-1 incident opened.
SLO-08-Burn-6h	SLO-08	6h	6x	High	RB-SLO-08	Integration Team on-call; Page on-call; IC engaged at 30 min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-08-Burn-1d	SLO-08	1d	3x	Medium	RB-SLO-08	Integration Team on-call; Ticket to owning team; SLA 4 h to acknowledge; SEV-3 incident if not acknowledged.
SLO-08-Burn-3d	SLO-08	3d	1x	Low	RB-SLO-08	Integration Team on-call; Ticket to owning team; SLA 1 business day to acknowledge; reviewed at weekly SLO review.
SLO-09-Burn-1h	SLO-09	1h	14.4x	Critical	RB-SLO-09	Preservation Team on-call; Page on-call;

						wake secondary if unacknowledged within 5 min; Incident Commander engaged at 15 min; SEV-1 incident opened.
SLO-09-Burn-6h	SLO-09	6h	6x	High	RB-SLO-09	Preservation Team on-call; Page on-call; IC engaged at 30 min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-09-Burn-1d	SLO-09	1d	3x	Medium	RB-SLO-09	Preservation Team on-call; Ticket to owning team; SLA 4 h to acknowledge; SEV-3 incident if not acknowledged.
SLO-09-Burn-3d	SLO-09	3d	1x	Low	RB-SLO-09	Preservation Team on-call; Ticket to owning team; SLA 1 business day to acknowledge; reviewed at weekly SLO review.
SLO-10-Burn-1h	SLO-10	1h	14.4x	Critical	RB-SLO-10	Security Team on-call; Page on-call; wake secondary if unacknowledged within 5 min; Incident Commander engaged at 15 min; SEV-1 incident opened.

SLO-10-Burn-6h	SLO-10	6h	6x	High	RB-SLO-10	Security Team on-call; Page on-call; IC engaged at 30 min if unacknowledged; SEV-2 incident opened; team lead notified.
SLO-10-Burn-1d	SLO-10	1d	3x	Medium	RB-SLO-10	Security Team on-call; Ticket to owning team; SLA 4 h to acknowledge; SEV-3 incident if not acknowledged.
SLO-10-Burn-3d	SLO-10	3d	1x	Low	RB-SLO-10	Security Team on-call; Ticket to owning team; SLA 1 business day to acknowledge; reviewed at weekly SLO review.

The four severity levels correspond to the platform's incident classification scheme (Volume I Chapter 21): Critical = SEV-1, High = SEV-2, Medium = SEV-3, Low = SEV-4. The Escalation column is the operational form of the incident-response runbook selection logic: it tells the on-call responder who to engage next and on what timeline. The Release Certification Authority SHALL verify, before each release, that the alerting backend contains exactly 40 burn-rate alert rules matching the rows above, and that each rule is wired to the on-call rotation specified in the Escalation column. A missing or misrouted alert rule is a SEV-2 traceability defect and SHALL be remediated before release.

9.5 SLA Traceability

A Service Level Agreement (SLA) is an SLO extended with an external consumer, an explicit measurement protocol, and a penalty for breach. Opus Publica maintains three external SLAs, each grounded in the platform's role in the scholarly publishing ecosystem: an OAI-PMH availability SLA owed to the harvester community (Crossref, OpenAIRE, DOAJ, and any conforming harvester); a Crossref deposit SLA owed to Crossref as the DOI registration agency; and a DOI resolution SLA owed to every reader and citing author who resolves a DOI through doi.org. These three SLAs are the externally visible operationalization of SLO-03, SLO-08, and SLO-01/SLO-06 respectively; the SLA target MAY differ from the SLO target (the SLA

target is always the lower of the SLO target and the consumer's requirement, to preserve an SLO headroom buffer).

SLA	Consumer	Commitment	Measurement	Penalty	Evidence	Owner
SLA-OAI-PMH-Availability	Harvester community (Crossref Metadata Harvester, OpenAIRE, DOAJ, conforming third-party harvesters).	$\geq 99.5\%$ monthly availability of the /oai-pmh endpoint; p99 ListRecords response ≤ 30 s for a 100-record page.	External uptime probe (3rd-party, 5-min cadence) plus internal p99 latency monitor over the same window.	Public posting of the month's OAI-PMH availability report on status.opuspublica.com; email notice to registered harvesters within 72 h of any month the SLA is missed.	EVID-SLA-01 (monthly availability report, signed by Metadata Team Lead)	Metadata Team
SLA-Crossref-Deposit	Crossref (DOI Registration Agency).	$\geq 99\%$ of deposits acknowledged by Crossref within 24 h of publication; $\geq 99.9\%$ within 7 d.	crossref-reconcile-svc polls Crossref daily for deposit acknowledgment status; monthly rollup is the SLA measurement.	Manual re-deposit of any article > 7 d without acknowledgment; quarterly reconciliation report filed with Crossref; public incident report if $> 1\%$ of any month's deposits are > 24 h.	EVID-SLA-02 (monthly Crossref reconciliation report)	Integration Team
SLA-DOI-Resolution	Every reader and citing author who resolves an Opus Publica DOI through doi.org.	$\geq 99.9\%$ resolution success to the article landing page; p95 resolution ≤ 2 s end-to-end (doi.org redirect + landing page render).	Synthetic DOI resolution probe from 3 geographic regions at 1-min cadence; sampling of real resolutions via Edge logs.	Public status-page posting within 1 h of any incident reducing resolution success below 99.5% ; root-cause and remediation report published within 7 d.	EVID-SLA-03 (monthly DOI resolution availability + latency report)	Platform SRE

The SLA Penalty column is the platform's externally visible commitment: a missed SLA SHALL produce a public communication, not merely an internal record. This rule reflects the scholarly-publishing-specific norm that the published record and its infrastructure are a public good whose operators owe transparency to the communities that depend on them. The Release Certification Authority SHALL verify, before each release, that the measurement protocols for all three SLAs are operational and that the evidence artifacts cited above exist and are current. An SLA without a measurement protocol is, for audit purposes, not an SLA.

Chapter 10 — Constitutional Thresholds Traceability

10.1 Purpose

This chapter traces the 103 constitutional thresholds defined by Volume II, Chapter 31 (Constitutional Numbers) of the Engineering Constitution through to their enforcement, monitoring, testing, and runtime evidence. A constitutional threshold is a non-negotiable numerical bound on a platform operation or resource: a latency ceiling, a throughput cap, a size limit, a timeout, a retry budget, a concurrency bound, a queue-depth limit, a storage-retention rule, a reliability target, or a resource-utilization ceiling. The 103 thresholds are organized into ten budget classes (Section 10.2); the matrix in this chapter carries forward the five most operationally critical classes in detail (Latency, Throughput, Size, Time, Reliability; Sections 10.3 through 10.7).

The threshold catalogue is the engineering-mathematical complement of the SLO framework traced in Chapter 9. SLOs express the platform's user-visible service-level commitments; thresholds express the platform's internal operational bounds. An SLO without underlying thresholds is unverifiable (there is no way to know whether the platform can sustain the SLO); a threshold without an underlying SLO is unmotivated (there is no way to know whether the threshold matters). The two regimes are normatively linked: every threshold SHALL trace to at least one SLO whose target it protects, and every SLO SHALL trace to at least one threshold whose enforcement guarantees it. This Matrix carries the threshold-side of that link; Chapter 9 carries the SLO-side.

Where the threshold count within a class is large (Latency carries 18 thresholds; Size carries 15; Time carries 15), the matrix traces every individual threshold. Where the count is smaller and the budget class is operationalized by a single, uniform regime (Retry/Backoff, Concurrency, Queue Depth, Storage/Retention, Resource Utilization), the budget-class summary in Section 10.2 carries the class-level traceability; the individual thresholds are documented in Volume II Chapter 31 and enforced by the runtime gates named in the summary table. An auditor verifying threshold traceability SHALL verify the detailed matrices in Sections 10.3 through 10.7 and SHALL sample at least three thresholds from each of the five classes summarized only in Section 10.2.

10.2 Budget Class Summary

The 103 constitutional thresholds are partitioned into ten budget classes by the kind of quantity they bound. Each class is governed by a single rule (MATH-001 through MATH-010), enforced by one or more runtime gates, and owned by a single engineering function. The matrix below carries the class-level traceability: the threshold count, the enforcement layer(s), the owning function, and the traceability status of the class as a whole. A class is marked Traced only when every individual threshold within it is itself Traced; the class-level status is the AND of its member thresholds' statuses.

Budget Class	Threshold Count	Enforcement Layer	Owner	Status
Latency	18 (4 thresholds each:	CI gate (latency-	Head of Platform	Traced

	p50, p95, p99, hard limit)	budget assert) + runtime (request deadline + histogram monitor)	Engineering	
Throughput	8 (3 thresholds each: max rate, burst, sustained)	Runtime (token-bucket rate limiter at edge) + CI (load test asserts)	Head of Platform Engineering	Traced
Size	15 (2 thresholds each: soft warning, hard limit)	CI (schema validator) + runtime (request validator + storage upload limit)	Head of Data Engineering	Traced
Time	15 (2 thresholds each: timeout, hard limit)	Runtime (per-operation deadline + circuit breaker) + CI (timeout-drift assert)	Head of Platform Engineering	Traced
Retry / Backoff	7 (3 thresholds each: max attempts, base delay, max delay)	Runtime (retry-policy executor in client library) + CI (retry-storm load test)	Head of Platform Engineering	Traced
Concurrency	7 (1 threshold each: max concurrent)	Runtime (semaphore + worker pool cap) + CI (saturation load test)	Head of Platform Engineering	Traced
Queue Depth	8 (3 thresholds each: warning, hard limit, overflow action)	Runtime (queue monitor + DLQ) + CI (back-pressure load test)	Head of Observability	Traced
Storage / Retention	9 (1 threshold each: retention)	Runtime (lifecycle policy executor) + CI (retention audit)	Head of Data Engineering	Traced
Reliability	10 (3 thresholds each: availability target, error budget, MTTR)	Runtime (SLO monitor + on-call) + CI (chaos test)	Head of Observability	Traced
Resource Utilization	6 (3 thresholds each: warning, hard limit, scale action)	Runtime (autoscaler + alert) + CI (capacity headroom test)	Head of Platform Engineering	Traced
TOTAL	103 thresholds across 10 rules	MATH-001 — MATH-010	Office of the Chief Systems Architect	Traced

All ten budget classes are presently Traced. The Release Certification Authority SHALL verify, before each release, that every rule (MATH-001 through MATH-010) has at least one live enforcement gate per the Enforcement Layer column, and that the latest enforcement-gate evidence is no older than the

maximum evidence age (Chapter 13). A budget class whose enforcement gate is dark (failing, missing, or stale) is a release-blocking defect.

The threshold counts within each budget class are not arbitrary: they reflect the operational surface area of the platform and the engineering organization's tolerance for specification ambiguity. The Latency class carries 18 thresholds because the platform performs 18 distinct operations whose latency is user-visible or integration-visible; the Throughput class carries 8 because the platform accepts work in 8 distinct operation classes; the Size class carries 15 because the platform accepts, stores, or produces 15 distinct resource kinds; the Time class carries 15 because 15 operations have non-negotiable abandonment deadlines; the Reliability class carries 10 because 10 user-facing or integration-facing surfaces have availability commitments. The remaining five classes (Retry/Backoff, Concurrency, Queue Depth, Storage/Retention, Resource Utilization) carry fewer thresholds because they govern internal platform behaviour rather than user-visible contracts. The total of 103 is the platform's constitutional number: every threshold in the catalogue is individually justified, individually enforced, and individually traced to runtime evidence. An amendment that adds a threshold (e.g., a new operation class that requires a latency budget) SHALL be processed via the amendment procedure (Volume I Chapter 25) and SHALL be added to this Matrix within 5 business days of the amendment being accepted.

10.3 Latency Threshold Traceability

Latency is the most operationally visible budget class: it is what users feel when they interact with the platform, what harvesters feel when they poll OAI-PMH, what Crossref feels when the platform deposits metadata, and what the editorial team feels when they generate a PDF. The matrix below traces all 18 latency thresholds defined by Volume II Chapter 31, Section 31.1 (Latency Budgets). Each row carries the four percentile thresholds (p50, p95, p99, hard limit) and the full operational chain: the enforcement gate (CI assert + runtime deadline), the monitor that observes latency at runtime, the alert that fires on sustained breach, the test that verifies the threshold in CI, the evidence artifact produced by that test, the owning team, and the status.

TH ID	Operation	p50	p95	p99	Hard Limit	Enforcement	Monitor	Alert	Test	Evidence	Owner	Status
TH-L-01	API read (GET /articles/{id})	40 ms	150 ms	300 ms	500 ms	CI + runtime	apm latency-histogram	LAT-API-READ-P95	TEST-PERF-001	EVID-L-01	API Team	Traced
TH-L-02	API write (POST /submissions)	200 ms	600 ms	1.2 s	2 s	CI + runtime	apm latency-histogram	LAT-API-WRITE-P95	TEST-PERF-002	EVID-L-02	API Team	Traced
TH-L-03	DOI resolution	300 ms	1 s	2 s	5 s	runtime	synthetic probe	LAT-DOI-RESOLV	TEST-PERF-003	EVID-L-03	Platform SRE	Traced

	(doi.org redirect + landing render)							E-P95				
TH-L-04	OAI-PMH ListRecords (100-record page)	500 ms	2 s	5 s	30 s	CI + runtime	oai-pmh latency monitor	LAT-OAI-LISTREC ORDS-P95	TEST-PERF-004	EVID-L-04	Metadata Team	Traced
TH-L-05	Crossref deposit acknowledgment (dispatch to ack)	2 s	30 s	5 min	1 h	runtime	crossref-reconcile-svc	LAT-CROSSREF-ACK-P95	TEST-PERF-005	EVID-L-05	Integration Team	Traced
TH-L-06	PDF generation (10-page article)	3 s	8 s	15 s	30 s	CI + runtime	render-job timer	LAT-PDF-10 PG-P95	TEST-PERF-006	EVID-L-06	Rendering Team	Traced
TH-L-07	PDF generation (50-page article)	12 s	30 s	60 s	120 s	CI + runtime	render-job timer	LAT-PDF-50 PG-P95	TEST-PERF-007	EVID-L-07	Rendering Team	Traced
TH-L-08	HTML render (article landing page)	100 ms	400 ms	800 ms	2 s	CI + runtime	render-job timer	LAT-HTML-RENDER-P95	TEST-PERF-008	EVID-L-08	Rendering Team	Traced
TH-L-09	JATS XML generation (full article)	500 ms	2 s	4 s	10 s	CI + runtime	render-job timer	LAT-JATS-GEN-P95	TEST-PERF-009	EVID-L-09	Rendering Team	Traced
TH-L-10	Storage GET (object retrieval)	30 ms	100 ms	300 ms	1 s	runtime	storage-svc timer	LAT-STORAGE-GET-P95	TEST-PERF-010	EVID-L-10	Storage Team	Traced
TH-L-11	Storage	100 ms	500 ms	1.5 s	5 s	runtime	storage	LAT-	TEST-	EVID-	Storage	Traced

	PUT (object upload, 5 MB)						-svc timer	STOR- PUT- P95	PERF-0 11	L-11	Team	
TH-L-12	DB query simple (PK lookup, articles table)	2 ms	10 ms	30 ms	100 ms	CI + runtime	db query metrics	LAT-DB- SIMPLE -P95	TEST- PERF-0 12	EVID- L-12	Data Enginee ring	Traced
TH-L-13	DB query comple x (full- text search SQL)	50 ms	300 ms	800 ms	2 s	CI + runtime	db query metrics	LAT-DB- COMPL EX-P95	TEST- PERF-0 13	EVID- L-13	Data Enginee ring	Traced
TH-L-14	Cache hit (Redis GET)	0.5 ms	2 ms	5 ms	20 ms	runtime	cache metrics	LAT- CACHE- HIT-P95	TEST- PERF-0 14	EVID- L-14	Platfor m SRE	Traced
TH-L-15	Cache miss (Redis GET miss + DB fallback)	10 ms	50 ms	150 ms	500 ms	runtime	cache metrics	LAT- CACHE- MISS- P95	TEST- PERF-0 15	EVID- L-15	Platfor m SRE	Traced
TH-L-16	Search query (Elastics earch, 10 results)	50 ms	200 ms	500 ms	2 s	CI + runtime	search query metrics	LAT- SEARCH -P95	TEST- PERF-0 16	EVID- L-16	Search Team	Traced
TH-L-17	Auth token validati on (JWT verify, online)	1 ms	5 ms	15 ms	50 ms	runtime	auth- svc metrics	LAT- AUTH- JWT- P95	TEST- PERF-0 17	EVID- L-17	Security Team	Traced
TH-L-18	Audit log append (single event)	1 ms	5 ms	20 ms	100 ms	runtime	audit- svc metrics	LAT- AUDIT- APPEN D-P95	TEST- PERF-0 18	EVID- L-18	Security Team	Traced

The four-percentile schema (p50 / p95 / p99 / hard limit) is the operationalization of Volume II Chapter 31, Rule MATH-001. The p50 column expresses the typical-case budget; p95 is the alerting threshold (sustained breach for 28 days is an SLO-02 burn); p99 is the capacity-planning threshold (sustained breach indicates the platform is approaching saturation); the hard limit is the runtime deadline beyond which the operation is abandoned and counted as failed. An operation whose p95 exceeds its p99 column, or whose p99 exceeds its hard limit, is internally inconsistent and SHALL be corrected before the next release.

10.4 Throughput Threshold Traceability

Throughput thresholds cap the rate at which the platform accepts work. Every operation class has a maximum rate (the sustained ceiling the platform is provisioned to serve), a burst (the peak the platform will accept for short windows via the token bucket), and a sustained rate (the rate at which the platform will continue to serve after the burst credit is exhausted). The matrix below traces all 8 throughput thresholds defined by Volume II Chapter 31, Section 31.2.

TH ID	Operation	Max Rate	Burst	Sustained	Enforcement	Monitor	Test	Evidence	Status
TH-T-01	API read (GET /api/v1/*)	1000 rps	2000 rps	800 rps	runtime rate limiter	rps metric per route	TEST-PERF-020	EVID-T-01	Traced
TH-T-02	API write (POST/PUT/DELETE /api/v1/*)	100 rps	200 rps	80 rps	runtime rate limiter	rps metric per route	TEST-PERF-021	EVID-T-02	Traced
TH-T-03	OAI-PMH ListRecords per harvester IP	5 rps	10 rps	5 rps	runtime rate limiter (per-IP)	oai-pmh rps per IP	TEST-PERF-022	EVID-T-03	Traced
TH-T-04	Crossref deposit dispatch	50 rps	100 rps	30 rps	runtime rate limiter	crossref-deposit rps	TEST-PERF-023	EVID-T-04	Traced
TH-T-05	PDF generation (concurrent render jobs)	10 jobs	20 jobs	10 jobs	worker-pool cap	render-job concurrency	TEST-PERF-024	EVID-T-05	Traced
TH-T-06	Search query (/api/v1/search)	50 rps	100 rps	40 rps	runtime rate limiter	search rps	TEST-PERF-025	EVID-T-06	Traced
TH-T-07	Audit log	5000 eps	10000 eps	5000 eps	queue +	audit eps	TEST-	EVID-T-07	Traced

	append (system-wide)				worker pool	metric	PERF-026		
TH-T-08	Webhook delivery (outbound)	100 rps	300 rps	100 rps	runtime rate limiter + retry	webhook rps + retry metric	TEST-PERF-027	EVID-T-08	Traced

Throughput thresholds are enforced by a token-bucket rate limiter deployed at the edge (for inbound traffic) and by semaphore-bounded worker pools (for internal jobs). The Enforcement column records the mechanism; the Monitor column records the metric that the platform observes to confirm the limiter is functioning. The Release Certification Authority SHALL verify, before each release, that the rate-limiter configuration matches the matrix exactly; a misconfigured limiter (e.g., a burst value of 5000 where the matrix specifies 2000) is a release-blocking defect.

10.5 Size Threshold Traceability

Size thresholds cap the magnitude of resources the platform accepts, stores, or produces. Every resource class has a soft warning (the size at which the platform flags the resource for review) and a hard limit (the absolute ceiling beyond which the resource is refused). The matrix below traces all 15 size thresholds defined by Volume II Chapter 31, Section 31.3. Size thresholds are critical because they bound the platform's storage cost, its render-time budget, and the data-volume its integrations must tolerate.

TH ID	Resource	Limit	Soft Warning	Hard Limit	Enforcement	Test	Evidence	Status
TH-S-01	Manuscript upload (single file)	100 MB	80 MB	100 MB	validator + proxy limit	TEST-SIZE-001	EVID-S-01	Traced
TH-S-02	Generated PDF (final artifact)	50 MB	30 MB	50 MB	render-svc output limit	TEST-SIZE-002	EVID-S-02	Traced
TH-S-03	JATS XML (full article)	10 MB	5 MB	10 MB	schema validator	TEST-SIZE-003	EVID-S-03	Traced
TH-S-04	Authors per article	200	150	200	schema validator	TEST-SIZE-004	EVID-S-04	Traced
TH-S-05	References per article	1500	1000	1500	schema validator	TEST-SIZE-005	EVID-S-05	Traced
TH-S-06	Abstract length (characters)	5000	4000	5000	schema validator	TEST-SIZE-006	EVID-S-06	Traced
TH-S-07	Title length (characters)	500	300	500	schema validator	TEST-SIZE-007	EVID-S-07	Traced
TH-S-08	Keywords	30	20	30	schema	TEST-	EVID-S-08	Traced

	per article				validator	SIZE-008		
TH-S-09	Figure file size (single image)	30 MB	20 MB	30 MB	validator + proxy limit	TEST-SIZE-009	EVID-S-09	Traced
TH-S-10	Supplementary file (single file)	200 MB	150 MB	200 MB	validator + proxy limit	TEST-SIZE-010	EVID-S-10	Traced
TH-S-11	ORCID iDs per author	5	3	5	schema validator	TEST-SIZE-011	EVID-S-11	Traced
TH-S-12	Affiliations per author	10	6	10	schema validator	TEST-SIZE-012	EVID-S-12	Traced
TH-S-13	Pages per article (final PDF)	200	100	200	render-svc output limit	TEST-SIZE-013	EVID-S-13	Traced
TH-S-14	Articles per issue	500	300	500	schema validator	TEST-SIZE-014	EVID-S-14	Traced
TH-S-15	API request body (JSON payload)	5 MB	2 MB	5 MB	validator + proxy limit	TEST-SIZE-015	EVID-S-15	Traced

Size thresholds are enforced at two layers: the schema validator (which rejects oversized content before it enters the platform's domain model) and the proxy/upload limit (which rejects oversized payloads before they reach the application). The dual-layer enforcement is a defence-in-depth measure: a validator bypass (a misconfigured plugin, a malformed request that slips past the schema check) is caught by the proxy limit. An auditor SHALL verify that both layers are configured consistently; a divergence between the validator and the proxy is a release-blocking defect.

10.6 Time Threshold Traceability

Time thresholds cap the duration the platform will spend on a single operation before abandoning it. Every operation has a timeout (the duration after which the operation is abandoned and the caller is notified) and a hard limit (the absolute ceiling beyond which the operation is forcibly terminated even if the caller has not timed out). The matrix below traces all 15 time thresholds defined by Volume II Chapter 31, Section 31.4. Time thresholds are critical because they bound the platform's resource hold time: an operation without a timeout holds a database connection, a worker slot, and a memory allocation indefinitely, which is the primary cause of cascading saturation under load.

TH ID	Operation	Timeout	Hard Limit	Enforcement	Monitor	Test	Status
TH-TM-01	API request (end-to-end)	30 s	30 s	request deadline at gateway	apm request duration	TEST-TIME-001	Traced
TH-TM-02	Crossref	60 s	120 s	client timeout	crossref-	TEST-	Traced

	deposit (single deposit dispatch)			+ circuit breaker	deposit duration	TIME-002	
TH-TM-03	PDF render (single render job)	120 s	180 s	render-job deadline + worker kill	render-job duration	TEST- TIME-003	Traced
TH-TM-04	HTML render (single render job)	60 s	90 s	render-job deadline + worker kill	render-job duration	TEST- TIME-004	Traced
TH-TM-05	JATS XML generation (single render job)	90 s	120 s	render-job deadline + worker kill	render-job duration	TEST- TIME-005	Traced
TH-TM-06	DB transaction (single transaction)	5 s	10 s	transaction timeout	db transaction duration	TEST- TIME-006	Traced
TH-TM-07	DB lock wait (single lock acquisition)	2 s	5 s	lock_timeout session setting	db lock wait metric	TEST- TIME-007	Traced
TH-TM-08	Storage GET (single object retrieval)	10 s	30 s	client timeout + retry	storage-svc duration	TEST- TIME-008	Traced
TH-TM-09	Storage PUT (single object upload)	60 s	180 s	client timeout + multipart	storage-svc duration	TEST- TIME-009	Traced
TH-TM-10	OAI-PMH request (single request)	300 s	600 s	request deadline	oai-pmh duration	TEST- TIME-010	Traced
TH-TM-11	Search query (single Elasticsearch query)	15 s	30 s	client timeout + cancel	search query duration	TEST- TIME-011	Traced
TH-TM-12	Auth token validation (single JWT verify)	5 s	10 s	client timeout + cache fallback	auth-svc duration	TEST- TIME-012	Traced
TH-TM-13	Webhook delivery (single outbound attempt)	30 s	60 s	client timeout + retry	webhook delivery duration	TEST- TIME-013	Traced
TH-TM-14	Workflow	600 s	1200 s	step deadline	workflow-	TEST-	Traced

	step (default per-step)			+ compensatio n	step duration	TIME-014	
TH-TM-15	Invariant monitor check interval	60 s	300 s	scheduler tick + alert on miss	invariant monitor cadence	TEST-TIME-015	Traced

The Hard Limit column is always equal to or greater than the Timeout column: the timeout is the duration the caller experiences; the hard limit is the duration the platform internally enforces to reclaim resources even if the caller has gone away. An operation whose hard limit is less than its timeout is internally inconsistent (the platform would kill the operation before the caller knew it had failed) and SHALL be corrected. The TH-TM-15 row is unusual in that its operation is the monitor itself rather than a user-facing operation: the invariant-monitor check interval is bounded because a monitor that runs too slowly is itself a reliability defect (it produces stale evidence, which Volume II Chapter 20 treats as missing evidence).

10.7 Reliability Threshold Traceability

Reliability thresholds are the third regime in the platform's service-level hierarchy (the first being the SLOs of Chapter 9, the second being the constitutional thresholds of this chapter). Each reliability threshold binds a user-facing or integration-facing surface to an availability target, an error budget, and a Mean Time To Recover (MTTR) target. The matrix below traces all 10 reliability thresholds defined by Volume II Chapter 31, Section 31.9. Reliability thresholds differ from SLOs in that they specify the operational response (MTTR) in addition to the user-visible commitment (availability); the SLO is what the user sees, the reliability threshold is what the engineering organization commits to internally.

TH ID	Surface	Availability Target	Error Budget	MTTR	Enforceme nt	Monitor	Evidence	Status
TH-R-01	Public website	99.9 %	43.2 min/mo	30 min	SLO monitor + on-call	SLO-01 monitor	EVID-R-01	Traced
TH-R-02	REST + GraphQL API (/api/v1/*)	99.9 %	43.2 min/mo	30 min	SLO monitor + on-call	SLO-02 monitor	EVID-R-02	Traced
TH-R-03	OAI-PMH endpoint (/oai-pmh)	99.5 %	216 min/mo	60 min	SLO monitor + on-call	SLO-03 monitor	EVID-R-03	Traced
TH-R-04	Crossref deposit (outbound)	99 % within 24 h	1 % > 24 h	4 h	reconcile job + on-call	SLO-08 monitor	EVID-R-04	Traced
TH-R-05	DOI resolution	99.9 %	43.2 min/mo	15 min	synthetic probe + on-	SLO-01/06 monitor	EVID-R-05	Traced

	(doi.org redirect + landing)				call			
TH-R-06	Storage layer (object store)	99.99 %	4.32 min/mo	30 min	durability probe + on-call	SLO-06 monitor	EVID-R-06	Traced
TH-R-07	Database (primary)	99.95 %	21.6 min/mo	15 min	replica health + on-call	DB health monitor	EVID-R-07	Traced
TH-R-08	Search index (Elasticsearch)	99.5 %	216 min/mo	60 min	cluster health + on-call	search health monitor	EVID-R-08	Traced
TH-R-09	Audit log subsystem	99.99 %	4.32 min/mo	15 min	completeness monitor + on-call	SLO-10 monitor	EVID-R-09	Traced
TH-R-10	Preservation receipts (CLOCKSS/LOCKSS/Portico)	100 % within 30 d	0 (immediate)	24 h	receipt reconciler + on-call	SLO-09 monitor	EVID-R-10	Traced

The Error Budget column is the operational form of the Availability Target: it is the maximum time per month the surface MAY be in error while still meeting its target. For TH-R-10 (Preservation receipts), the error budget is zero (any published artifact without a preservation receipt within 30 days exhausts the budget immediately), reflecting the platform's commitment that no published artifact is ever without a preservation copy. The MTTR column binds the engineering organization to a recovery duration: a surface whose availability is 99.9 % but whose MTTR exceeds 30 minutes is in breach of TH-R-0x even if its availability target is met, because an outage that takes longer than MTTR to recover is, by definition, an outage that exceeded the platform's recovery commitment.

10.8 Cross-Threshold Dependencies

The 103 thresholds are not independent: they interact, and a change to one threshold can silently invalidate another. This section traces the most operationally significant cross-threshold dependencies so that an engineer amending a threshold knows which other thresholds to re-verify. The dependencies recorded here are normative: an amendment to a threshold SHALL trigger a re-verification of every threshold named in its Depends On column, and the re-verification SHALL be recorded as evidence before the amendment is merged.

The most pervasive cross-threshold dependency is between the latency budget (Section 10.3) and the time budget (Section 10.6). The latency p99 of an operation is, in effect, the operation's expected

maximum duration under normal load; the time budget's hard limit is the duration beyond which the platform abandons the operation. A time budget hard limit set lower than the latency p99 is internally inconsistent: the platform would abandon operations that, by the latency budget's own admission, are expected to take that long under tail-load conditions. The engineering rule is that the time hard limit SHALL be at least 2x the latency p99, to provide headroom for non-typical load. An auditor SHALL verify this 2x invariant for every operation that appears in both Section 10.3 and Section 10.6 (the join is by Operation column).

The second pervasive dependency is between the throughput budget (Section 10.4) and the concurrency budget (Volume II Chapter 31, Section 31.6). The throughput sustained rate SHALL equal, at most, the concurrency cap divided by the operation's p50 latency: a platform that allows 10 concurrent operations each taking 200 ms p50 cannot sustain more than 50 operations per second. An amendment to either the concurrency cap or the latency p50 of an operation SHALL trigger a re-derivation of the throughput sustained rate. The Release Certification Authority SHALL verify this invariant before each release.

The third pervasive dependency is between the size budget (Section 10.5) and the latency budget (Section 10.3). The latency threshold for a render operation (e.g., TH-L-06 PDF generation 10-page) is calibrated against a representative input size; an amendment to the size threshold (e.g., raising the maximum manuscript upload from 100 MB to 200 MB) invalidates the latency threshold because larger inputs take longer to process. The engineering rule is that a size amendment SHALL trigger a re-benchmarking of every operation that consumes that resource; the re-benchmark SHALL produce fresh evidence for the latency threshold before the size amendment is merged.

The fourth pervasive dependency is between the reliability budget (Section 10.7) and the SLO framework (Chapter 9). The reliability threshold's Availability Target SHALL equal or exceed the corresponding SLO's target: a reliability threshold set below the SLO target means the platform's internal commitment is weaker than its user-visible commitment, which is a defect. The matrix in Section 10.7 is constructed so that every reliability threshold's Availability Target equals the corresponding SLO's target (or, where the SLO has a 100 % target, the reliability threshold is the operational form of that target with a concrete error budget of zero and a concrete MTTR). An amendment to an SLO target SHALL trigger a re-derivation of the corresponding reliability threshold.

The fifth and final pervasive dependency is between the queue-depth budget (Volume II Chapter 31, Section 31.7) and the time budget (Section 10.6). A queue's hard limit SHALL be calibrated so that the maximum queue wait (the queue depth divided by the drain rate) does not exceed the consumer operation's time hard limit. An amendment to either the queue depth or the consumer's time hard limit SHALL trigger a re-verification that the maximum queue wait remains within the consumer's time budget. The Release Certification Authority SHALL verify this invariant for every queue that has a documented consumer.

These five dependencies are not exhaustive; they are the five most operationally significant. The full cross-threshold dependency graph is maintained by the threshold catalogue tool (Vol II Ch 31, Section 31.11), which the Release Certification Authority SHALL run before each release to detect any

amendment that has not triggered the required re-verification. An amendment that bypasses the catalogue tool is a process defect, irrespective of whether the amendment itself was correct.

Chapter 11 — Volume I Constitutional Requirements Traceability

11.1 Purpose

This chapter traces the twenty-five chapters of Volume I of the Engineering Constitution through to their operational realization. Each Volume I chapter is a requirement class: a coherent set of constitutional requirements bound to a single domain (Product Mission, Engineering Philosophy, System Context, System Architecture, Data Constitution, Canonical Registry, ADRs, Repository Constitution, API Constitution, Workflow Constitution, Publication State Machine, Storage Constitution, Rendering Constitution, Metadata Constitution, Template Constitution, Security Constitution, Performance Constitution, Observability Constitution, Testing Constitution, Runtime Verification Constitution, Failure Constitution, Certification Constitution, Knowledge Graph Constitution, Documentation Constitution, Future Evolution).

The matrix in Section 11.2 carries one row per Volume I chapter. Each row identifies the chapter's key requirements, the architecture service(s) that realize them, the Playbook chapter(s) that operationalize them, the ADR(s) that justify the architecture decisions, the key APIs that expose them, the key database tables that back them, the key tests that verify them, the evidence artifact that demonstrates their satisfaction, the owning team, and the traceability status. Together, the twenty-five rows form the master traceability chain from Volume I's constitutional intent to runtime evidence: an auditor sampling any Volume I chapter SHALL be able to traverse the row left-to-right and arrive at a concrete, current evidence artifact without leaving the matrix.

Section 11.3 traces the cross-chapter dependencies: which Volume I chapters depend normatively on which others. The dependency graph is acyclic (a chapter MAY NOT depend on itself, directly or transitively); its topological order is the order in which the chapters are presented in Volume I, which is therefore the order in which the Constitution is best read. An auditor verifying constitutional completeness SHALL verify that every cross-chapter dependency is documented in Section 11.3 and that no undocumented dependency exists in the implementation (an implementation that imports from a chapter it does not declare a dependency on is a traceability defect).

11.2 The Volume I Chapter Traceability Matrix

The matrix below carries the full 25-row traceability chain for Volume I. Cell content is specific: the Architecture column names the service(s) that realize the chapter's requirements; the Playbook Ch column names the chapter(s) that operationalize those requirements day-to-day; the ADR(s) column names the specific ADR numbers that justify the architecture decisions; the Key APIs column names the API endpoints; the Key DB Tables column names the database tables; the Key Tests column names the test identifier family; the Evidence column names the evidence artifact ID. Cells with '—' indicate the link

is not applicable (for example, Volume I Chapter 2 Engineering Philosophy has no API endpoint because it is a foundational chapter that binds all APIs but exposes none itself).

Ch #	Chapter Title	Key Requirements	Architecture	Playbook Ch	ADR(s)	Key APIs	Key DB Tables	Key Tests	Evidence	Owner	Status
1	Product Mission	Mission statement; north-star metrics; user personas; success criteria.	Cross-cutting (all services)	PB Ch 2	ADR-001	—	—	TEST-MISSION-001	EVID-V1-CH01	Chief Systems Architect	Traced
2	Engineering Philosophy	Runtime truth principles; production acceptance gate; global expansion rule; completeness criterion.	Cross-cutting (governance)	PB Ch 1, PB Ch 2	ADR-002	—	—	TEST-PHIL-001	EVID-V1-CH02	Chief Systems Architect	Traced
3	System Context	External integrations (Crossref, ORCID, CLOCKSS, DOAJ, OpenAI, RE, OAI-PMH);	integration-gateway; crossref-deposit-svc; oai-pmh-svc; orcid-svc; preserv	PB Ch 14	ADR-010	/ crossref /*, /orcid/*, /oai-pmh/*	integration_log, crossref_deposit, preservation_receipt	TEST-INT-001..010	EVID-V1-CH03	Integration Team	Traced

		system boundary; trust boundaries.	ation-svc								
4	System Architecture	Service decomposition; bounded context; deployment topology; inter-service communication contracts.	api-gateway; workflow-svc; registry-svc; storage-svc; rendering-svc; metadata-svc; audit-svc; search-svc	PB Ch 3, PB Ch 7	ADR-003, ADR-004	— (cross-cutting)	—	TEST-ARCH-001..010	EVID-V1-CH04	Principal Architect	Traced
5	Data Constitution	Canonical data model; immutability of published artifacts; append-only audit log; data ownership; data lineage.	registry-svc; audit-svc; storage-svc	PB Ch 8	ADR-005, ADR-006	/ articles /*, /audit/*	articles, article_version, audit_log, canonical_fields	TEST-DATA-001..020	EVID-V1-CH05	Data Architecture	Traced
6	Canonical Registry	Single source of truth for canonical	registry-svc	PB Ch 9	ADR-007	/ registry /*, /articles/*, /issues/	articles, issues, authors, doi_assignmentmen	TEST-REG-001..030	EVID-V1-CH06	Registry Team	Traced

		objects (Articles, Issues, Authors, DOIs, Affiliations); registry invariants; DOI uniqueness.				*, /authors/*	ts, affiliations				
7	Architectural Decision Records	Every architecture decision justified by an ADR; ADR lifecycle (proposed → accepted → deprecated → superseded); ADR numbering; ADR review gate.	adr-repo (docs service)	PB Ch 5	ADR-001..040+	/adrs/*	adr_index	TEST-ADR-001..005	EVID-V1-CH07	Principal Architect	Traced
8	Repository Constitution	Repository layout; monorepo vs polyrepo; branch protection	ci-pipeline; repo-tooling	PB Ch 3, PB Ch 4, PB Ch 5, PB Ch 6	ADR-008, ADR-009	—	—	TEST-REPO-001..010	EVID-V1-CH08	Platform SRE	Traced

		on; Cl gating; pre- merge checks; build- agent policy.									
9	API Constit ution	RESTful contrac t; URL versioni ng; idemp tency keys; rate limiting ; OpenA PI 3.1; pagina tion; error model; deprec ation policy.	api- gatewa y	PB Ch 12	ADR-01 1, ADR-01 2, ADR-01 3	All /api/v1 /* endpoi nts	api_key s, idemp tency_r ecords, rate_li mit_bu ckets, api_de precati ons	TEST- API-001 ..050	EVID- V1- CH09	API Team	Traced
10	Workfl ow Constit ution	Workfl ow definiti on; state machin es; idemp tent steps; compe nsation ; saga pattern ; workflo w versioni ng.	workflo w-svc; state- machin e- engine	PB Ch 11	ADR-01 4, ADR-01 5	/ workflo ws/*, /tasks/ *	workflo ws, workflo w_step s, workflo w_even ts	TEST- WF-001 ..040	EVID- V1- CH10	Workfl ow Team	Traced

11	Publication State Machine	States (Draft → Submitted → Under Review → Revised → Accepted → Production → Published → Retracted); transitions; immutability of Published; transition guards.	publication-sm-svc	PB Ch 10	ADR-016	/articles/{id}/state,/articles/{id}/transition	article_states, state_transitions	TEST-PSM-001..020	EVID-V1-CH11	Workflow Team	Traced
12	Storage Constitution	Object storage; content-addressed (SHA-256); immutable published artifacts; tiered storage; retention; preserv	storage-svc; preservation-svc	PB Ch 8, PB Ch 13	ADR-017, ADR-018	/storage/*,/artifacts/*	storage_objects, storage_tiers, retention_policies, preservation_receipts	TEST-STOR-001..025	EVID-V1-CH12	Storage Team	Traced

		ation copies.									
13	Renderi ng Constit ution	Idempo tent renderi ng; determi nistic output; reprod ucible builds; render fidelity; PDF/A, JATS, HTML outputs ; render- pipelin e stages.	renderi ng-svc; jats- gen- svc; pdf- gen- svc; html- render- svc	PB Ch 15	ADR-01 9, ADR-02 0	/ render/ *, /pdf/*, /jats/*, /html/*	render_ jobs, render_ artifact s	TEST- REND-0 01..030	EVID- V1- CH13	Renderi ng Team	Traced
14	Metada ta Constit ution	Dublin Core, JATS, Crossre f/CitePr oc metada ta; metada ta comple teness; crossw alk fidelity; metada ta versioni ng.	metada ta-svc; crossw alk-svc	PB Ch 14	ADR-02 1, ADR-02 2	/ metada ta/*, /crossw alk/*	metada ta_reco rds, crossw alk_ma ppings	TEST- META-0 01..040	EVID- V1- CH14	Metada ta Team	Traced
15	Templa te Constit ution	Templa te versioni ng; sandbo	templat e-svc	PB Ch 15	ADR-02 3	/ templat es/*	templat es, templat e_versi	TEST- TPL-00 1..015	EVID- V1- CH15	Renderi ng Team	Traced

		xed renderi ng; templat e review; templat e inherita nce; templat e variable s contrac t.					ons				
16	Securit y Constit ution	AuthN/ AuthZ; OAuth2 /OIDC; RBAC; secrets manag ement; encryp tion at rest/tra nsit; vulnera bility manag ement; audit logging.	auth- svc; secrets- svc; iam-svc	PB Ch 16	ADR-02 4, ADR-02 5, ADR-02 6	/auth/ *, /oauth/ *, /iam/*, /secret s/*	users, roles, permiss ions, session s, secrets _refs	TEST- SEC-00 1..060	EVID- V1- CH16	Securit y Team	Traced
17	Perfor mance Constit ution	Perfor mance budget s (latenc y, throug hput); capacit y plannin g; load testing;	apm- collecto r; capacit y- planner ; load- test- harness	PB Ch 17	ADR-02 7	—	perfor mance_ budget s, capacit y_recor ds, load_te st_runs	TEST- PERF-0 01..050	EVID- V1- CH17	Perfor mance Enginee ring	Traced

		profilin g; perfor mance regressi on detecti on.									
18	Observ ability Constit ution	Logs, metrics , traces (three pillars); structur ed logging; correla tion IDs; metric naming ; trace samplin g; alert routing.	observa bility- stack (logs, metrics , traces)	PB Ch 18	ADR-02 8, ADR-02 9	—	log_ind ex, metric_ series, trace_s amples, alert_r outes	TEST- OBS-00 1..030	EVID- V1- CH18	Reliabili ty Team	Traced
19	Testing Constit ution	Test pyrami d; coverag e thresho lds (unit 80 %, integra tion 60 %, e2e 100 % critical paths); mutatio n testing; fixture manag ement; test	test- harness ; fixture- svc	PB Ch 6, PB Ch 19	ADR-03 0, ADR-03 1	—	test_re sults, coverag e_repor ts, fixtures	TEST- TEST-00 1..020	EVID- V1- CH19	QA Team	Traced

		data hygiene									
20	Runtime Verification Constitution	Five pillars (runtime, visual, database, storage, API); invariant monitors; evidence freshness; evidence storage	invariant-monitor-svc; evidence-store	PB Ch 20	ADR-032, ADR-033	/invariants/*, /evidence/*	invariant_results, evidence_artifacts	TEST-RV-001.040	EVID-V1-CH20	Reliability Team	Traced
21	Failure Constitution	Failure modes; incident classification (SEV-1..4); MTTR/MTTA targets; postmortem culture; blameless reviews; incident command system.	incident-svc; postmortem-repo	PB Ch 21, PB Ch 22	ADR-034	/incidents/*, /postmortems/*	incidents, postmortems, action_items	TEST-FAIL-001.020	EVID-V1-CH21	Reliability Team	Traced

22	Certification Constitution	Release certification; seven pillars of certification; RCA sign-off; rollback readiness; certification record lifecycle.	certification-svc	PB Ch 23	ADR-035	/certifications/*	certification_records, rca_signoffs, rollback_drills	TEST-CERT-001..015	EVID-V1-CH22	Release Certification Authority	Traced
23	Knowledge Graph Constitution	13-node traceability chain; node/edge types; integrity rules; graph queries; orphan-node detection; graph reconciliation.	kg-store (graph DB); traceability-svc	PB Ch 24	ADR-036, ADR-037	/trace/*, /graph/*	kg_nodes, kg_edges, graph_reconciliations	TEST-KG-001..020	EVID-V1-CH23	Reliability Team	Traced
24	Documentation Constitution	Doc-as-code; audience-driven docs; doc review	docs-svc; docs-pipeline	PB Ch 25	ADR-038	/docs/*	docs_index, doc_versions, doc_debt	TEST-DOC-001..010	EVID-V1-CH24	Developer Experience	Traced

		gates; doc debt trackin g; doc freshne ss; doc coverag e.									
25	Future Evoluti on	Amend ment process ; deprec ation policy; backwa rd compa tibility; versioni ng strateg y; sunset proced ures.	govern ance- svc	PB Ch 26	ADR-03 9, ADR-04 0	/ amend ments/ *, /deprec ations/ *, /sunset s/*	amend ments, deprec ations, sunsets	TEST- EVOL-0 01..010	EVID- V1- CH25	Chief System s Archite ct	Traced

All twenty-five Volume I chapters are presently Traced. The Status column reflects the static verification (every applicable cell populated) and the dynamic verification (every evidence artifact current per Chapter 13 maximum evidence age). The Release Certification Authority SHALL NOT certify a release in which any Volume I chapter row is marked Gap, Untraced, or Stale; an untraced Volume I chapter is a chapter whose constitutional requirements cannot be proven implemented, which is a release-blocking defect regardless of severity.

Three observations on the matrix above bear explicit mention. First, chapters 1, 2, 4, 7, 8, 17, 18, 19, and 24 have an API column of '—' because they are foundational or cross-cutting: they bind the platform's APIs and services but do not themselves expose a domain API. Second, the Owner column reflects the platform's accountability assignment: every chapter has a single owning function, and that function is the named accountable party for the chapter's traceability. Third, the Playbook Ch column is the operational bridge from Volume I to the Engineering Playbook: a Volume I chapter without a corresponding Playbook chapter is a chapter whose requirements are specified but not operationalized, which is a constitutional defect (the Global Expansion Rule, Volume I Chapter 2).

11.3 Volume I Cross-Chapter Dependencies

The twenty-five Volume I chapters are not independent: they form a directed acyclic graph of normative dependencies. A dependency from chapter A to chapter B means that the requirements of chapter A cannot be satisfied without the requirements of chapter B being satisfied first. The matrix below records the dependencies in three columns: the chapter under consideration, the chapters it depends on (its predecessors in the topological order), and the chapters that depend on it (its successors). The Type column distinguishes normative dependencies (a violation of the predecessor breaks the successor) from informative dependencies (the predecessor provides context the successor uses but does not require).

Chapter	Depends On	Depended By	Type
Ch 1 Product Mission	—	Ch 2, 3, 4, 5, 6, 25	Normative
Ch 2 Engineering Philosophy	Ch 1	Ch 3, 4, 5, 7, 8, 19, 20, 22, 25	Normative
Ch 3 System Context	Ch 1, 2	Ch 4, 9, 14, 16	Normative
Ch 4 System Architecture	Ch 1, 2, 3	Ch 5, 6, 9, 10, 11, 12, 16, 17, 18, 23	Normative
Ch 5 Data Constitution	Ch 4	Ch 6, 12, 14, 18, 23	Normative
Ch 6 Canonical Registry	Ch 4, 5	Ch 11, 14, 23	Normative
Ch 7 ADRs	Ch 2, 4	Ch 8, 24, 25	Normative
Ch 8 Repository Constitution	Ch 7	Ch 19, 24	Normative
Ch 9 API Constitution	Ch 4, 5	Ch 10, 16, 17, 18	Normative
Ch 10 Workflow Constitution	Ch 5, 9	Ch 11, 18, 21, 23	Normative
Ch 11 Publication State Machine	Ch 5, 6, 10	Ch 14, 18, 20, 21	Normative
Ch 12 Storage Constitution	Ch 5	Ch 13, 20	Normative
Ch 13 Rendering Constitution	Ch 12, 15	Ch 11 (production stage), 20	Normative
Ch 14 Metadata Constitution	Ch 5, 6	Ch 3 (crosswalks), 18, 23	Normative
Ch 15 Template Constitution	Ch 13	Ch 13	Normative
Ch 16 Security Constitution	Ch 4, 9	Ch 18, 21, 22	Normative
Ch 17 Performance Constitution	Ch 4, 9	Ch 18, 22	Normative
Ch 18 Observability Constitution	Ch 4, 9, 16, 17	Ch 20, 21, 22	Normative
Ch 19 Testing Constitution	Ch 8, 9	Ch 20, 22	Normative
Ch 20 Runtime Verification Constitution	Ch 18, 19	Ch 22, 23	Normative
Ch 21 Failure Constitution	Ch 18, 20	Ch 22	Normative

Ch 22 Certification Constitution	Ch 19, 20, 21	Ch 23	Normative
Ch 23 Knowledge Graph Constitution	All (cross-cutting)	Ch 24, 25	Normative
Ch 24 Documentation Constitution	Ch 7, 8, 23	Ch 25	Informative
Ch 25 Future Evolution	All (cross-cutting)	— (terminal)	Normative

The dependency graph is acyclic, and its topological order is the order in which the chapters are presented in Volume I: Chapter 1 has no predecessors and is the root of the graph; Chapter 25 has no successors and is the terminal node. This ordering is deliberate. It means that an engineer reading Volume I from front to back encounters each chapter's predecessors before the chapter itself, so each chapter can be understood in terms of concepts already introduced. It also means that an engineer verifying the Constitution's internal consistency can do so in a single pass: a forward reference (a chapter citing a successor) is a defect, because it cites a concept not yet introduced. The Knowledge Graph Constitution (Chapter 23) depends normatively on every other chapter because it is the traceability spine that links them all; the Future Evolution chapter (Chapter 25) similarly depends on every other chapter because it governs how they may be amended. The Documentation Constitution (Chapter 24) is the only chapter with an Informative dependency: it depends on Chapters 7, 8, and 23 for context (the ADR process, the repository layout, and the traceability graph that documentation indexes), but its own requirements can be satisfied without those chapters being satisfied first.

An auditor verifying the cross-chapter dependency graph SHALL confirm three properties. First, that the graph is acyclic (no chapter transitively depends on itself); a cycle would indicate a circular constitutional dependency, which is a defect. Second, that every implementation-level dependency (a service in one chapter's architecture column that imports from or calls a service in another chapter's architecture column) is reflected in the dependency graph; an undocumented implementation dependency is a traceability defect. Third, that every normative dependency has a corresponding test (the successor chapter's test suite SHALL include at least one test that fails if the predecessor chapter's requirements are violated); a normative dependency without a test is an unverifiable dependency, which Volume I Chapter 2 (Foundational Principle I: Runtime Truth) forbids.

11.4 Volume I Coverage by Architecture Service

The matrix in Section 11.2 traces Volume I chapters vertically (one row per chapter, all chapters enumerated). This section traces the same data horizontally: for each architecture service that appears in the Architecture column of Section 11.2, which Volume I chapters it realizes. The horizontal trace is the operational complement to the vertical trace: it tells the engineer responsible for a service which constitutional requirements their service is accountable for, and it tells the auditor which services to sample when verifying a particular Volume I chapter. A service that appears in many chapter rows is a service with broad constitutional accountability; a service that appears in only one row is a service with

narrow, specialized accountability. Both are legitimate; what matters is that every service's constitutional coverage is documented.

Architecture Service	Volume I Chapters Realized	Test Family	Owner
api-gateway	Ch 4 (Architecture), Ch 9 (API), Ch 16 (Security), Ch 17 (Performance), Ch 18 (Observability)	TEST-API-*, TEST-SEC-*, TEST-PERF-*	API Team
workflow-svc	Ch 4 (Architecture), Ch 10 (Workflow), Ch 11 (PSM)	TEST-WF-*, TEST-PSM-*	Workflow Team
registry-svc	Ch 4 (Architecture), Ch 5 (Data), Ch 6 (Canonical Registry), Ch 11 (PSM), Ch 14 (Metadata)	TEST-REG-*, TEST-DATA-*	Registry Team
storage-svc	Ch 4 (Architecture), Ch 5 (Data), Ch 12 (Storage)	TEST-STOR-*	Storage Team
rendering-svc (+ jats-gen, pdf-gen, html-render)	Ch 4 (Architecture), Ch 13 (Rendering), Ch 15 (Templates)	TEST-REND-*, TEST-TPL-*	Rendering Team
metadata-svc (+ crosswalk-svc)	Ch 5 (Data), Ch 6 (Registry), Ch 14 (Metadata)	TEST-META-*, TEST-REG-*	Metadata Team
audit-svc	Ch 5 (Data), Ch 16 (Security), Ch 18 (Observability)	TEST-DATA-*, TEST-SEC-*, TEST-OBS-*	Security Team
search-svc	Ch 4 (Architecture), Ch 9 (API)	TEST-API-*, TEST-ARCH-*	Search Team
auth-svc (+ iam-svc, secrets-svc)	Ch 9 (API), Ch 16 (Security)	TEST-SEC-*, TEST-API-*	Security Team
invariant-monitor-svc	Ch 18 (Observability), Ch 20 (Runtime Verification)	TEST-OBS-*, TEST-RV-*	Reliability Team
evidence-store	Ch 8 (Repository), Ch 20 (Runtime Verification), Ch 22 (Certification)	TEST-RV-*, TEST-CERT-*	Reliability Team
certification-svc	Ch 20 (RV), Ch 22 (Certification), Ch 23 (Knowledge Graph)	TEST-CERT-*, TEST-KG-*	Release Certification Authority
traceability-svc (+ kg-store)	Ch 23 (Knowledge Graph), Ch 24 (Documentation)	TEST-KG-*, TEST-DOC-*	Reliability Team
incident-svc (+ postmortem-repo)	Ch 18 (Observability), Ch 21 (Failure)	TEST-OBS-*, TEST-FAIL-*	Reliability Team
preservation-svc	Ch 3 (System Context), Ch 12 (Storage)	TEST-INT-*, TEST-STOR-*	Preservation Team

crossref-deposit-svc (+ crossref-reconcile-svc)	Ch 3 (System Context), Ch 14 (Metadata)	TEST-INT-*, TEST-META-*	Integration Team
oai-pmh-svc (+ indexer)	Ch 3 (System Context), Ch 14 (Metadata)	TEST-INT-*, TEST-META-*	Metadata Team
ci-pipeline (+ repo-tooling)	Ch 8 (Repository), Ch 19 (Testing)	TEST-REPO-*, TEST-TEST-*	Platform SRE
apm-collector (+ capacity-planner)	Ch 17 (Performance), Ch 18 (Observability)	TEST-PERF-*, TEST-OBS-*	Performance Engineering
docs-svc (+ docs-pipeline)	Ch 7 (ADRs), Ch 24 (Documentation)	TEST-ADR-*, TEST-DOC-*	Developer Experience
governance-svc	Ch 2 (Philosophy), Ch 25 (Future Evolution)	TEST-PHIL-*, TEST-EVOL-*	Chief Systems Architect

Every architecture service in the platform's service catalogue appears in the matrix above. An auditor verifying service-level constitutional coverage SHALL confirm that the service catalogue (maintained by the Platform SRE team) and the matrix above agree: a service in the catalogue but not in the matrix is a service whose constitutional accountability is undocumented, which is a defect; a service in the matrix but not in the catalogue is a service that has been retired without its traceability row being removed, which is also a defect.

11.5 Volume I Coverage by Owning Team

The Owner column of Section 11.2 assigns each Volume I chapter to a single accountable engineering function. This section inverts that view: for each owning team, the matrix below lists the Volume I chapters in its portfolio, the total evidence artifacts it is accountable for producing, and the aggregate traceability status of its portfolio. The team-portfolio view is the basis for the engineering leadership team's quarterly review: each team lead SHALL attest, at the quarterly review, that every chapter in their portfolio is Traced and that every evidence artifact is current.

Owning Team	Volume I Chapter Portfolio	Evidence Artifacts	Status
Chief Systems Architect	Ch 1 (Mission), Ch 2 (Philosophy), Ch 25 (Future Evolution)	EVID-V1-CH01, EVID-V1-CH02, EVID-V1-CH25	Traced
Principal Architect	Ch 4 (Architecture), Ch 7 (ADRs)	EVID-V1-CH04, EVID-V1-CH07	Traced
Platform SRE	Ch 8 (Repository)	EVID-V1-CH08	Traced
API Team	Ch 9 (API)	EVID-V1-CH09	Traced
Workflow Team	Ch 10 (Workflow), Ch 11 (PSM)	EVID-V1-CH10, EVID-V1-CH11	Traced
Storage Team	Ch 12 (Storage)	EVID-V1-CH12	Traced

Rendering Team	Ch 13 (Rendering), Ch 15 (Templates)	EVID-V1-CH13, EVID-V1-CH15	Traced
Metadata Team	Ch 14 (Metadata)	EVID-V1-CH14	Traced
Security Team	Ch 16 (Security)	EVID-V1-CH16	Traced
Performance Engineering	Ch 17 (Performance)	EVID-V1-CH17	Traced
Reliability Team	Ch 18 (Observability), Ch 20 (Runtime Verification), Ch 21 (Failure), Ch 23 (Knowledge Graph)	EVID-V1-CH18, EVID-V1-CH20, EVID-V1-CH21, EVID-V1-CH23	Traced
QA Team	Ch 19 (Testing)	EVID-V1-CH19	Traced
Release Certification Authority	Ch 22 (Certification)	EVID-V1-CH22	Traced
Developer Experience	Ch 24 (Documentation)	EVID-V1-CH24	Traced
Data Architecture	Ch 5 (Data)	EVID-V1-CH05	Traced
Registry Team	Ch 6 (Canonical Registry)	EVID-V1-CH06	Traced
Integration Team	Ch 3 (System Context)	EVID-V1-CH03	Traced

The portfolio distribution above reflects a deliberate design choice: reliability-sensitive chapters (Observability, Runtime Verification, Failure, Knowledge Graph) are concentrated in the Reliability Team, which provides the engineering organization with a single accountable function for the platform's traceability spine. Chapters with broad cross-cutting impact (Mission, Philosophy, Future Evolution) are held by the Chief Systems Architect, reflecting that the Office of the Chief Systems Architect is the constitutional authority. The remaining chapters are distributed across the engineering teams by domain, with each team owning the chapters whose requirements their services primarily realize. No team owns more than four Volume I chapters, ensuring that constitutional accountability is distributed rather than concentrated in a single function below the Chief Systems Architect.

An auditor verifying team-portfolio coverage SHALL confirm three properties. First, that every Volume I chapter appears in exactly one team's portfolio (no chapter is owned by zero teams; no chapter is owned by more than one). Second, that the team portfolios above and the Owner column of Section 11.2 agree (a chapter whose Section 11.2 Owner does not match its Section 11.5 portfolio is a traceability defect). Third, that every team with a non-empty portfolio has a named team lead who has signed the most recent quarterly attestation; a portfolio without a signed attestation is treated, for audit purposes, as Untraced.

The three views of this chapter — the per-chapter matrix (Section 11.2), the cross-chapter dependency graph (Section 11.3), the per-service coverage matrix (Section 11.4), and the per-team portfolio matrix (Section 11.5) — are four projections of the same underlying traceability graph maintained by the Knowledge Graph Constitution (Volume I Chapter 23). An auditor verifying the Volume I chapter traceability SHALL verify all four projections against each other: the per-chapter matrix SHALL agree with

the per-service matrix on which services realize which chapters; the per-service matrix SHALL agree with the per-team matrix on which teams own which services; the cross-chapter dependency graph SHALL agree with the per-chapter matrix on which chapters exist. A disagreement between any two projections is a traceability defect and SHALL be remediated before the next release. The Release Certification Authority SHALL verify this cross-projection consistency at every release certification cycle, and the verification SHALL be recorded as evidence in the certification record (Volume I Chapter 22).

A final note on the relationship between this chapter and Chapter 12 of this Matrix (which traces Volume II chapters 26 through 36). Volume I and Volume II are not independent documents: Volume II operationalizes Volume I, extending the foundational chapters' requirements into the runtime, invariant, contract, registry, event, threshold, and SLO domains. An auditor verifying the platform's constitutional traceability SHALL treat Chapters 11 and 12 of this Matrix as a single audit unit: every Volume I chapter traced in Chapter 11 SHALL be cross-referenced against the Volume II chapters that operationalize it (traced in Chapter 12), and every Volume II chapter traced in Chapter 12 SHALL be cross-referenced against the Volume I chapter that authorizes it. A Volume II chapter without an authorizing Volume I chapter is a constitutional overreach; a Volume I chapter without operationalizing Volume II chapters is a constitutional aspiration. Both are defects, and both SHALL be remediated before the next release certification cycle.

Chapter 12 — Volume II Constitutional Requirements Traceability

12.1 Purpose

Volume II of the Engineering Constitution establishes the operational, mathematical, economic, and governance specifications that bind the Opus Publica platform beyond the foundational Volume I chapters. Volume II comprises eleven chapters (Chapters 26 through 36) that, together, specify the engineering invariants, the system contracts, the canonical object registry, the event constitution, the UI constitution, the engineering mathematics, the production SLO book, the database constitution, the operational governance actions, the engineering economics, and the evolution governance lifecycle. Each of these chapters is, in the language of the Knowledge Graph Constitution (Vol II Ch 23), a source of requirement nodes in the traceability graph; each requirement node SHALL trace through the full thirteen-link chain to runtime evidence.

This chapter traces each of the eleven Volume II chapters at chapter granularity, mapping each chapter to its architecture realization, its Playbook operationalization, its key ADRs, its key APIs and components, its key monitors, its key tests, its evidence artifacts, its owning team, and its current traceability status. The chapter-level matrix in Section 12.2 is the chapter-granularity view; the row-granularity views for the most audit-critical requirement classes — invariants, non-negotiable rules, system contracts, canonical fields, events, SLOs, and thresholds — are the detailed matrices in Chapters 4 through 10. Where this chapter traces a Volume II chapter, the detailed matrices in Chapters 4 through 10 trace that chapter's individual requirements. The two views are complementary: this chapter answers “is the chapter as a whole traced?”; the detailed matrices answer “is each requirement within the chapter traced?”.

The purpose of this chapter, then, is to give the auditor, the Engineering Leadership Team, and the Chief Systems Architect a single page on which the traceability posture of every Volume II chapter can be read at a glance. A Volume II chapter that is marked “Untraced” in this matrix is a chapter whose requirements have not yet been entered into the traceability graph; such a chapter is a critical defect for any chapter containing SEV-1 or SEV-2 requirements. A chapter marked “Gap” is a chapter whose requirements have been entered but whose chain is incomplete in at least one link; the gap SHALL be remediated before the next release. A chapter marked “Traced” is a chapter whose requirements have been entered and whose chain is complete and whose evidence is current.

The chapter differs from Chapter 11 (Volume I Constitutional Requirements Traceability) in three respects. First, Volume II chapters are specification chapters that establish numerical, structural, and procedural requirements (invariants, thresholds, SLOs, schema fields, event types, governance actions), whereas Volume I chapters are charter chapters that establish the platform's mission, principles, and cross-cutting engineering discipline; the traceability posture of a Volume II chapter is therefore more concretely tied to runtime evidence than the posture of a Volume I chapter. Second, Volume II chapters are mutually coupled in ways Volume I chapters are not: the invariants (Ch 26) are bounded by the SLOs

(Ch 32); the contracts (Ch 27) are realized by the events (Ch 29); the canonical fields (Ch 28) are persisted by the database constitution (Ch 33); the governance actions (Ch 34) are amended by the evolution governance (Ch 36). The chapter therefore traces not only each chapter in isolation but also the couplings between chapters. Third, Volume II chapters are the chapters an external compliance auditor is most likely to sample, because they are the chapters that bind the platform's scholarly-publishing-specific behaviour (DOI minting, Crossref deposit, OAI-PMH exposure, preservation, accessibility). The chapter-level matrix is therefore the auditor's primary entry point into the Volume II traceability surface.

12.2 The Volume II Chapter Traceability Matrix

The matrix below traces each of the eleven Volume II chapters through the thirteen-node chain, summarized at chapter granularity. The columns are: Ch # (the Volume II chapter number); Chapter Title; Key Requirements/Invariants (the count and category of requirements the chapter establishes); Architecture (the architecture components that realize the chapter's requirements); Playbook Ch (the Playbook chapter that operationalizes the requirements); ADR(s) (the Architecture Decision Records that justify the architecture decisions); Key APIs/Components (the API surfaces and code components that implement the requirements); Key Monitors (the runtime monitors that verify the requirements hold at runtime); Key Tests (the tests that verify the requirements); Evidence (the runtime evidence artifact IDs); Owner (the team accountable for the chapter's traceability); and Status (Traced / Gap / Untraced). The matrix is read row by row; each row is a chapter's complete chapter-level traceability summary.

Ch #	Chapter Title	Key Requirements / Invariants	Architecture	Playbook Ch	ADR(s)	Key APIs / Components	Key Monitors	Key Tests	Evidence	Owner	Status
26	Engineering Invariants	38 invariants: integrity (8), metadata (4), runtime (4), cryptographic (4), continuity (4), accessibility (4), verification	ARCH-INV: Integrity Service, Metadata Service, Runtime Telemetry, Cryptography, Service, Continuity, Service, Accessi	PB Ch 8 (Runtime Invariant Monitoring)	ADR-002, ADR-014, ADR-021	inv-monitor service; immutable-store; crypto-chain; accessibility-audit-runner.	38 invariant monitors INV-I-01... INV-V-02 (one per invariant); meta-monitor of monitor	INV-T-01... INV-T-38 (one test per invariant); reconciliation tests INV-RECON-01...03.	EVID-INV-001 (daily invariant dashboard screenshots); EVID-INV-002 (reconciliation job logs, 24h).	Platform Engineering	Traced

		tion (4), validati on (2), econo mic (2), version (2). See Ch 4 for row- level trace.	bility Service, Verifica tion Service.				health.				
27	System Contrac ts	20 subsyst em contrac ts: Submis sion, Peer Review, Editoria l Decisio n, Produc tion, Publica tion, DOI Minting , Crossre f Deposit , OAI- PMH, Search, Preserv ation, Authen tication , Authori zation, Notifica tion, Audit, Reporti	ARCH- CONTR ACT: 20 contrac t bounda ries; contrac t- registry service; consum er- driven contrac t test harness .	PB Ch 6 (QA), PB Ch 19 (Master Checkli st Catalog)	ADR-00 1, ADR-00 7	20 subsyst em API surface s; contrac t-test suite; contrac t- registry API.	Contrac t- violatio n monito r; API- confor mance monito r; consum er- compa tibility monito r.	CONTR ACT- T-01... CONTR ACT- T-20 (one per subsyst em); cross- contrac t integra tion tests.	EVID- CONTR ACT-00 1 (CI contrac t test logs per build); EVID- CONTR ACT-00 2 (confor mance dashbo ard, 24h).	Platfor m Enginee ring + subsyst em owners	Traced

		ng, Storage , Workfl ow, Indexin g, Citation Process ing, Export. See Ch 6 for row- level trace.									
28	Canonic al Object Registr y	276 canonic al fields across Article, Issue, Journal, Author, Review er, Editor, Review, Decisio n, Produc tion Task, Deposit , Preserv ation entities . 50 key fields traced in detail in Ch 7.	ARCH- REG: Canoni cal Schema Service; Validati on Service; Registr y- Migrati on Service.	PB Ch 6 (QA)	ADR-00 3, ADR-01 9	schema -service API; field- validat or library; registry - migrati on runner.	Schema - violatio n monito r; field- coverag e monito r; registry -drift monito r.	REG- T-01... REG- T-50 (50 key fields); full- registry schema tests in CI (276 fields).	EVID- REG-00 1 (registr y schema snapsh ot per release); EVID- REG-00 2 (valida tion pipelin e logs, 24h).	Data Enginee ring	Traced (key subset in detail, Ch 7)
29	Event Constit ution	34 event types	ARCH- EVT: Event	PB Ch 6 (QA), PB Ch 8	ADR-00 8, ADR-01	event- bus produc	Event- throug hput	EVT- T-01... EVT-	EVID- EVT-00 1	Platfor m Enginee	Traced

		across Submission, Review, Editorial, Production, Publication, Preservation domains; per-event schema, idempotency, ordering, and replay invariants. See Ch 8 for row-level trace.	Bus, Event Store, Event Handlers, Event Replay Service.	(Invariant Monitoring)	3, ADR-024	er/cons umer SDKs; event-store persistence; replay CLI.	monito r; event-loss monitor; idempotency-violation monitor; event-latency monitor.	T-34 (per-event contract tests); event-replay tests; schema - compa tibility tests.	(event-log sample, real-time); EVID-EVT-002 (replay-test logs per CI run).	ring	
30	UI Constitution	11 UI components (Reader, Author Portal, Reviewer Portal, Editor Console, Admin Console, Search, Discovery, Dashboard,	ARCH-UI: 11 component trees; design-system tokens; SSR + hydration runtime; accessibility-test harness.	PB Ch 6 (QA), PB Ch 10 (Visual Certification Pillar)	ADR-004, ADR-011, ADR-018	11 React component roots; design-token library; Lightho use-CI runner; axe-core runner.	Lightho use-CI monitor (per build); axe-core monitor; visual-regression monitor.	UI-T-01... UI-T-11 (per-component tests); axe accessibility tests; visual-regression snapshot tests.	EVID-UI-001 (Lighthouse report per release); EVID-UI-002 (axe report per release).	Frontend Engineering	Traced

		Profile, Notification, Auth); WCAG 2.2 AA conformance; visual consistency.									
31	Engineering Mathematics	103 numerical thresholds: SLOs, error budgets, queue depths, retry counts, timeouts, batch sizes, rate limits, capacity bounds . Key subset traced in detail in Ch 10.	ARCH-MATH: Threshold Service; Configuration Service; Capacity Planner ; Threshold-Drift Detector.	PB Ch 15 (SLO Management), PB Ch 8 (Invariant Monitoring)	ADR-005, ADR-016	threshold-config service; capacity-planner service; threshold-drift-detector.	Threshold-violation monitor; SLO dashboard; capacity-forecast monitor; threshold-drift monitor.	THR-T-01... THR-T-N (key subset, Ch 10); threshold-drift tests in CI; capacity-forecast tests.	EVID-THR-001 (threshold config snapshot per release); EVID-THR-002 (capacity report, monthly).	SRE + Platform Engineering	Traced (key subset in detail, Ch 10)
32	Production SLO Book	10 SLOs: availability, p95 latency, p99 latency,	ARCH-SLO: SLO Engine, SLI Collectors, Alerting	PB Ch 15 (SLO Management)	ADR-009, ADR-022	slo-engine; slicollector; alert-router; error-budget-	10 SLI dashboards; 4 multi-window multi-burn-rate	SLO-T-01... SLO-T-10 (alert-condition tests);	EVID-SLO-001 (SLO dashboard screenshots, 1h);	SRE	Traced

		freshness, DOI mint latency, Crossref deposit success rate, OAI-PMH availability, search latency, recovery time, throughput. See Ch 9 for row-level trace.	, Error-Budget Tracker.			tracker.	alerts per SLO (40 alert rules total).	synthetic-probe tests; error-budget-rollover tests.	EVID-SLO-002 (alert test logs per CI run).		
33	Database Constitution	14 section: Schema, Migrations, Indexing, Replication, Backup, Point-in-Time Recovery, Read Replicas, Connection Pooling, Transac	ARCH-DB: Primary DB, Read Replicas, PITR Store, Migration Service, Replication Manager.	PB Ch 14 (Operational Governance Execution)	ADR-006, ADR-012, ADR-017	db-migration service; pitr-runner; replication-manager; connection-pool; audit-log writer.	Replication-lag monitor; backup-success monitor; migration-health monitor; PITR-restore monitor.	DB-T-01... DB-T-14 (per-section tests); migration-reversibility tests; restore-drill tests.	EVID-DB-001 (migration log per release); EVID-DB-002 (restore-drill report, quarterly).	Data Engineering + SRE	Traced

		tion Isolation, Audit Trail, Encryption-at- Rest, Access Control , Capacity, Disaster Recovery.									
34	Operational Governance	14 governance actions: Publish, Retract, Correct, Rotate External Credentia l, Rotate TLS, Approve DB Migrati on, Approv e Deploy ment, Approv e Rollbac k, Initiate DR, Declare Incident, Grant Break-	ARCH- OPS- GOV: Action Authorization Service, Audit Log, Break- Glass Vault, Authority Matrix Service.	PB Ch 14 (Operational Governance Execution)	ADR-01 0, ADR-02 3	action- authorizer; audit-log; break-glass- vault; authority- matrix service.	Unautho rized-action- attempt monitor; audit- log-integrit y monitor; break-glass- invocation monitor.	GOV- T-01... GOV- T-14 (per-action authorization tests); break- glass-invocati on test; audit- log-tamper test.	EVID- GOV-00 1 (audit-log sample, real- time); EVID- GOV-00 2 (authori ty-matrix conformance report, quarter ly).	SRE + Engineering Leadership	Traced

		Glass, Modify Audit Log (forbid den), Delete Publish ed Artifact (forbid den), Overrid e Invaria nt (forbid den).									
35	Enginee ring Econo mics	13 cost categor ies: Compu te, Storage , Bandwi dth, Egress, DOI Registr ation, Crossre f Fees, Preserv ation, Email/ Notifica tions, Auth/Id entity, Monito ring/Lo gging, Disaste r Recove ry, Develo	ARCH- ECON: Cost Collect or, Budget Enforce r, Anomal y Detecto r, Per- Unit- Cost Calculat or.	PB Ch 14 (Opera tional Govern ance Executi on), PB Ch 18 (Vol II Operati onaliza tion Matrix)	ADR-01 5, ADR-02 0	cost- collecto r; budget- enforce r; cost- anomal y- detecto r; per- unit- cost- calculat or.	Cost- anomal y monito r; budget- burn monito r; per- unit- cost monito r.	ECON- T-01... ECON- T-13 (per- categor y budget tests); cost- anomal y alert tests; budget- ceiling enforce ment tests.	EVID- ECON-0 01 (month ly cost report); EVID- ECON-0 02 (budget - confor mance report, monthl y).	SRE + Finance Operati ons	Traced

		ment, Operati onal Labor; per- unit- cost bounds ; aggrega te budget ceiling.									
36	Evoluti on Govern ance	10- state amend ment lifecycl e: Propos ed, Drafted , Review ed, Approv ed, Adopte d, Operati onal, Deprec ated, Sunset, Archive d, Rescind ed; RFC 2119 keywor d gating; traceab ility- impact assess ment.	ARCH- EVO: Amend ment Service, Lifecycl e Tracker , Tracea bility- Impact Assesso r, Archive Service.	PB Ch 18 (Vol II Operati onaliza tion Matrix)	ADR-01 4, ADR-02 6	amend ment- service; lifecycl e- tracker; traceab ility- impact- assesso r; archive - service.	Amend ment- stuck- in-state monito r; orphan ed- traceab ility- node monito r (post- amend ment); lifecycl e- regressi on monito r.	EVO- T-01... EVO- T-10 (per- state- transiti on tests); traceab ility- impact- assess ment test.	EVID- EVO-00 1 (amend ment ledger per cycle); EVID- EVO-00 2 (tracea bility- impact report per amend ment).	Chief System s Archite ct + Enginee ring Leaders hip	Traced

The matrix above establishes the chapter-level traceability posture of every Volume II chapter as of the RC1 release. All eleven chapters are marked “Traced”, meaning that each chapter's requirements have been entered into the traceability graph, each chapter's chain is complete at chapter granularity, and each chapter's evidence is current. Three chapters — Chapter 28 (Canonical Object Registry), Chapter 31 (Engineering Mathematics), and the row-level detail for the others — are traced at row granularity in the detailed matrices of Chapters 4 through 10. The chapter-level matrix in this chapter is the roll-up; the detailed matrices are the drill-down. An auditor who requires row-granularity evidence for any Volume II chapter SHALL consult the cross-referenced detailed matrix.

Several chapters warrant specific commentary. Chapter 26 (Engineering Invariants) is the most audit-critical Volume II chapter: its 38 invariants are the platform's runtime constitution, and each invariant has a dedicated monitor (operational or planned) and a dedicated test. The invariant monitors are the platform's first line of defense against scholarly-record corruption; an invariant monitor that fires is, by definition, a SEV-1 or SEV-2 incident, and the invariant's traceability row (in Chapter 4) is the auditor's entry point to the incident's root cause. Chapter 27 (System Contracts) establishes the 20 contract boundaries that prevent subsystem coupling; its traceability chain verifies that every contract has a consumer-driven contract test and that every contract test produces runtime evidence per CI build.

Chapter 28 (Canonical Object Registry) governs the 276 canonical fields that constitute the platform's data vocabulary. The registry is the single source of truth for field names, types, validation rules, and ownership; its traceability chain verifies that the registry schema is snapshot per release, that the validation pipeline enforces the schema at every write path, and that the field-coverage monitor detects schema drift. Chapter 29 (Event Constitution) governs the 34 event types that constitute the platform's integration vocabulary; its traceability chain verifies that every event has a contract test, an idempotency guarantee, and a replay capability. Chapter 30 (UI Constitution) governs the 11 UI components and the accessibility contract (WCAG 2.2 AA); its traceability chain verifies that every component has an axe-core test, a Lighthouse audit, and a visual-regression snapshot.

Chapter 31 (Engineering Mathematics) governs the 103 numerical thresholds that bind the platform's runtime behaviour; its traceability chain verifies that every threshold is configuration-driven, that the threshold-violation monitor fires when a threshold is breached, and that the threshold-drift detector catches configuration regressions. Chapter 32 (Production SLO Book) governs the 10 SLOs that bind the platform's production behaviour; its traceability chain verifies that every SLO has an SLI collector, a multi-window multi-burn-rate alert, and an error-budget policy. Chapter 33 (Database Constitution) governs the 14 database sections; its traceability chain verifies that every migration is reversible, that every backup is restorable, and that the point-in-time recovery window meets the recovery-time objective.

Chapter 34 (Operational Governance) governs the 14 governance actions, including the three forbidden actions (Modify Audit Log, Delete Published Artifact except under legal-removal, Override Invariant); its traceability chain verifies that every action has an authorization check, that every authorized action is logged in the audit log, and that every forbidden action is detected and alerted. Chapter 35 (Engineering Economics) governs the 13 cost categories; its traceability chain verifies that every category has a per-

unit-cost bound, that the budget-burn monitor tracks aggregate spend against the ceiling, and that the cost-anomaly detector catches unexpected spend. Chapter 36 (Evolution Governance) governs the 10-state amendment lifecycle; its traceability chain verifies that every amendment has a traceability-impact assessment, that every state transition is logged in the amendment ledger, and that the orphaned-traceability-node monitor detects requirements whose traces were not updated after an amendment.

The chapter-level matrix above is the roll-up view; the detailed matrices in Chapters 4 through 10 are the drill-down view. The relationship between the two views is governed by the following convention: each row in the chapter-level matrix summarizes a chapter whose individual requirements are traced in the cross-referenced detailed matrix. A chapter whose row in this matrix is marked “Traced” is a chapter whose individual requirements are all traced (or traced through a parent, per Chapter 15.2) in the detailed matrix. A chapter whose row is marked “Gap” is a chapter with at least one untraced or gapped requirement in the detailed matrix; the gap is identified at row granularity in the detailed matrix. A chapter whose row is marked “Untraced” is a chapter whose requirements have not been entered into the detailed matrix at all; this is a critical defect for any chapter above SEV-3, and the Release Certification Authority SHALL NOT certify a release with an “Untraced” Volume II chapter.

The chapter-level matrix is the view an auditor consults first, because it gives the platform's Volume II traceability posture at a glance. The detailed matrices are the view an auditor consults second, after identifying the specific chapter or requirement class to investigate. The two views are designed to be read together: the chapter-level matrix's Status column is the roll-up of the detailed matrix's Status column. If the two disagree, the disagreement is a defect in the Matrix's maintenance (the chapter-level row was not updated after a gap was identified at row granularity in the detailed matrix). The Constitution-vs-Matrix reconciliation (Chapter 13.6) and the Matrix-vs-implementation reconciliation SHALL catch such disagreements; the weekly engineering review SHALL verify the two views agree.

Several Volume II chapters have cross-cutting traceability implications that warrant explicit mention. Chapter 26 (Engineering Invariants) and Chapter 32 (Production SLO Book) are tightly coupled: every invariant has an SLO that bounds its runtime behaviour, and every SLO has an invariant that defines its failure mode. An auditor tracing an invariant in Chapter 4 SHALL verify the corresponding SLO in Chapter 9, and vice versa. Chapter 27 (System Contracts) and Chapter 29 (Event Constitution) are similarly coupled: every contract boundary emits and consumes events, and every event crosses a contract boundary. An auditor tracing a contract in Chapter 6 SHALL verify the events the contract emits and consumes in Chapter 8, and vice versa. Chapter 33 (Database Constitution) and Chapter 28 (Canonical Object Registry) are coupled through the schema: every canonical field is backed by a database column, and every database column realises a canonical field. The cross-cutting coupling is captured in the traceability graph by the edge types ‘backed-by’ (Architecture → Database) and ‘realizes’ (Requirement → Architecture); the auditor traverses these edges to verify the coupling.

Chapter 34 (Operational Governance) and Chapter 36 (Evolution Governance) are coupled through the amendment lifecycle: every amendment to a governance action (e.g., adding a new authorized action, changing the authority matrix) is itself a governance action and SHALL be authorized per Chapter 34's

authority matrix. This recursive coupling is intentional: governance of governance is itself governed, and the traceability graph captures the recursion by linking the amendment record (Chapter 36) to the authorization record (Chapter 34). An auditor tracing an amendment SHALL verify the authorization that permitted the amendment, and an auditor tracing an authorization SHALL verify the amendment that established the authorized action. The recursion terminates at the Constitution's first issuance, which is the unmoved mover of the governance regime.

The traceability posture documented in this chapter is the posture as of the RC1 release. The posture will change as the platform evolves: new requirements will be added by amendment (Vol II Ch 36) and SHALL be traced before the next release (Integrity Rule IR-08); existing requirements will be deprecated and their traces archived; gaps will be remediated and new gaps will be discovered. The Matrix is a living document, and this chapter is a living chapter. The Engineering Leadership Team SHALL review this chapter at every quarterly integrity check and SHALL update the Status column to reflect the current posture. The CSA SHALL verify the update at every RC milestone.

Volume II Traceability Posture (RC1)

All eleven Volume II chapters (Chapters 26 through 36) are traced at chapter granularity as of the RC1 release. The most audit-critical requirement classes — the 38 invariants (Ch 4), the 47 non-negotiable rules (Ch 5), the 20 system contracts (Ch 6), the 50 key canonical fields (Ch 7), the 34 events (Ch 8), the 10 SLOs (Ch 9), and the key thresholds (Ch 10) — are traced at row granularity in their respective detailed matrices. The Release Certification Authority SHALL NOT certify RC1 release until every chapter in this matrix is marked “Traced” AND every detailed matrix in Chapters 4-10 has zero “Untraced” rows above SEV-3.

Chapter 13 — Traceability Graph Integrity

13.1 Purpose

A traceability graph that is not verified is a graph that cannot be trusted. This chapter specifies the integrity rules, the orphan-node detection regime, the stale-evidence detection regime, the maximum evidence age schedule, and the graph reconciliation cadence that, together, ensure the traceability graph remains a trustworthy representation of the platform's actual compliance posture. The integrity rules are the operational form of Volume II, Chapter 23, section 23.7 (Graph Integrity Rules), restated and expanded here with verification procedures, detection mechanisms, remediation steps, and owners for each rule.

The chapter is structured as follows. Section 13.2 restates the eight integrity rules (IR-01 through IR-08) introduced in Chapter 2 §2.5 and, for each rule, specifies the verification procedure, the detection mechanism, the remediation, and the owner. Section 13.3 specifies the orphan-node detection regime for all thirteen node types. Section 13.4 specifies the stale-evidence detection regime, including the maximum evidence age per evidence type, the refresh procedure, and the escalation. Section 13.5 consolidates the maximum evidence age schedule. Section 13.6 specifies the graph reconciliation cadence, including the five reconciliation types and their on-failure actions. Together, these sections define the operational regime that keeps the traceability graph honest.

13.2 Integrity Rules

The traceability graph SHALL satisfy the following eight integrity rules. Each rule is restated from Chapter 2 §2.5 and expanded here with its verification procedure, detection mechanism, remediation, and owner. The rules are normative: a graph that violates any rule is a defective graph, and the defect SHALL be remediated within the SLA specified for the rule's severity class.

Rule	Verification Procedure	Detection Mechanism	Remediation	Owner
IR-01 — Every requirement SHALL trace to at least one architecture component (no orphan requirements).	For each REQ-* node, query the graph for at least one outgoing 'realizes' edge to an ARCH-* node. The query SHALL return non-empty for every requirement.	Nightly automated graph scan: SELECT req FROM requirements WHERE req.id NOT IN (SELECT src FROM edges WHERE edge_type='realizes'). Any row returned is an orphan.	Create a remediation task citing the orphan requirement ID. The owning team SHALL either add the missing architecture link or mark the requirement obsolete. SLA: 5 business days for SEV-1/2; 30 days for SEV-3.	Engineering Leadership
IR-02 — Every architecture component SHALL	For each ARCH-* node, query the graph for at least one	Nightly automated graph scan plus ADR-directory	Open an ADR-drafting task. The Principal Architect SHALL either	Principal Architect

trace to at least one ADR (no undocumented design decisions).	outgoing 'justified-by' edge to an ADR-* node.	reconciliation: every ARCH-* node MUST appear in the ADR registry with a linked ADR-* id.	author the missing ADR within 10 business days or document the component as deprecated. SLA: 10 business days.	
IR-03 — Every API endpoint SHALL trace to at least one test (no unverified contracts).	For each API-* node, query the graph for at least one incoming 'verified-by' edge from a TEST-* node.	CI gate: the contract-test suite SHALL execute every API-* endpoint at least once per build. The coverage report SHALL flag any API-* with zero test coverage.	Block the build. The API owner SHALL add a contract test for the uncovered endpoint before the next release. SLA: blocks release.	API Owner (per subsystem)
IR-04 — Every test SHALL trace to at least one runtime evidence artifact in the most recent release (no tests that did not run).	For each TEST-* node, query the graph for at least one outgoing 'produces' edge to an EVID-* node whose release field matches the most recent REL-* node.	Post-release evidence reconciliation: every TEST-* that ran in CI SHALL produce an EVID-* artifact. Missing EVID-* is a defect.	Re-run the test in CI; if the test passes, regenerate the EVID-* artifact. If the test fails, the failure is a separate defect. SLA: before next release.	Release Certification Authority
IR-05 — Every SEV-1 requirement SHALL have a complete traceability chain with no gaps.	For each REQ-* node with severity=SEV-1, traverse the full 13-node chain. Every applicable link SHALL be populated (not '—', not 'GAP', not 'STALE').	Pre-release certification gate: the RCA SHALL run the SEV-1 chain-completeness check before sign-off. Any SEV-1 requirement with a gap blocks release.	Block release. The owning team SHALL remediate the gap. The CSA SHALL NOT override a SEV-1 gap. SLA: blocks release until remediated.	Release Certification Authority + Chief Systems Architect
IR-06 — Every runtime evidence artifact SHALL be no older than the maximum evidence age (Chapter 13.5).	For each EVID-* node, compute age = now - evidence.timestamp. Verify age ≤ max_age(evidence_type).	Weekly stale-evidence scan: every EVID-* node is checked against the maximum evidence age schedule. Stale evidence is flagged 'STALE' in the Matrix.	Refresh the evidence: re-run the test, re-snapshot the dashboard, re-collect the monitor output. SLA: 1 business day for invariant-monitor evidence; 5 business days for other classes.	Evidence Owner (per evidence class)
IR-07 — No node SHALL be an orphan (no incoming or outgoing edges).	For every node, query the graph for at least one incoming OR outgoing edge of any type.	Nightly orphan-node scan across all 13 node types (see Section 13.3 for per-type detection).	Either connect the orphan to its expected neighbours (preferred) or remove the orphan from the graph (if it is	Graph Steward (Engineering Leadership)

			genuinely obsolete). SLA: 5 business days.	
IR-08 — The traceability graph SHALL be reconciled against the Constitution on every Constitution amendment.	On every amendment adoption (Vol II Ch 36), run the Constitution-vs-Matrix reconciliation (Section 13.6). Requirements added by the amendment SHALL be traced; requirements removed SHALL have their traces archived.	Amendment-triggered reconciliation job: diff the Constitution requirement list against the Matrix requirement list. Added/removed requirements are flagged.	Add rows for new requirements within 5 business days; archive rows for removed requirements within 5 business days. SLA: 5 business days post-amendment.	Chief Systems Architect

The eight integrity rules are not aspirational; they are operational. Each rule has a defined verification procedure (the query or gate that checks the rule), a defined detection mechanism (the automated scan or gate that catches violations), a defined remediation (the action that fixes the violation and the SLA within which it SHALL be fixed), and a defined owner (the role accountable for the rule's enforcement). The Release Certification Authority SHALL verify that all eight rules hold before signing off on a release. The Chief Systems Architect SHALL verify that all eight rules hold before signing off on an RC milestone. An external auditor SHALL sample the rules' verification logs during any external audit.

13.3 Orphan Node Detection

An orphan node is a node in the traceability graph that has no incoming or outgoing edges of any type. An orphan is a defect: it represents an artifact that exists in the graph but is not connected to any other artifact, which means the artifact is either obsolete (and SHOULD be removed) or was never linked (and SHOULD be linked). The table below specifies, for each of the thirteen node types, how orphans are detected, the remediation, and the cadence on which the detection runs.

Node Type	How Orphans Are Detected	Remediation	Cadence
N1 — Requirement	Nightly graph scan: REQ-* nodes with zero outgoing 'realizes' edges. Cross-check against the Constitution requirement list; a REQ-* not present in the Constitution is also an orphan.	Add the missing 'realizes' edge; or, if the requirement was removed by amendment, archive the REQ-* node.	Nightly
N2 — Architecture Component	Nightly graph scan: ARCH-* nodes with zero incoming 'realizes' edges AND zero outgoing 'justified-by' edges.	Either link the component to a requirement and an ADR, or mark the component deprecated and remove it from the active architecture registry.	Nightly

N3 — ADR	Weekly ADR-registry reconciliation: ADR-* records that exist in the ADR directory but have no incoming 'justified-by' edges AND no outgoing 'backed-by'/'exposed-by'/'implemented-by' edges.	Either link the ADR to the component(s) it justifies, or mark the ADR 'Superseded' per ADR lifecycle.	Weekly
N4 — Database Table	Nightly schema-vs-graph reconciliation: tables present in the production schema but absent from the graph, and vice versa.	Add the table to the graph (with its owning service and schema); or, if the table was dropped, remove its DB-* node.	Nightly
N5 — API Endpoint	CI gate + nightly scan: API-* endpoints registered in the API gateway but with zero incoming 'verified-by' edges from tests.	Add a contract test for the endpoint (blocks release until added).	Per CI build (gate) + nightly
N6 — Code Component	Nightly repository-vs-graph reconciliation: CODE-* modules present in the repository but with zero incoming 'implemented-by' edges; CODE-* in the graph but absent from the repo.	Link the module to its architecture component; or, if the module was deleted, archive its CODE-* node.	Nightly
N7 — Storage Object	Weekly storage-vs-graph reconciliation: buckets/keys present in the storage provider but absent from the graph; STORE-* in the graph but absent from the storage provider.	Add the storage object to the graph (with retention class); or, if the object was deleted, archive its STORE-* node.	Weekly
N8 — Workflow	Weekly workflow-registry reconciliation: WF-* workflows registered in the workflow engine but with zero outgoing 'exercised-by' edges.	Link the workflow to the code component(s) it exercises; or decommission the workflow.	Weekly
N9 — UI Component	Weekly component-catalogue reconciliation: UI-* components in the design-system catalogue but with zero incoming 'surfaced-by' edges; UI-* in the graph but absent from the catalogue.	Link the component to its surfacing workflow; or remove the component from the catalogue.	Weekly

N10 — Test	CI gate + nightly scan: TEST-* records that exist in the test registry but have zero outgoing 'produces' edges to EVID-* nodes in the most recent release.	Re-run the test in CI to regenerate its EVID-* artifact; or, if the test is obsolete, archive it.	Per CI build + nightly
N11 — Runtime Evidence	Weekly stale-evidence scan (IR-06): EVID-* nodes whose age exceeds the maximum evidence age are flagged 'STALE' and treated as missing.	Refresh the evidence (re-run the test, re-snapshot the dashboard, re-collect the monitor output).	Weekly
N12 — Certification Record	Per-release reconciliation: CERT-* records that exist but have zero incoming 'certified-by' edges from EVID-* nodes, or zero outgoing 'released-in' edges to REL-* nodes.	Either bundle the evidence into the certification record, or, if the certification was revoked, mark the CERT-* node 'Revoked' and re-certify.	Per release
N13 — Release Record	Per-release reconciliation: REL-* records that exist but have zero incoming 'released-in' edges from CERT-* nodes (i.e., a release without certification).	Block the release. The RCA SHALL NOT sign off on a release without a CERT-* record.	Per release (gate)

The orphan-node detection regime is comprehensive: every one of the thirteen node types has a defined detection mechanism, a defined remediation, and a defined cadence. The cadences are tiered: nightly for the most volatile node types (requirements, architecture, database, API, code); weekly for the moderately volatile types (ADR, storage, workflow, UI, runtime evidence); per-release for the least volatile types (certification, release). The cadence reflects the rate at which each node type changes in practice and the cost of an undetected orphan of each type. The Engineering Leadership Team SHALL review the orphan-detection dashboard at every weekly engineering review.

13.4 Stale Evidence Detection

Evidence that is too old is, for audit purposes, missing evidence. The traceability graph treats stale evidence as a defect equivalent to a missing link: the Matrix cell is marked 'STALE', the Status of the affected requirement is downgraded to 'Gap', and the evidence SHALL be refreshed within the SLA specified for its evidence class. The table below specifies, for each evidence type, the maximum age, the detection mechanism, the refresh procedure, and the escalation if the refresh SLA is missed.

Evidence Type	Max Age	Detection	Refresh Procedure	Escalation
---------------	---------	-----------	-------------------	------------

Invariant monitor output (INV-I-01...INV-V-02)	1 hour	Invariant-monitor freshness probe: each monitor SHALL emit a heartbeat; absence of heartbeat for >1 hour triggers a SEV-2 alert.	On-call engineer acknowledges the alert; investigates the monitor's failure mode; restores the monitor; regenerates the evidence artifact.	If not refreshed within 4 hours: SEV-1 escalation to Engineering Leadership; invariant treated as violated.
Reconciliation report (Crossref, OAI-PMH, DOI, ORCID, preservation)	24 hours	Reconciliation-job completion check: each reconciliation job SHALL complete within 24 hours of its scheduled run; non-completion is flagged.	On-call engineer investigates the failed job; re-runs the job; produces a fresh reconciliation report; stores it under a new EVID-* ID.	If not refreshed within 48 hours: SEV-2 escalation; the affected requirement is marked 'Gap' in the Matrix.
Certification artifact (CERT-*)	Per release cycle (release-to-release)	Certification-record age check: a CERT-* record older than the previous release is stale.	Re-certify the feature in the next release cycle; produce a new CERT-* record; archive the old one.	If a release ships without a fresh CERT-* record: the release is blocked by the RCA.
Audit log sample (operational governance actions)	Real-time (continuous)	Audit-log integrity monitor: the audit log SHALL be append-only and tamper-evident; any modification or deletion is detected immediately.	On-call engineer investigates the integrity alert; restores the audit log from backup; identifies the unauthorized modification; files an incident.	If the audit log is tampered: SEV-1 incident; CSA notified immediately; forensic review.
Dashboard screenshot (SLO, invariant, capacity dashboards)	1 hour	Dashboard-screenshot freshness probe: the screenshot service SHALL capture a screenshot hourly; absence triggers a SEV-3 alert.	On-call engineer verifies the dashboard is reachable; re-runs the screenshot service; stores the new screenshot under a new EVID-* ID.	If not refreshed within 4 hours: SEV-2 escalation; the affected SLO/invariant is marked 'STALE' in the Matrix.
Test execution log (per CI run)	Per CI run (build-to-build)	CI evidence collector: every CI run SHALL produce an EVID-* artifact per test that ran; missing artifacts are flagged.	Re-run the CI build; verify the EVID-* artifacts are produced; investigate any test that did not produce evidence.	If a release ships without per-build EVID-* artifacts for its tests: the release is blocked by the RCA.
Deposit receipt (Crossref, preservation)	Real-time (per deposit)	Deposit-receipt collector: every Crossref/preservation deposit SHALL return a	On-call engineer re-deposits the affected item; verifies the receipt is returned;	If a deposit receipt is missing for >24 hours: SEV-2 escalation; the affected publication's

		receipt; missing receipts are flagged within 1 hour.	stores the receipt under a new EVID-* ID.	traceability is marked 'Gap' until the receipt is produced.
--	--	--	---	---

The stale-evidence detection regime is designed to catch evidence currency failures before an auditor encounters them. An auditor who encounters stale evidence in the Matrix is encountering a defect that the stale-evidence detection regime was supposed to catch and remediate before the audit. The detection regime is therefore the platform's first line of defense against the credibility loss that stale evidence inflicts on the Matrix. The escalation column specifies the consequence of a missed refresh SLA: the affected requirement's Status is downgraded, and the escalation path triggers. The Engineering Leadership Team SHALL review the stale-evidence dashboard at every weekly engineering review.

13.5 The Maximum Evidence Age Schedule

The maximum evidence age schedule consolidates the maximum-acceptable age for each evidence class. The schedule is the operational form of Integrity Rule IR-06; it is the table that the stale-evidence detection regime in Section 13.4 checks against. The schedule reflects the rate at which each evidence class changes in practice: invariant monitor outputs change continuously (the monitor emits a heartbeat every minute); reconciliation reports change daily; certification artifacts change per release; audit log samples change continuously; dashboard screenshots change hourly; test execution logs change per CI build; deposit receipts change per deposit. The maximum age for each class is set to the shortest interval at which a meaningful change could occur without the evidence being regenerated.

Evidence Class	Max Age	Rationale
Invariant monitor output	1 hour	Invariant monitors emit a heartbeat every minute; an evidence artifact older than 1 hour indicates the monitor has been silent for at least 60 heartbeat cycles, which is a strong signal of monitor failure.
Reconciliation report (Crossref, OAI-PMH, DOI, ORCID, preservation)	24 hours	Reconciliation jobs run on a daily cadence; an evidence artifact older than 24 hours indicates the previous day's job did not complete, which is a defect.
Certification artifact (CERT-*)	Per release cycle	Certification is bound to releases; a CERT-* record older than the previous release is stale because the implementation it certified has been superseded by a new release.
Audit log sample	Real-time (continuous)	The audit log is the platform's tamper-evident record of operational actions; any delay in audit-log evidence is a

		tamper-evidence failure, which is a SEV-1 incident.
Dashboard screenshot (SLO, invariant, capacity)	1 hour	Dashboards update continuously; a screenshot older than 1 hour may not reflect the current state of the platform, which makes it useless as audit evidence.
Test execution log	Per CI run (build-to-build)	Tests produce evidence on every CI run; a test execution log older than the most recent CI build indicates the test did not run in the most recent build, which is a defect.
Deposit receipt (Crossref, preservation)	Real-time (per deposit)	Deposit receipts are produced synchronously with each deposit; a missing receipt indicates the deposit failed, which is a scholarly-record corruption risk.

The schedule is normative: an evidence artifact whose age exceeds the maximum for its class is stale, and the affected requirement's Status SHALL be downgraded. The schedule is also conservative: the maximum ages are set to the shortest interval that avoids false-positive stale-evidence alerts. If an evidence class is found to produce excessive false-positive stale-evidence alerts, the maximum age MAY be relaxed by a Constitution amendment (Vol II Ch 36), subject to the amendment lifecycle and the traceability-impact assessment. The CSA SHALL approve any relaxation of the schedule.

13.6 Graph Reconciliation Cadence

The traceability graph is reconciled against five sources of truth on the cadences specified below. Reconciliation is the process by which the graph is verified to match a source of truth; a reconciliation failure is a defect that SHALL be remediated within the SLA specified for the failure's severity class. The five reconciliation types are: Constitution-vs-Matrix (the Matrix's requirement list matches the Constitution's requirement list); Matrix-vs-implementation (the Matrix's architecture, API, code, and storage columns match the actual implementation); Matrix-vs-evidence (the Matrix's evidence column references evidence that exists and is current); Matrix-vs-Crossref (the Matrix's Crossref-deposit evidence matches Crossref's actual record); and Matrix-vs-preservation (the Matrix's preservation-receipt evidence matches the preservation service's actual record).

Reconciliation Type	What Is Reconciled	Cadence	Owner	On-Failure Action
Constitution-vs-Matrix	The Matrix's requirement list is diffed against the Constitution's requirement list	On every Constitution amendment; quarterly integrity check.	Chief Systems Architect	Open remediation tasks for added/removed requirements. SLA: 5 business days.

	(compiled from Vol I Ch 3 + Vol I Ch 4-25 + Vol II Ch 26-36). Added requirements SHALL be added to the Matrix; removed requirements SHALL have their traces archived.			Unremediated additions block the next release.
Matrix-vs-implementation	The Matrix's architecture, ADR, database, API, code, storage, workflow, and UI columns are reconciled against the actual implementation (repo, schema, ADR directory, API gateway, storage provider, workflow engine, design-system catalogue).	Quarterly; nightly for the API and code columns (via CI gates).	Engineering Leadership	Open remediation tasks for discrepancies. Drift is a defect. SLA: 30 days for non-SEV-1; blocks release for SEV-1.
Matrix-vs-evidence	The Matrix's evidence column is reconciled against the evidence repository. Every EVID-* referenced in the Matrix SHALL exist in the repository and SHALL be within its maximum age (Section 13.5).	Weekly stale-evidence scan; per-release evidence reconciliation.	Release Certification Authority	Refresh stale evidence; restore missing evidence. SLA: per Section 13.4 escalation.
Matrix-vs-Crossref	The Matrix's Crossref-deposit evidence is reconciled against the Crossref REST API. Every DOI the Matrix claims is deposited SHALL appear in Crossref's API with matching metadata.	Daily reconciliation job; weekly full reconciliation.	SRE + Editorial Production	Re-deposit missing or mismatched DOIs; file SEV-2 incident for any DOI missing from Crossref for >24 hours.
Matrix-vs-preservation	The Matrix's preservation-receipt evidence is reconciled against the preservation service	Monthly reconciliation job; quarterly full reconciliation.	SRE + Editorial Production	Re-deposit missing artifacts; file SEV-2 incident for any artifact missing from preservation for >7

	(CLOCKSS / LOCKSS / Portico). Every artifact the Matrix claims is preserved SHALL appear in the preservation service's record.			days.
--	--	--	--	-------

The reconciliation cadence is the operational form of Integrity Rule IR-08, extended beyond the Constitution-vs-Matrix reconciliation that IR-08 explicitly mandates to include four additional reconciliation types that bind the Matrix to the external sources of truth that matter most for scholarly publishing: the implementation, the evidence repository, Crossref, and the preservation service. The Crossref and preservation reconciliations are particularly audit-critical: a scholarly publishing platform whose Matrix claims a DOI is deposited but whose Crossref record shows no such deposit has a traceability defect that is also a scholarly-record corruption defect, and the two defects SHALL be remediated together. The Engineering Leadership Team SHALL review the reconciliation dashboard at every weekly engineering review, and the CSA SHALL review the dashboard at every quarterly integrity check.

Chapter 14 — The Audit Procedure

14.1 Purpose

This chapter specifies how an auditor — internal or external, Engineering Leadership or Release Certification Authority or third-party compliance auditor — uses the Traceability Matrix to verify the platform's compliance posture. The Matrix is the auditor's primary instrument; this chapter is the auditor's procedure manual. The chapter specifies the audit scope and objectives, the audit types, the standard audit procedure (a twelve-step sequence), the audit sampling strategy, the audit report template, the audit finding severity classification, and the audit evidence retention policy.

The audit procedure is designed to be executable by an auditor who is not an Opus Publica engineer. The procedure assumes the auditor is competent in software-engineering audit practices and is familiar with scholarly-publishing compliance regimes (Crossref, DOAJ, Scopus, Web of Science, CLOCKSS, LOCKSS, Portico, OAI-PMH, ORCID), but it does not assume the auditor has prior knowledge of the Opus Publica codebase, architecture, or operational history. The Matrix is designed to be self-explanatory: an auditor who reads Chapter 2 (the framework) and Chapter 3 (how to use the Matrix) SHALL be able to execute the procedure in this chapter without further onboarding.

14.2 Audit Scope and Objectives

An audit of the Opus Publica platform verifies four things. First, constitutional compliance: the platform's implementation matches the Constitution's specification. The audit verifies that every requirement the Constitution establishes is realized by an architecture component, implemented by code, exposed by an API (where applicable), and verified by tests. Second, traceability chain integrity: the thirteen-node chain from Requirement to Release is complete for every sampled requirement, with no gaps and no orphan nodes. Third, evidence currency: the runtime evidence referenced by the Matrix is within its maximum age (Chapter 13.5) and demonstrates the requirement's satisfaction. Fourth, gap remediation: gaps identified in prior audits have been remediated within their SLAs, and the remediation has been verified.

The audit does not verify the Constitution itself (the Constitution is the auditor's premise, not the auditor's subject), nor does it verify the platform's product strategy, editorial decisions, or commercial policy (these are outside the Matrix's scope; see Chapter 15.9). The audit verifies the engineering execution of the Constitution's requirements. An audit finding is therefore a finding about the engineering execution, not about the Constitution or the product strategy. Findings are classified by severity (Section 14.7) and remediated under the SLAs specified for each severity class.

14.3 Audit Types

Five audit types are recognized. Each type has a defined scope, cadence, auditor, and output. The types are not mutually exclusive: a single audit may be of multiple types (for example, an RC milestone audit may also be a funder-mandated audit if the funder's compliance window aligns with the RC milestone).

Audit Type	Scope	Cadence	Auditor	Output
Internal quarterly audit	Sample of requirements across all nine requirement classes; integrity-rule verification; reconciliation-cadence verification; stale-evidence spot check.	Quarterly	Engineering Leadership Team (internal auditor designated by the CSA).	Internal audit report filed in the certification repository; findings tracked to closure in the engineering issue tracker.
RC milestone audit	Full traceability chain verification for every SEV-1 requirement; sample of SEV-2/3; complete integrity-rule verification; complete reconciliation verification.	On every RC milestone transition (RC1, RC2, RC3, GA).	Chief Systems Architect + Release Certification Authority.	RC milestone sign-off (or block); report filed in the certification repository; findings block the milestone transition until remediated.
External compliance audit	Sample of requirements weighted toward funder-mandated and indexer-eligibility requirements (DOI minting, Crossref deposit, OAI-PMH, preservation, ORCID attribution, accessibility).	Annual; or on funder mandate.	External auditor (third-party compliance firm or funder-designated auditor).	External audit report; findings communicated to the funder and to Engineering Leadership; remediation tracked in the engineering issue tracker.
Funder-mandated audit	Scope defined by the funder's compliance regime (e.g., Plan S compliance, COPE compliance, DOAJ compliance).	On funder mandate; typically annual or per-grant.	Funder-designated auditor.	Funder compliance report; findings communicated to the funder; remediation tracked in the engineering issue tracker.
Incident-triggered audit	Scope defined by the incident: the affected requirement class and the affected traceability chain are audited in full.	On every SEV-1 incident; on SEV-2 incidents at the CSA's discretion.	Chief Systems Architect + Engineering Leadership.	Incident audit report appended to the incident postmortem; findings tracked as incident action items.

14.4 The Standard Audit Procedure

The standard audit procedure is the twelve-step sequence that an auditor SHALL follow for any audit type. The procedure is designed to be repeatable: two auditors executing the procedure on the same requirement sample SHALL produce substantially the same findings. The procedure is also designed to be efficient: an experienced auditor SHALL complete the procedure for a standard quarterly audit in approximately one engineer-week of effort.

Step 1. Scope the audit. Identify the requirement classes to be audited (e.g., invariants, non-negotiable rules, system contracts, events, SLOs, thresholds, Volume I chapters, Volume II chapters). Document the scope in the audit report. For an RC milestone audit, the scope is all SEV-1 requirements plus a sample of SEV-2/3. For an external compliance audit, the scope is the funder's compliance regime.

Step 2. Sample requirements from each class. Apply the audit sampling strategy (Section 14.5) to select requirements for detailed traversal. Document the sample in the audit report, including the sampling method and the random seed (where applicable) so that the sample is reproducible.

Step 3. For each sampled requirement, traverse the traceability chain. Open the Matrix at the row for the sampled requirement. Read the row left to right, following the chain from the Requirement through Architecture, ADR, Database, API, Code, Storage, Workflow, UI, Tests, Evidence, Certification, Release. Record each node's ID and the link type traversed.

Step 4. Verify each link in the chain exists and is populated. For each cell in the row, verify that the cell is populated (not '—' where the link is applicable, not 'GAP', not 'STALE'). A single gap or stale evidence flag breaks the chain. Record the break.

Step 5. Retrieve and inspect the runtime evidence. For each EVID-* referenced in the Evidence column, retrieve the evidence artifact from the certification repository. Inspect the artifact to confirm it exists, is accessible, and contains the expected content (e.g., the dashboard screenshot shows the expected metrics, the test execution log shows the test passed, the deposit receipt shows the DOI was deposited).

Step 6. Verify the evidence is within the maximum age. Compute $\text{age} = \text{audit_date} - \text{evidence.timestamp}$. Verify $\text{age} \leq \text{max_age}(\text{evidence_type})$ per the schedule in Chapter 13.5. Evidence older than the maximum age is stale; record the staleness.

Step 7. Verify the evidence demonstrates the requirement's satisfaction. This is the auditor's substantive judgment. The auditor verifies that the evidence — read on its own — would persuade a reasonable engineer that the requirement is satisfied. A test that passes but does not exercise the requirement's failure mode is not sufficient evidence; a dashboard that shows green but does not surface the requirement's metric is not sufficient evidence. The auditor records the substantive judgment.

Step 8. Document findings (Traced / Gap / Stale / Untraced). For each sampled requirement, record the finding: 'Traced' (the chain is complete and the evidence is current and persuasive); 'Gap' (one or more links are missing); 'Stale' (the chain is complete but the evidence is stale); 'Untraced' (the requirement is not in the Matrix).

Step 9. Compile the audit report. Using the audit report template (Section 14.6 and Appendix E), compile the findings into a structured report. The report SHALL include the executive summary, the scope, the methodology, the findings by class, the gap analysis, and the remediation recommendations.

Step 10. Submit to Engineering Leadership and the Chief Systems Architect. The audit report SHALL be submitted to Engineering Leadership and the CSA within five business days of the audit's completion. The CSA SHALL acknowledge receipt within two business days. The report is filed in the certification repository under a new AUDIT-* ID.

Step 11. Track findings to closure. Each finding with a remediation SLA SHALL be tracked in the engineering issue tracker. The auditor (or the auditor's designate) SHALL verify closure at the next audit cycle. A finding that is not remediated within its SLA is escalated per Section 14.7.

Step 12. Publish the audit report to the certification repository. The final audit report, including the findings, the remediation tracking, and the closure verification, SHALL be published to the certification repository. The report is retained per the audit evidence retention policy (Section 14.8).

14.5 Audit Sampling Strategy

The audit sampling strategy specifies how requirements are selected from each requirement class for detailed traversal. The strategy is risk-based: SEV-1 requirements are sampled at 100% (every SEV-1 requirement is traversed in every audit), SEV-2 requirements are sampled at a higher rate than SEV-3, and the sample is stratified by requirement class to ensure coverage of the full constitutional surface. The sampling method is documented in the audit report so that the sample is reproducible.

Requirement Class	Sample Size	Sampling Method
SEV-1 requirements (any class)	100% (census)	All SEV-1 requirements are traversed in every audit. No sampling; the auditor SHALL traverse the full chain for every SEV-1 requirement.
SEV-2 requirements	30% or 20 requirements, whichever is greater	Stratified random sampling: within each requirement class, select 30% of SEV-2 requirements (or 20, whichever is greater) using a documented random seed.
SEV-3 requirements	10% or 10 requirements, whichever is greater	Stratified random sampling: within each requirement class, select 10% of SEV-3 requirements (or 10, whichever is greater) using a documented random seed.
Engineering invariants (INV-*)	100% (census) for SEV-1 invariants; 30% for SEV-2	All SEV-1 invariants are traversed; 30% of SEV-2 invariants are randomly sampled. The invariant monitor outputs are spot-checked for every invariant (regardless of severity).

System contracts (CONTRACT-*)	100% (census)	All 20 system contracts are traversed in every audit (the count is small enough that a census is feasible).
Canonical fields (REG-*)	20 key fields (40% of the 50 traced in detail)	Stratified random sampling across the 50 key fields traced in Chapter 7, stratified by entity (Article, Issue, Journal, Author, Review, Editor, Deposit, Preservation).
Events (EVT-*)	10 events (30% of the 34 events)	Stratified random sampling across the 6 event domains (Submission, Review, Editorial, Production, Publication, Preservation).
SLOs (SLO-*)	100% (census)	All 10 SLOs are traversed in every audit (the count is small enough that a census is feasible).
Thresholds (THR-*)	15 thresholds (key subset)	Risk-based sampling: thresholds with the highest operational impact (queue depths, retry counts, timeouts, rate limits) are always sampled; others are sampled randomly.
Volume I chapters (Ch 1-25)	5 chapters (20%)	Risk-based sampling: chapters with the highest requirement density (Ch 3 Non-Negotiable Rules, Ch 4 Architecture, Ch 8 Runtime Certification, Ch 9 API Constitution, Ch 19 Testing) are always sampled; others are sampled randomly.
Volume II chapters (Ch 26-36)	3 chapters (27%)	Risk-based sampling: Ch 26 (Invariants), Ch 32 (SLOs), and Ch 34 (Operational Governance) are always sampled; one additional chapter is sampled randomly per audit cycle.

The sampling strategy is conservative: it errs on the side of over-sampling. A standard quarterly audit samples approximately 100-150 requirements (across all classes), which is approximately 20-30% of the Matrix's total requirement count. An RC milestone audit samples approximately 200-250 requirements (40-50% of the Matrix). An external compliance audit samples per the funder's regime, but SHALL not sample less than the quarterly audit's sample size. The auditor SHALL document any deviation from this strategy in the audit report.

14.6 Audit Report Template

Every audit report SHALL follow the standard template below. The template is the operational form of the audit report structure specified in Appendix E; this section describes the template's sections, and Appendix E provides the fill-in template.

The audit report has seven sections. The Executive Summary is a one-page summary of the audit's findings, suitable for reading by the CSA and Engineering Leadership in under five minutes. The Scope documents the requirement classes audited, the sample size, and the sampling method. The Methodology documents the audit procedure followed (Chapter 14.4) and any deviations. The Findings by Class presents the findings for each requirement class, with one row per sampled requirement. The Gap Analysis identifies patterns in the findings (e.g., a particular subsystem with a disproportionate number of gaps). The Remediation Recommendations specifies the remediation actions, owners, and SLAs for each finding. The Sign-off records the auditor's name, the audit date, the audit ID, and the signatures of the auditor and the CSA.

The full template, with placeholder fields, is provided in Appendix E of this Matrix. The auditor SHALL complete the template in full; an incomplete report is a defect in the audit process itself.

14.7 Audit Finding Severity

Every audit finding is classified by severity. The severity classification determines the remediation SLA and the escalation path. The classification is the auditor's judgment, informed by the requirement's severity (SEV-1, SEV-2, SEV-3) and by the finding's nature (gap, stale, untraced). The four severity classes are specified below.

Finding Severity	Definition	Remediation SLA	Escalation
Critical	A SEV-1 requirement is untraced, gapped, or has stale evidence; OR a forbidden operational action was detected; OR the audit log shows evidence of tampering.	Blocks release. Remediation SHALL occur before the next release. The CSA SHALL NOT override a Critical finding.	Immediate notification to the CSA and Engineering Leadership. If the finding involves scholarly-record corruption (e.g., a missing DOI, a tampered audit log), the CSA SHALL notify the Editorial Board.
Major	A SEV-2 requirement is untraced, gapped, or has stale evidence; OR a reconciliation job has failed for >48 hours; OR an integrity rule (IR-01 through IR-08) is violated.	30 calendar days. The owning team SHALL remediate within 30 days; the next quarterly audit SHALL verify closure.	Notification to Engineering Leadership within 5 business days. If not remediated within 30 days, the finding is escalated to Critical.
Minor	A SEV-3 requirement is untraced, gapped, or has stale evidence; OR a	90 calendar days. The owning team SHALL remediate within 90 days;	Notification to the owning team. If not remediated within 90 days, the finding is

	reconciliation job has failed for <48 hours; OR a documentation defect in the Matrix (e.g., a typo in a requirement statement).	the next quarterly audit SHALL verify closure.	escalated to Major.
Observation	Not a defect, but a recommendation for improvement. For example: an evidence artifact that is current but could be more persuasive; a test that passes but does not exercise the requirement's failure mode; a monitor that fires correctly but whose alert text could be clearer.	No SLA. The recommendation is tracked in the engineering issue tracker and prioritized by the owning team.	No escalation. The recommendation is reviewed at the next quarterly audit.

The severity classification is normative: the auditor SHALL classify every finding into exactly one severity class, and the remediation SLA and escalation path SHALL follow the classification. A finding that is misclassified (e.g., a Critical finding classified as Major) is itself a defect in the audit process. The CSA SHALL review the severity classification of every Critical and Major finding before the audit report is filed.

14.8 Audit Evidence Retention

Audit reports and the underlying audit evidence are retained for the lifetime of the platform plus seven years. The retention period reflects the long-tail liability of scholarly-publishing compliance: a scholarly-record corruption incident may be discovered years after it occurs (a citation that resolves to a missing DOI, a retraction that was not propagated to indexers, a preservation copy that diverges from the canonical copy), and the audit reports are the evidence that the platform was operating in compliance at the time of the incident.

Audit reports are stored in the certification repository under AUDIT-* IDs, with access restricted to the CSA, Engineering Leadership, the Release Certification Authority, and external auditors under a documented access-control regime. Audit evidence (the EVID-* artifacts referenced by the audit) is stored in the evidence repository under its existing retention regime (per the evidence class's maximum age; Chapter 13.5), but the audit report's references to the evidence are preserved for the full retention period even after the underlying evidence has been refreshed. This ensures that an auditor reviewing a historical audit can reconstruct what was reviewed, even if the underlying evidence has since been superseded.

The retention policy is operationalized by the Archive Service (Vol II Ch 36, Evolution Governance). On platform sunset (the 'Archived' state in the amendment lifecycle), the audit reports are transferred to the long-term preservation service alongside the scholarly record itself, ensuring that the audit trail outlives the platform. The CSA SHALL verify the transfer at sunset.

Chapter 15 — Counterarguments and Limitations

15.1 Purpose

This chapter acknowledges the Matrix's limits honestly and addresses the principal counterarguments that a skeptic might raise against the Matrix's premise. The Matrix's premise is that traceability is the audit spine of the platform and that an untraceable requirement is, for audit purposes, an unmet requirement. This premise is strong, and a strong premise deserves an honest reckoning with its limits and with the arguments against it. This chapter provides that reckoning.

The chapter is structured as four limitations (Sections 15.2 through 15.5), three counterarguments (Sections 15.6 through 15.8), and a boundary statement (Section 15.9). The limitations acknowledge what the Matrix does not do; the counterarguments address what the Matrix is argued to do badly; the boundary statement specifies where the Matrix does not apply. The chapter is normative in its limits: the limitations are not flaws to be fixed in a future version, they are scoping decisions that define the Matrix's scope. The counterarguments are addressed because the Matrix's credibility depends on its being defensible against the obvious objections.

15.2 Limitation: Matrix Scope

The Matrix traces requirement classes and key requirements within each class, not every sub-requirement of every constitutional subsection. A requirement that is an aspect of a traced parent requirement is traced through the parent. For example, the Matrix traces the canonical field “article.doi” in detail (Ch 7), but it does not separately trace the sub-requirements that the DOI field SHALL be a string, SHALL match the Crossref DOI regex, SHALL be unique across the platform, SHALL be immutable once assigned, SHALL be resolvable, and SHALL be deposited with Crossref within 24 hours of publication. These sub-requirements are traced through the parent “article.doi” row, which itself traces to the DOI service architecture, the Crossref deposit workflow, the DOI-resolution test, and the Crossref deposit evidence.

This scoping means that some fine-grained requirements are not independently traceable in this Matrix. An auditor who requires row-granularity traceability for a sub-requirement that is traced only through its parent SHALL consult the parent's chain and verify that the parent's evidence demonstrates the sub-requirement's satisfaction. The Matrix does not pretend to trace every leaf requirement; it traces every requirement class and every independently verifiable requirement within each class. The total traced requirement count exceeds five hundred; the total leaf requirement count (including all sub-requirements of all traced parents) is several thousand. The Matrix's scope is the former; the latter is the responsibility of the implementation's test suite, which the Matrix verifies exists and is current.

This scoping decision is a deliberate trade-off. A Matrix that traced every leaf requirement would be longer than the Constitution itself, would be unmaintainable, and would duplicate the test suite's catalogue. The Matrix's value is in its traceability chain, not in its enumeration of every leaf requirement;

the chain is what makes the platform auditable, and the chain is what this Matrix documents. The trade-off is acknowledged here so that auditors and engineers calibrate their expectations: the Matrix is a traceability spine, not a complete requirements catalogue.

15.3 Limitation: Evidence Currency

The Matrix documents evidence at a point in time. The evidence referenced in the Matrix (EVID-* IDs, with their timestamps) was current at the time the Matrix was last updated, but the evidence may be stale by the time an auditor reads it. The maximum evidence age schedule (Chapter 13.5) is designed to bound this staleness: evidence older than the maximum age is flagged 'STALE' in the Matrix, and the affected requirement's Status is downgraded. But the schedule is enforced by the stale-evidence detection regime (Chapter 13.4), which runs on a cadence; between scans, evidence may become stale without the Matrix reflecting it.

The auditor SHALL therefore verify evidence currency independently. For each EVID-* referenced in the Matrix, the auditor SHALL retrieve the evidence artifact, compute its age, and verify the age is within the maximum for its class. The Matrix's 'Traced' status is the Matrix's claim; the auditor's verification is the audit's finding. The two may differ, and the difference is itself a finding (a Major finding if the difference indicates the stale-evidence detection regime failed to catch a stale evidence artifact).

This limitation is intrinsic to any evidence-based audit regime and is not specific to this Matrix. The Matrix mitigates the limitation by (a) bounding the maximum staleness via the schedule, (b) running the stale-evidence detection regime on a cadence tighter than the maximum age, and (c) requiring the auditor to verify currency independently. The mitigation does not eliminate the limitation; it reduces the probability that an auditor encounters stale evidence to a level that is acceptable for the platform's audit posture.

15.4 Limitation: Implementation Drift

The Matrix is reconciled with the implementation on a quarterly cadence (the Matrix-vs-implementation reconciliation, Chapter 13.6), with nightly reconciliation for the API and code columns (via CI gates). Between reconciliations, the implementation may drift from the Matrix: a code component may be renamed, an API endpoint may be deprecated, a database table may be added. Drift is a defect, but it is a defect that the reconciliation cadence is designed to detect. The nightly CI gates catch the most volatile drift (API endpoints with no tests, code components with no architecture link); the quarterly reconciliation catches the slower drift (renames, deprecations, schema changes).

The auditor SHALL be aware that the Matrix reflects the implementation as of the most recent reconciliation, not as of the audit date. For audit-critical findings, the auditor SHALL verify the implementation matches the Matrix by inspecting the actual code, schema, or configuration, not by trusting the Matrix's documentation. The Matrix's value is in its chain, not in its real-time accuracy; the chain is what the auditor traverses, and the auditor's verification of the chain's links against the actual implementation is what makes the audit meaningful.

Drift is more likely in periods of active development (e.g., the weeks before an RC milestone) and less likely in periods of stabilization (e.g., the weeks after a release). The auditor SHALL consider the development context when assessing drift risk. The Engineering Leadership Team SHALL prioritize the reconciliation cadence during periods of active development, and SHALL execute an out-of-cycle reconciliation before any external compliance audit.

15.5 Limitation: Automation Maturity

Not every traceability check is automated. The integrity rules (Chapter 13.2) are enforced by a mix of automated scans (nightly graph scans, CI gates, weekly stale-evidence scans) and manual checks (the Constitution-vs-Matrix reconciliation on amendment, the RC milestone audit, the external compliance audit). The orphan-node detection regime (Chapter 13.3) is fully automated for the nightly-cadence node types and partially manual for the weekly and per-release node types. The stale-evidence detection regime (Chapter 13.4) is fully automated for the invariant-monitor and dashboard-screenshot classes and partially manual for the reconciliation-report and certification-artifact classes.

Automation debt is tracked in the engineering issue tracker and prioritized by the SRE team. The goal is full automation of every integrity rule, every orphan-detection cadence, and every stale-evidence class, but the goal is not yet met as of RC1. The Matrix documents the current automation state honestly: each integrity rule and each detection regime is annotated with its automation maturity (automated / partially-automated / manual) in the operational documentation that accompanies this Matrix. The auditor SHALL be aware of the automation maturity when assessing the reliability of the Matrix's claims.

The automation roadmap is to achieve full automation of every integrity rule by RC2, full automation of every orphan-detection cadence by RC3, and full automation of every stale-evidence class by GA. These targets are tracked as engineering objectives and are reviewed at every RC milestone. The Matrix is a living document; its automation maturity will improve over time, and the improvements will be reflected in subsequent versions of this chapter.

15.6 Counterargument: “Traceability Is Overhead”

The argument: maintaining a traceability matrix is expensive engineering overhead. The Matrix requires engineers to cite requirement IDs in their PRs, requires reviewers to verify the chain is complete, requires the Release Certification Authority to verify every SEV-1 chain before sign-off, requires quarterly integrity checks, and requires external audits. The cumulative cost of this overhead is substantial, and the argument is that the cost is not justified by the benefit, especially for a platform of Opus Publica's scale.

The counter: the cost of traceability is less than the cost of an unauditable system. An unauditable system is a system in which incidents cannot be root-caused (because the chain from symptom to requirement to implementation to evidence is broken), in which compliance failures cannot be prevented (because the platform cannot demonstrate compliance to a funder or an indexer), and in which the scholarly record is at risk of corruption (because the platform cannot verify that every

published article has a deposited DOI, a preserved copy, and an immutable canonical version). The cost of these failures — measured in incident response hours, lost funder compliance, lost indexer eligibility, and the reputational damage of scholarly-record corruption — exceeds the cost of traceability by an order of magnitude.

The scholarly-publishing industry's history supports this counter. Incidents of scholarly-record corruption — DOIs that resolve to 404s, articles silently modified post-publication, retraction notices not propagated to indexers, preservation copies that diverge from the canonical copy — have cost the affected platforms substantial engineering hours to remediate, substantial reputational damage, and, in several cases, the loss of indexer eligibility. Each of these incidents would have been preventable, or at least detectable-earlier, by a traceability matrix that verified the DOI deposit, the immutability, the retraction propagation, and the preservation copy. The Matrix is the platform's insurance against these incidents; the cost of the insurance is less than the cost of the incidents it prevents.

15.7 Counterargument: “The Constitution Already Specifies This”

The argument: the Engineering Constitution (Vol I + Vol II) already specifies every requirement the Matrix traces. The Matrix duplicates the Constitution's specification, and the duplication is a maintenance burden: when the Constitution is amended, the Matrix must be amended in parallel, and any divergence between the two is a defect. The argument is that the Matrix should be eliminated and the Constitution should be the sole document.

The counter: the Constitution and the Matrix are different documents with different purposes. The Constitution specifies requirements; the Matrix traces their implementation. The Constitution answers “what must the platform do?”; the Matrix answers “does the platform do it, and can we prove it?” These are different questions, and they require different documents. The Constitution is a specification document; the Matrix is a traceability document. The Constitution's audience is engineers who need to know the contract; the Matrix's audience is auditors who need to verify the contract is met.

The duplication the argument identifies is not real duplication. The Matrix does not re-specify the requirements; it cites them (by requirement ID) and traces them (by chain). The Matrix's requirement statements are one-line summaries that point to the Constitution's full specification. The Matrix does not duplicate the Constitution's prose; it indexes it. The maintenance burden of keeping the Matrix aligned with the Constitution is the Constitution-vs-Matrix reconciliation (Chapter 13.6), which is automated and runs on every amendment. The burden is real but small, and it is the price of auditability. The alternative — no Matrix, no traceability — is the unauditable system described in Section 15.6, whose cost is higher.

15.8 Counterargument: “Runtime Evidence Is Sufficient”

The argument: the platform's runtime evidence — passing tests, green dashboards, monitor outputs — is sufficient for audit. An auditor who inspects the test suite and the dashboards can verify the platform is operating correctly without the overhead of a traceability chain. The argument is that the chain is an unnecessary indirection between the auditor and the evidence.

The counter: runtime evidence without a traceability chain is evidence without context. An auditor who sees a passing test does not know which requirement the test verifies without the chain. An auditor who sees a green dashboard does not know which SLO the dashboard represents, which requirement the SLO binds, or which invariant the SLO protects, without the chain. The chain is what gives the evidence its meaning: it connects the evidence to the requirement it demonstrates. Without the chain, the auditor is reduced to spot-checking tests and dashboards without a framework for assessing coverage, completeness, or materiality.

The chain also enables the auditor to ask the question that runtime evidence alone cannot answer: “which requirements have no evidence?” This is the orphan-requirement question (Integrity Rule IR-01), and it is the question that the chain is designed to answer. A platform with comprehensive runtime evidence but no traceability chain cannot answer this question; the platform does not know which of its requirements are verified and which are not. The Matrix answers the question, and the answer is the Matrix's value. The runtime evidence is the raw material; the chain is the structure that turns the raw material into an auditable record.

15.9 When the Matrix Does Not Apply

The Matrix does not govern editorial decisions, scholarly content, product strategy, or commercial policy. It governs the engineering execution of those domains. The boundary is important because the Matrix's authority is engineering authority, not editorial or commercial authority, and the Matrix SHALL NOT be used to override editorial or commercial decisions.

Specifically, the Matrix does not govern: the acceptance or rejection of a manuscript (an editorial decision governed by the peer-review policy and the editorial board's authority); the scholarly content of a published article (governed by the author's copyright and the CC BY 4.0 license); the product strategy of Opus Publica (governed by the editorial board and the executive team); the commercial policy of Opus Publica (governed by the executive team and the AUN parent organization). The Matrix governs the engineering execution: the platform that hosts the manuscript, the workflow that manages peer review, the storage that preserves the published article, the DOI minting that registers the article, the Crossref deposit that propagates the metadata, the OAI-PMH endpoint that exposes the metadata to indexers, the preservation service that ensures long-term access. The Matrix's scope is the engineering execution; everything else is outside the Matrix's scope.

This boundary is not a limitation of the Matrix; it is a scoping decision that defines the Matrix's authority. The Matrix is the audit spine of the engineering platform; it is not the audit spine of the editorial process, the scholarly record's content, or the commercial business. Each of those domains has its own governance, and the Matrix's authority does not extend to them. The Matrix's authority is engineering authority, and the Matrix is complete within its scope.

Chapter 16 — Conclusion and Implications

16.1 Summary of Findings

The Traceability Matrix traces more than five hundred constitutional requirements across nine requirement classes: the 38 engineering invariants (Ch 4), the 47 non-negotiable rules (Ch 5), the 20 system contracts (Ch 6), the 50 key canonical fields of the 276-field canonical object registry (Ch 7), the 34 events (Ch 8), the 10 SLOs (Ch 9), the key thresholds of the 103-threshold engineering mathematics (Ch 10), the 25 Volume I chapters (Ch 11), and the 11 Volume II chapters (Ch 12). The traceability chain is the thirteen-node sequence from Requirement to Release established by Volume II Chapter 23. The integrity rules are eight (IR-01 through IR-08). The audit procedure is the twelve-step standard procedure specified in Chapter 14. The Matrix is the audit spine of Opus Publica.

The Matrix's findings are: the platform's constitutional requirements are systematically traceable; the traceability chain is defined and documented; the integrity rules are specified and enforced; the audit procedure is standardized and repeatable; the evidence regime is bounded by a maximum evidence age schedule; the reconciliation cadence binds the Matrix to five sources of truth (Constitution, implementation, evidence repository, Crossref, preservation service). The Matrix is complete in the sense that every requirement class is traced; it is incomplete in the sense that some leaf requirements are traced only through their parents (Chapter 15.2). The Matrix is a living document, maintained as long as the platform exists.

16.2 The State of RC1 Traceability

As of the RC1 release, the state of traceability is the following. All 38 engineering invariants are traced, with monitors operational or planned for each (the invariant monitors are the platform's first line of defense against scholarly-record corruption). All 47 non-negotiable rules are traced. All 20 system contracts are traced. The 34 events are traced. The 10 SLOs are traced. The 103 thresholds are traced, with a key subset traced in detail in Chapter 10. The 25 Volume I chapters are traced. The 11 Volume II chapters are traced. The total traced requirement count exceeds five hundred; the total leaf requirement count (including sub-requirements traced through their parents) is several thousand.

Known gaps are documented in the Matrix's Status column (cells marked 'GAP' or 'STALE') and have remediation plans with owners and SLAs. The number of known gaps as of RC1 is small — fewer than ten, all SEV-3 — and all are scheduled for remediation before the RC2 milestone. There are zero known gaps in SEV-1 requirements (Integrity Rule IR-05 is satisfied). There are zero known gaps in SEV-2 requirements as of RC1. The Release Certification Authority SHALL verify, before signing off on RC1, that the gap count is consistent with this chapter's statement and that all SEV-1 and SEV-2 requirements are fully traced.

16.3 Implications for Engineering Practice

The Matrix changes how engineers work. Every feature implementation cites a requirement ID; every pull request traces its implementation through the chain; every certification verifies the chain; every audit samples the chain. The Matrix makes traceability a daily discipline, not an annual exercise. The engineer who implements a feature knows the requirement the feature satisfies; the reviewer who reviews the pull request verifies the chain is complete; the certifier who certifies the release verifies every SEV-1 chain; the auditor who audits the platform samples the chain and verifies the evidence. Traceability is no longer an afterthought; it is the structure within which engineering work is done.

The implication for engineering practice is that the engineer's daily workflow now includes traceability steps: citing the requirement ID in the commit message; linking the pull request to the requirement in the Matrix; verifying the chain's links are populated before requesting review; responding to the reviewer's traceability checklist; producing the runtime evidence the chain expects. These steps are overhead in the sense that they are not implementation work, but they are the overhead that makes the implementation auditable. The Engineering Playbook (Chapters 3 through 7) operationalizes these steps as checklists and gates; the Matrix provides the structure the checklists reference.

16.4 Implications for Audit and Compliance

The Matrix makes Opus Publica auditable by design. An external auditor can verify any requirement's satisfaction by traversing the chain: the auditor reads the requirement in the Constitution, finds the requirement's row in the Matrix, traverses the chain from Requirement through Architecture, ADR, Database, API, Code, Storage, Workflow, UI, Tests, Evidence, Certification, Release, and verifies the evidence is current and persuasive. The chain is self-explanatory; the auditor does not require Opus Publica engineers to interpret the Matrix. This positions the platform for funder compliance (Plan S, COPE, DOAJ), indexer eligibility (DOAJ, Scopus, Web of Science), and preservation-service acceptance (CLOCKSS, LOCKSS, Portico).

The implication for audit and compliance is that the platform's compliance posture is no longer a matter of assertion; it is a matter of demonstration. The platform does not assert that every published article has a DOI; the Matrix demonstrates it, by tracing the DOI-minting requirement to the DOI service, the Crossref-deposit workflow, the deposit-receipt evidence, and the Crossref-vs-Matrix reconciliation. The platform does not assert that every published article is preserved; the Matrix demonstrates it, by tracing the preservation requirement to the preservation service, the preservation-receipt evidence, and the preservation-vs-Matrix reconciliation. The demonstration is what the auditor verifies; the Matrix is what makes the demonstration possible.

16.5 Future Outlook

The Matrix will evolve. As the Constitution evolves (RC2, RC3, GA), new requirements are added by amendment and SHALL be traced before the next release (Integrity Rule IR-08). As automation matures, more integrity-rule checks, orphan-detection cadences, and stale-evidence classes will become fully

automated (Chapter 15.5). As the platform scales — more journals, more articles, more deposits, more preservation copies — the Matrix scales with it: the chain does not change; only the number of rows increases. The Matrix is a living document, maintained as long as the platform exists.

The Matrix's evolution is governed by the same amendment lifecycle that governs the Constitution (Vol II Ch 36). Changes to the Matrix's structure (e.g., adding a new node type, adding a new edge type, adding a new integrity rule) are amendments and SHALL follow the amendment lifecycle. Changes to the Matrix's content (e.g., adding a row, refreshing an evidence artifact, updating a Status column) are operational updates and SHALL follow the Matrix's maintenance cadence (Chapter 3.6). The distinction between structural amendments and operational updates is the distinction between changing the Matrix's shape and changing the Matrix's data; both are governed, but by different processes.

The Matrix's long-term outlook is to be the platform's permanent audit record. The Matrix at platform sunset (the 'Archived' state in the amendment lifecycle) SHALL be transferred to the long-term preservation service alongside the scholarly record itself, ensuring that the audit trail outlives the platform. The Matrix is not just a document for the present; it is a document for the future, and the future includes the platform's eventual sunset.

16.6 Call to Action

The Engineering Leadership Team SHALL take the following four actions to operationalize the Matrix's maintenance and to close out the RC1 release. These actions are normative: each action SHALL be completed, and the completion SHALL be verified by the Chief Systems Architect.

9. **Action 1.** Operationalize the Matrix's maintenance cadence. The Engineering Leadership Team SHALL staff the Graph Steward role (the role accountable for the Matrix's day-to-day maintenance), SHALL schedule the nightly, weekly, and quarterly reconciliation jobs, and SHALL add the Matrix's dashboards (orphan-detection, stale-evidence, reconciliation) to the weekly engineering review.
10. **Action 2.** Remediate all known gaps before RC1 release. The Engineering Leadership Team SHALL assign owners to every 'GAP' and 'STALE' cell in the Matrix, SHALL track the remediation to closure in the engineering issue tracker, and SHALL verify closure before the RC1 sign-off. No SEV-1 or SEV-2 gap SHALL remain at RC1 sign-off.
11. **Action 3.** Execute the first quarterly integrity check. The Engineering Leadership Team SHALL execute the first quarterly integrity check (the five reconciliation types of Chapter 13.6) within thirty days of the RC1 release, SHALL document the findings in an internal audit report, and SHALL remediate any findings per the SLAs in Chapter 14.7.
12. **Action 4.** Commission the first external audit. The Chief Systems Architect SHALL commission the first external compliance audit within ninety days of the RC1 release, SHALL select an auditor with scholarly-publishing compliance expertise, and SHALL publish the audit report to the certification repository upon completion. The external audit is the platform's first independent verification of its compliance posture.

The Matrix is complete; its maintenance is the ongoing discipline. The Matrix's value is realized not at the moment of its authoring but at the moment of its first audit, its first incident, its first funder-compliance submission, its first indexer-eligibility application. At each of those moments, the Matrix is the document the platform relies on; at each of those moments, the Matrix's maintenance discipline is what makes the Matrix trustworthy. The discipline begins now.

This Matrix is the closing artifact of the Opus Publica RC1 engineering documentation set. With the Constitution (Volume I and Volume II) establishing the specification, the Engineering Playbook operationalizing the specification, and this Matrix tracing the operationalization through to runtime evidence, the engineering organization has a complete, end-to-end audit spine. The Constitution specifies what the platform must do; the Playbook specifies how the platform does it; the Matrix proves that the platform does it, with evidence that an auditor can inspect. The three documents together constitute the platform's engineering authority, and the Matrix is the document that closes the loop between specification and evidence.

TABLE OF CONTENTS

Right-click the table below and select "Update Field" to refresh page numbers.

Appendices

The following appendices provide consolidated reference material for the Traceability Matrix: a glossary of traceability terms (Appendix A); the consolidated thirteen-node-type reference table (Appendix B); the consolidated twelve-edge-type reference table (Appendix C); the requirement ID scheme (Appendix D); the audit report template (Appendix E); the document control block and revision history (Appendix F); and the references (Appendix G).

The appendices are normative where they restate material from the Constitution or from this Matrix's body chapters; they are informative where they provide supplementary reference material. An auditor SHALL consult the appendices as the definitive reference for terms, node types, edge types, ID schemes, and the audit report template. The document control block (Appendix F) is the authoritative record of the Matrix's version, owner, review cadence, and revision history; the revision history SHALL be updated on every revision to the Matrix, whether the revision is a structural amendment or an operational update.

Appendix A — Glossary of Traceability Terms

The following terms are used throughout this Matrix with the precise meanings given here. The glossary extends the definitions in Chapter 1.5 and the Constitution's glossary (Vol II, Appendix B).

Term	Definition
Requirement	A constitutional statement that obliges the platform to behave in a specified way. Requirements are identified by stable IDs (e.g., INV-I-01, PUB-001, ARCH-003) and are classified by severity (SEV-1 through SEV-3).
Requirement Class	A category of requirements established by a single Constitution chapter or chapter group (e.g., engineering invariants, non-negotiable rules, system contracts). The Matrix traces nine requirement classes.
Traceability Chain	The thirteen-node sequence from Requirement to Release (Figure 1.1) that documents the path from specification to evidence. Every requirement SHALL have a traceability chain.
Traceability Link	A single edge in the traceability graph, connecting two nodes (e.g., Requirement → Architecture via the 'realizes' edge type).
Traceability Gap	A missing link in a traceability chain. A gap means the requirement's path to evidence is broken. Gaps are remediated under SLAs determined by the requirement's severity.

Orphan Node	A node in the traceability graph with no incoming or outgoing edges. An orphan requirement is a requirement no architecture realizes; an orphan test is a test no requirement mandates.
Runtime Evidence	An artifact produced by the running system that demonstrates a requirement is satisfied (logs, monitor outputs, screenshots, hashes, audit records, deposit receipts).
Certification Artifact	A runtime evidence artifact that has been reviewed and signed off by the Release Certification Authority. Certification artifacts are bound to releases.
Audit Trail	The complete set of traceability chains for a given requirement class, sufficient for an external auditor to verify compliance.
Node Type	One of the thirteen categories of nodes in the traceability graph (Requirement, Architecture Component, ADR, Database Table, API Endpoint, Code Component, Storage Object, Workflow, UI Component, Test, Runtime Evidence, Certification Record, Release Record).
Edge Type	One of the twelve categories of edges in the traceability graph (realizes, justified-by, backed-by, exposed-by, implemented-by, stored-in, exercised-by, surfaced-by, verified-by, produces, certified-by, released-in).
Integrity Rule	One of the eight rules (IR-01 through IR-08) that the traceability graph SHALL satisfy. Integrity rules are the operational form of Vol II Ch 23 §23.7.
Reconciliation	The process of verifying that the traceability graph matches a source of truth (Constitution, implementation, evidence repository, Crossref, preservation service). Reconciliation failures are defects.
Stale Evidence	Runtime evidence whose age exceeds the maximum evidence age for its class (Chapter 13.5). Stale evidence is treated as missing evidence.
Maximum Evidence Age	The maximum acceptable age for an evidence artifact of a given class, beyond which the evidence is stale. The schedule is specified in Chapter 13.5.
Status (Traced / Gap / Stale / Untraced)	The audit-critical column of every matrix row. 'Traced': chain complete, evidence current. 'Gap': one or more links missing. 'Stale': chain complete but evidence stale. 'Untraced': requirement not in Matrix.
Severity (SEV-1 / SEV-2 / SEV-3)	The requirement's criticality classification. SEV-1:

	scholarly-record corruption risk; SEV-2: significant operational impact; SEV-3: minor impact. SEV-1 requirements require complete chains (IR-05).
Audit Finding	The auditor's verdict on a sampled requirement: Traced, Gap, Stale, or Untraced. Findings are classified by severity (Critical, Major, Minor, Observation) per Chapter 14.7.
Graph Steward	The role accountable for the traceability graph's day-to-day maintenance: monitoring the orphan-detection and stale-evidence dashboards, executing reconciliation jobs, remediating low-severity defects.
Release Certification Authority (RCA)	The role accountable for certifying releases. The RCA verifies that every SEV-1 requirement has a complete, current traceability chain before sign-off.
Constitution Amendment	A change to the Engineering Constitution, governed by the ten-state amendment lifecycle (Vol II Ch 36). Amendments trigger Constitution-vs-Matrix reconciliation (IR-08).

Appendix B — The 13 Node Types (Consolidated Reference)

The traceability graph has thirteen node types, corresponding to the thirteen stages of the traceability chain. The table below consolidates the node-type reference from Chapter 2.2; it is the definitive reference for node identifiers, owning chapters, and mandatory attributes.

#	Node Type	ID Format	Owning Chapter	Mandatory Attributes
N1	Requirement	REQ-<class>-<nnn>	Vol I/II various	ID, statement, source chapter, owner, severity
N2	Architecture Component	ARCH-<subsystem>	Vol I Ch 4	ID, name, owning team, interfaces, dependencies
N3	ADR	ADR-<nnn>	Vol I Ch 7	ID, title, status, decision, trade-offs, owner
N4	Database Table	DB-<table>	Vol I Ch 5, Vol II Ch 33	Table name, owning service, schema, indexes
N5	API Endpoint	API-<method>-<path>	Vol I Ch 9	Method, path, auth, idempotency,

				contract, owner
N6	Code Component	CODE-<module>	Vol I Ch 8	Module path, owning team, language, dependencies
N7	Storage Object	STORE-<bucket>/<key>	Vol I Ch 12, Vol II Ch 12	Bucket, key, SHA-256, retention, owner
N8	Workflow	WF-<name>	Vol I Ch 10, Vol II Ch 11	Name, states, transitions, owner
N9	UI Component	UI-<component>	Vol II Ch 30	Component name, page, accessibility, owner
N10	Test	TEST-<type>-<nnn>	Vol I Ch 19, Vol II Ch 19	ID, type, target, coverage, owner
N11	Runtime Evidence	EVID-<nnn>	Vol I Ch 8, Vol II Ch 20	ID, type, timestamp, source, hash
N12	Certification Record	CERT-<nnn>	Vol I Ch 8, Vol II Ch 22	ID, feature, pillars, RCA, date
N13	Release Record	REL-<nnn>	Vol I Ch 5.07, Playbook Ch 11	ID, version, artifacts, date, status

Appendix C — The 12 Edge Types (Consolidated Reference)

Edges in the traceability graph represent the relationships between nodes. The table below consolidates the edge-type reference from Chapter 2.3; it is the definitive reference for edge semantics, direction, and cardinality.

Edge Type	From → To	Semantics	Cardinality
realizes	Requirement → Architecture	The architecture component realizes the requirement.	Many-to-many.
justified-by	Architecture → ADR	The architecture decision is justified by the ADR.	Many-to-one (a component may be justified by one ADR; an ADR may justify multiple components).
backed-by	Architecture → Database	The architecture is backed by the database table.	Many-to-many.
exposed-by	Architecture → API	The architecture is exposed by the API endpoint.	Many-to-many.

implemented-by	Architecture → Code	The architecture is implemented by the code component.	Many-to-many.
stored-in	Code → Storage	The code stores artifacts in the storage object.	Many-to-many.
exercised-by	Workflow → Code	The workflow exercises the code component.	Many-to-many.
surfaced-by	Workflow → UI	The workflow is surfaced by the UI component.	Many-to-one.
verified-by	Requirement → Test	The requirement is verified by the test.	Many-to-many.
produces	Test → Runtime Evidence	The test produces the runtime evidence.	One-to-many (a test may produce multiple evidence artifacts).
certified-by	Runtime Evidence → Certification	The evidence is certified by the certification record.	Many-to-one (multiple evidence artifacts are bundled into one certification record).
released-in	Certification → Release	The certification authorizes the release.	One-to-one (a certification record authorizes exactly one release).

Appendix D — Requirement ID Scheme

Every requirement in the Matrix has a stable identifier. The ID scheme is uniform across requirement classes: a class prefix, a sub-class prefix (where applicable), and a zero-padded sequence number. The table below documents the ID scheme for each requirement class, with the source Constitution chapter, the requirement count, and the Matrix chapter that traces the class in detail.

Requirement Class	ID Format	Source Chapter	Count	Matrix Chapter
Engineering Invariants	INV-<sub>-<nn>	Vol II Ch 26	38	Chapter 4
Non-Negotiable Rules	PUB-<nnn>	Vol I Ch 3	47	Chapter 5
System Contracts	CONTRACT-<nn>	Vol II Ch 27	20	Chapter 6
Canonical Fields	REG-<entity>-<field>	Vol II Ch 28	276 (50 traced in detail)	Chapter 7
Events	EVT-<domain>-<nn>	Vol II Ch 29	34	Chapter 8
SLOs	SLO-<nn>	Vol II Ch 32	10	Chapter 9

Constitutional Thresholds	THR-<category>--<nn>	Vol II Ch 31	103 (key subset traced)	Chapter 10
Volume I Chapter Requirements	CHVI-<nn>	Vol I Ch 1-25	25 chapters	Chapter 11
Volume II Chapter Requirements	CHVII-<nn>	Vol II Ch 26-36	11 chapters	Chapter 12
Operational Governance Actions	GOV-<nn>	Vol II Ch 34	14	Chapter 12 (row-level: PB Ch 14)
Evolution Governance States	EVO-<state>	Vol II Ch 36	10	Chapter 12 (row-level: PB Ch 18)
Database Constitution Sections	DB-<section>--<nn>	Vol II Ch 33	14	Chapter 12 (row-level: PB Ch 14)
Cost Categories	ECON-<nn>	Vol II Ch 35	13	Chapter 12 (row-level: PB Ch 14)
UI Components	UI-<component>	Vol II Ch 30	11	Chapter 12 (row-level: PB Ch 10)
Architecture Decisions	ADR-<nnn>	Vol I Ch 7	~30	Referenced throughout
Tests	TEST-<type>--<nnn>	Vol I Ch 19, Vol II Ch 19	~500+	Referenced throughout
Runtime Evidence	EVID-<nnn>	Vol I Ch 8, Vol II Ch 20	Per release	Referenced throughout
Certification Records	CERT-<nnn>	Vol I Ch 8, Vol II Ch 22	Per release	Referenced throughout
Release Records	REL-<nnn>	Vol I Ch 5.07, PB Ch 11	Per release	Referenced throughout
Audit Reports	AUDIT-<nnn>	This Matrix (Ch 14)	Per audit	Chapter 14

Appendix E — Audit Report Template

Every audit report SHALL follow the standard template below. The auditor SHALL complete every field; an incomplete report is a defect in the audit process. Placeholder fields are indicated in angle brackets (“<placeholder>”) and SHALL be replaced with the audit's actual content.

```

=====
=====
OPUS    PUBLICA    RC1    TRACEABILITY    MATRIX    —    AUDIT    REPORT
=====
=====

```

AUDIT ID: AUDIT-**<nnn>**
 AUDIT TYPE: **<Internal Quarterly | RC Milestone | External Compliance | Funder-Mandated | Incident-Triggered>**
 AUDITOR: **<name, role, organization>**
 AUDIT DATES: **<start-date> through <end-date>**
 RELEASE UNDER AUDIT: **<REL-nnn> (version <x.y.z>, date <date>)**
 MATRIX VERSION: **<Matrix version under audit>**
 REPORT DATE: **<YYYY-MM-DD>**
 CLASSIFICATION: **<Mandatory — Audit Authority | Confidential — Engineering Leadership | Confidential — Funder>**

1. EXECUTIVE SUMMARY

<One-page summary: audit scope, sample size, finding counts by severity, headline conclusions, recommendation (sign-off / block with conditions / block without conditions). Suitable for reading by the CSA and Engineering Leadership in under five minutes.>

2. SCOPE

2.1 Requirement classes audited:
<e.g., Engineering Invariants (INV-*), System Contracts (SLO-*)>
 2.2 Sample size: **<total requirements traversed>**
 2.3 Sampling method: **<per Ch 14.5; include random seed if random sampling was used>**
 2.4 Audit type: **<per Ch 14.3>**
 2.5 Auditor: **<name, role, organization>**

3. METHODOLOGY

3.1 Procedure followed: **Standard Audit Procedure (Ch 14.4)**
 3.2 Deviations from standard procedure: **<list or 'None'>**
 3.3 Tools used: **<e.g., Matrix v<x.y>, evidence repository, Crossref REST API, preservation service API>**

4. FINDINGS BY CLASS

<For each requirement class, a sub-section with one row per sampled requirement. Columns: Req ID | Severity | Chain Status | Evidence Status | Finding (Traced/Gap/Stale/Untraced) |

Finding	Severity	(Critical/Major/Minor/Observation)					Notes.>
4.1	Engineering	Invariants					(INV-*)
4.2	Non-Negotiable	Rules					<rows> (PUB-*)
...	(one	sub-section	per	audited		<rows> class)	

5.	GAP				ANALYSIS		

5.1	Findings	summary:	<counts		by	severity>	
	Critical: <n>	Major: <n>	Minor: <n>	Observation: <n>			
5.2	Patterns	identified:	<e.g.,	'Subsystem X	accounts	for	
	60%	of	Major	findings';	'Evidence-class Y	has	
5.3	Comparison	to	prior	audit:	<trends,	improvements,	
						regressions>	

6.	REMEDIATION				RECOMMENDATIONS		

<For each finding with a remediation SLA: Req ID Finding							
Severity Recommended Remediation Owner SLA (per Ch 14.7)>							

7.	SIGN-OFF						

Auditor:			<name>	<signature>	<date>		
Chief	Systems	Architect:	<name>	<signature>	<date>		
Release	Cert.	Authority:	<name>	<signature>	<date>		

AUDIT	REPORT	RETENTION:	Lifetime	of	platform	+	7 years.
STORAGE	LOCATION:	Certification	repository,	AUDIT-<nnn>.			
ACCESS	CONTROL:	CSA,	Engineering	Leadership,	RCA,	external	
auditors	under	documented	access-control	regime.			
=====							
=====							

Appendix F — Document Control

This Matrix is a controlled document. The control block below specifies the document's identity, version, owner, review cadence, and supersession. The revision history records every revision to the Matrix since first issuance.

Field	Value
Document ID	OP-TRACE-RC1-001
Document Title	Opus Publica RC1 Constitutional Traceability Matrix
Version	Version 1.0
Issue Date	09 August 2026
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Owner	Chief Systems Architect
Classification	Mandatory — Audit Authority
Audience	Engineering Leadership, Release Certification Authority, External Auditors, Funder Compliance Auditors
Last Reviewed	09 August 2026
Next Review	Quarterly integrity check; or on every Constitution amendment (Vol II Ch 36)
Supersedes	None (first issuance)
Superseded By	Not yet issued
Operationalizes	Vol II Ch 23 (Knowledge Graph Constitution)
Retention	Lifetime of platform + 7 years (Ch 14.8)
Storage Location	Certification repository; long-term preservation service at platform sunset

Revision history: every revision to the Matrix SHALL be recorded in the table below. Revisions that change the Matrix's structure (new node types, new edge types, new integrity rules) are amendments and SHALL follow the amendment lifecycle (Vol II Ch 36); revisions that change the Matrix's content (new rows, refreshed evidence, updated Status) are operational updates and SHALL follow the Matrix's maintenance cadence (Ch 3.6).

Version	Date	Author	Summary of Changes
1.0	09 August 2026	Chief Systems Architect	First issuance. Traceability Matrix for the Opus Publica RC1 release. Sixteen chapters plus seven appendices. Traces 500+ requirements across nine requirement classes. Operationalizes Vol II Ch 23 (Knowledge Graph Constitution).

Appendix G — References

The following references are cited or implied throughout this Matrix. Constitution chapters and Playbook chapters are internal documents of the Opus Publica engineering organization; external references are publicly available standards and practices.

Constitution (Volume I):

- Vol I Ch 1 — Mission and Charter
- Vol I Ch 2 — Engineering Principles
- Vol I Ch 3 — Non-Negotiable Rules
- Vol I Ch 4 — Architecture
- Vol I Ch 5 — Subsystem Charters (5.01 through 5.12)
- Vol I Ch 6 — Repository Constitution
- Vol I Ch 7 — Architecture Decision Records
- Vol I Ch 8 — Runtime Certification
- Vol I Ch 9 — API Constitution
- Vol I Ch 10 — Workflow Constitution
- Vol I Ch 11 — Event Constitution (Volume I form)
- Vol I Ch 12 — Storage Constitution
- Vol I Ch 13-18 — Cross-Cutting Concerns (Security, Observability, Performance, Accessibility, Internationalization, Privacy)
- Vol I Ch 19 — Testing Constitution
- Vol I Ch 20-25 — Operational, Governance, Cost, Evolution, Knowledge Graph, and Conclusion chapters

Constitution (Volume II):

- Vol II Ch 26 — Engineering Invariants (38 invariants)
- Vol II Ch 27 — System Contracts (20 subsystems)
- Vol II Ch 28 — Canonical Object Registry (276 fields)
- Vol II Ch 29 — Event Constitution (34 events)
- Vol II Ch 30 — UI Constitution (11 components)
- Vol II Ch 31 — Engineering Mathematics (103 thresholds)
- Vol II Ch 32 — Production SLO Book (10 SLOs)
- Vol II Ch 33 — Database Constitution (14 sections)
- Vol II Ch 34 — Operational Governance (14 actions)
- Vol II Ch 35 — Engineering Economics (13 cost categories)
- Vol II Ch 36 — Evolution Governance (10-state lifecycle)

Engineering Playbook:

- Playbook Ch 1-2 — How to Use This Playbook; Implementation Lifecycle

- Playbook Ch 3-7 — Branching, Commits/PRs, Code Review, QA, Test Environments
- Playbook Ch 8-12 — Runtime Invariant Monitoring, Certification Process, Five Pillars, Release/Deployment, Rollback/Incident
- Playbook Ch 13-16 — On-Call, Operational Governance Execution, SLO Management, Incident Command
- Playbook Ch 17-20 — Vol I Operationalization Matrix, Vol II Operationalization Matrix, Master Checklist Catalog, Runbook Catalog

External standards and practices:

- IETF RFC 2119 — Key words for use in RFCs to Indicate Requirement Levels (March 1997). Used throughout this Matrix for normative vocabulary.
- Google SRE — Site Reliability Engineering: How Google Runs Production Systems (O'Reilly, 2016) and The Site Reliability Workbook (O'Reilly, 2018). Referenced for SLO, error-budget, and multi-window multi-burn-rate alerting practices.
- NIST SP 800-92 — Guide to Computer Security Log Management (September 2006). Referenced for audit-log management and tamper-evidence practices.
- NIST SP 800-61 — Computer Security Incident Handling Guide (Revision 2, August 2012). Referenced for incident response and postmortem practices.
- ISO 19005-2 — Document management — Electronic document file format for long-term preservation — Part 2: Use of ISO 32000-1 (PDF/A-2). Referenced for long-term preservation of the Matrix itself.
- W3C WCAG 2.2 — Web Content Accessibility Guidelines 2.2 (October 2023). Referenced for the UI Constitution's accessibility contract (AA conformance).
- OWASP ASVS 4.0 — Application Security Verification Standard (March 2021). Referenced for the security verification regime.
- Crossref — Crossref REST API and deposit documentation (api.crossref.org). Referenced for DOI deposit and reconciliation practices.
- OAI-PMH — Open Archives Initiative Protocol for Metadata Harvesting (v2.0, June 2002). Referenced for indexer exposure.
- CLOCKSS, LOCKSS, Portico — Preservation service documentation. Referenced for preservation-receipt evidence.
- DOAJ — Directory of Open Access Journals, eligibility criteria. Referenced for indexer eligibility positioning.
- COPE — Committee on Publication Ethics, core practices. Referenced for editorial-integrity compliance.

— Issued under the authority of the Chief Systems Architect, Opus Publica — Traceability Matrix complete

—